

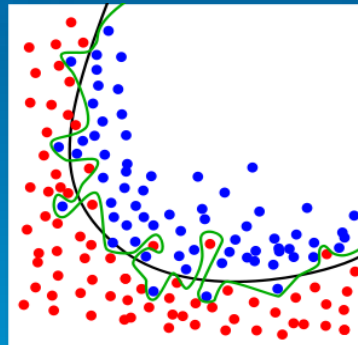
媒体与认知

Media and Cognition

——一种现代机器学习方法

第3讲 机器学习

Machine learning



清华大学电子系 王生进 李亚利

3.1 机器学习概要

3.2 感知机

3.3 回归方法与线性分类器

3.4 支持向量机SVM

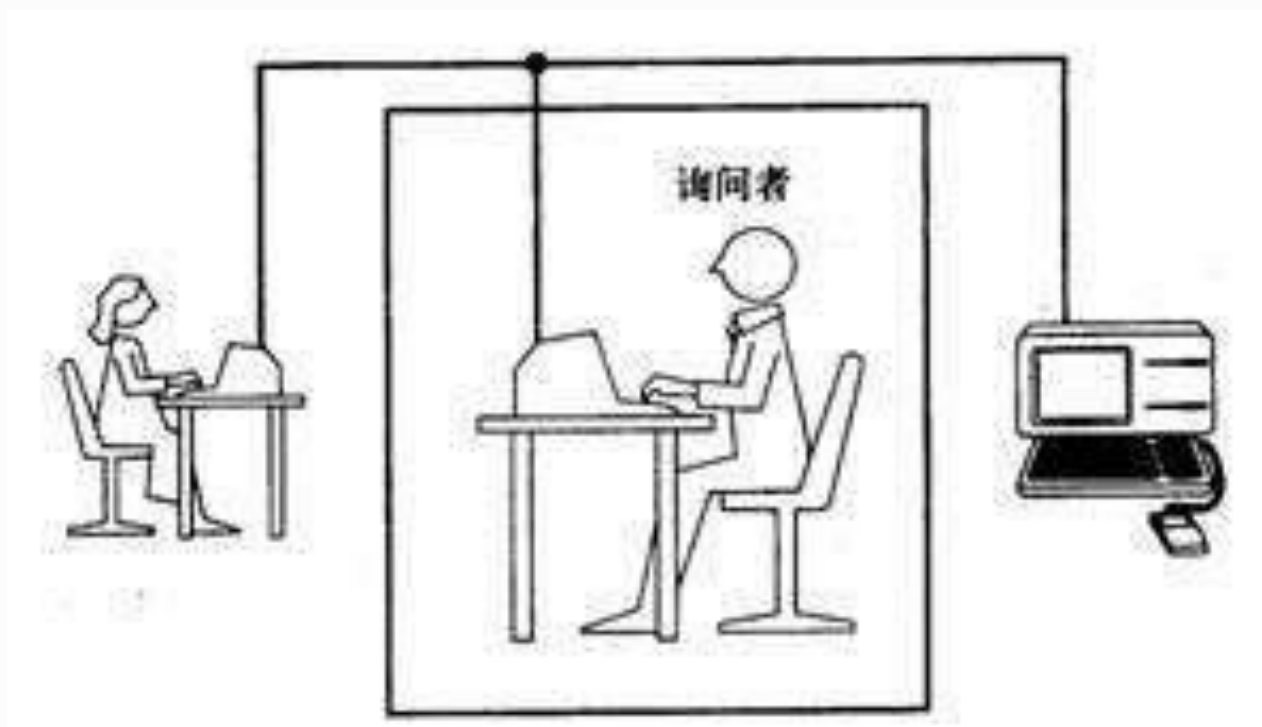
聚焦问题学习

本讲课程内容，要掌握机器学习的本质和核心，
机器学习的主要任务，
达到的主要目标，
研究的主要对象，等

- 机器学习的回归和分类任务有何区别与联系？
- 回归和分类问题的目标函数分别如何定义？

3.1 机器学习概要

机器学习经历了长期的发展。上溯到17世纪，[贝叶斯](#)、[拉普拉斯](#)关于[最小二乘法](#)的推导和[马尔可夫链](#)，构成了机器学习广泛使用的工具和基础。1950年，[图灵](#)提出建立学习机器；2000年初，深度学习提出和实际应用和进展（如2012年的AlexNet，后来的ResNet等），使得机器学习取得了巨大的进步。



3.1 机器学习概要

➤ 特别是移动互联网时代，媒体信息的感知和认知

◆ 图像

- 图像分类： 图像→类别**标号**
- 物体检测： 图像→物体**位置**

◆ 声音

- 语音识别： 语音信号→文本**序列**
- 声纹识别： 语音信号→类别**标号**

◆ 文字

- 机器翻译： 文本序列→文本**序列**

核心： 通过媒体内容的认知实现媒体形式的转换

3.1 机器学习概要—发展历程

- 机器学习是现阶段解决很多人工智能问题的主流方法，作为一个独立的方向，正处于高速发展之中。
- 最早的机器学习算法可以追溯到20世纪初，到今天为止，已经过去了100多年。
- 从1980年机器学习称为一个独立的方向开始算起，到现在也已经过去了近40年。在这100多年中，经过一代又一代人的努力，诞生出了大量经典的方法。
- 机器学习算法有很多，有分类、回归、聚类、推荐、图像识别领域等等，具体算法比如线性回归、逻辑回归、朴素贝叶斯、随机森林、支持向量机、神经网络等等。
- 在机器学习算法中，没有最好的算法，只有“更适合”解决当前任务的算法。

机器学习发展历程 road map

Probably Approximately Correct (PAC)

Deep Learning
(2006)

Breiman:
随机森林
(2001)

Hoch.等:
LSTM(1997)

LeCun:
CNN(1989)

Vapnik:
SVM (1991)

Freund:
AdaBoost
(1996)

Schapire:
弱学习定理
(1990)

Rumelhart:
BP (1986)

决策树
(1980s)

Cox:
Logistic(1958)

线性不可分问题
(Minsky 1969)

Valiant:
PAC (1984)

隐马尔可夫
模型 (1960)

Widrow:
自适应线性元件
Adaline (1960)

Thomas:
kNN (1967)

Rosenblatt:
感知机(1956)

贝叶斯分类
(1950s)

Hebb:
神经集合
体假设

Barlow:
功能单细胞
假设

James
(19世纪末):
神经元相互连接
McCulloch, Pitts
(20世纪中期):
“兴奋”和“抑制”
Hebb
(20世纪中期):
学习律

神经科学

Fisher:
LDA(1936)

机器学习关注的内容

➤ 特征提取

- ◆ 图像、语音、文本特征提取
- ◆ 特征降维方法

➤ 模式识别方法

- ◆ 分类与回归
- ◆ 贝叶斯决策

➤ 统计学习方法

- ◆ 支持向量机
- ◆ 聚类

➤ 人工神经网络

- ◆ 感知器与神经网络
- ◆ 深度学习

核心问题：如何借鉴人的认知特点研究机器学习方法？

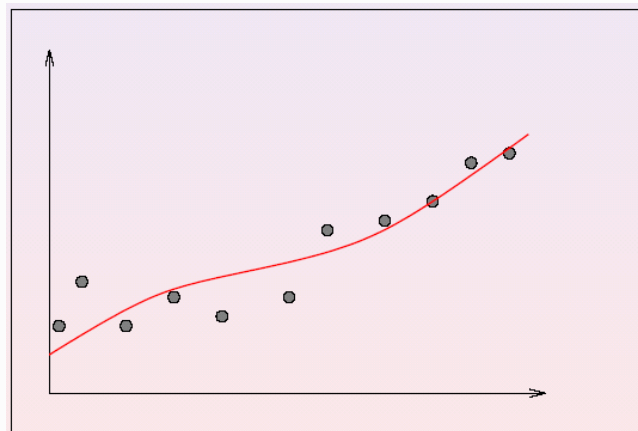
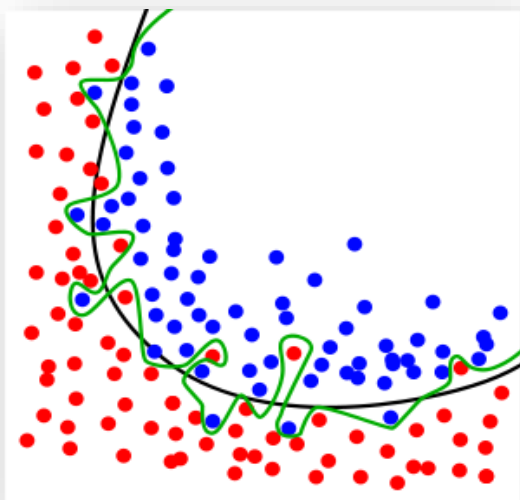
机器学习的基本任务

➤ 机器学习

- ◆ 根据训练样本对系统输入输出之间依赖关系进行建模，以便在测试阶段对输入做出预测。

➤ 基本任务

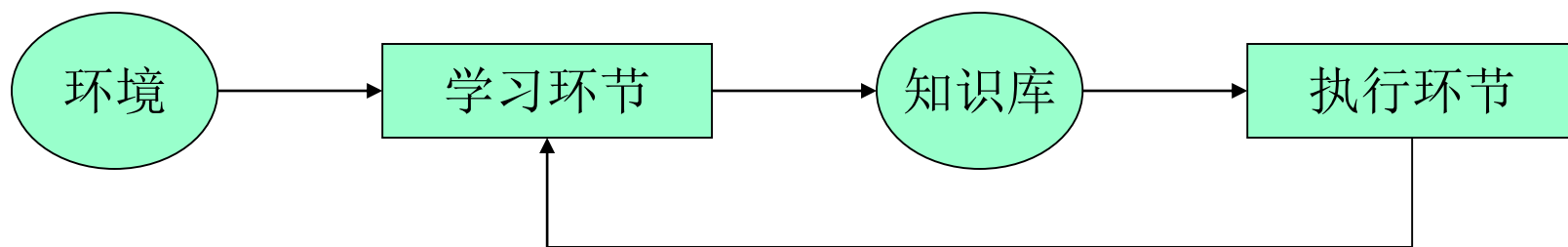
- ◆ 分类
- ◆ 回归



机器学习的基本问题和方法

► 机器学习

- ◆ 根据给定的训练样本求对某系统输入输出之间依赖关系的估计，使它能够对未知输出作出尽可能准确的预测。



Simon的学习模型

► 机器学习问题的表示

- ◆ 根据 n 个独立同分布观测样本确定预测函数 $f(\mathbf{x}, \mathbf{w})$ 。
- ◆ 在一组函数 $\{f(\mathbf{x}, \mathbf{w})\}$ 中求一个最优的函数 $f(\mathbf{x}, \mathbf{w}_0)$ 对依赖关系进行估计，使预测的期望风险最小。

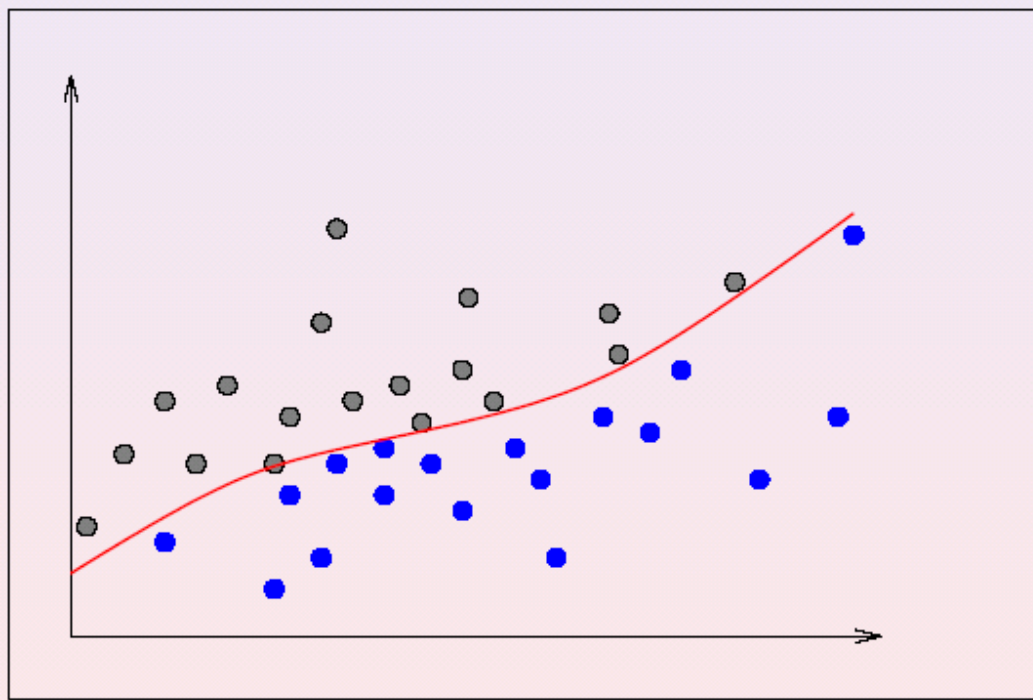
三个基本的机器学习问题—1

- **分类问题**：输出 y 是类别标号，两类情况下 $y=\{1,-1\}$ ，预测函数称作指示函数(Indicator Function)，损失函数定义见下式，使期望风险最小就是Bayes决策中使错误率最小。

$$L(y, f(\mathbf{x}, w)) = \begin{cases} 0 & y = f(\mathbf{x}, w) \\ 1 & y \neq f(\mathbf{x}, w) \end{cases}$$

图像分类，物体检测
和识别

支持向量机SVM，
卷积神经网络CNN



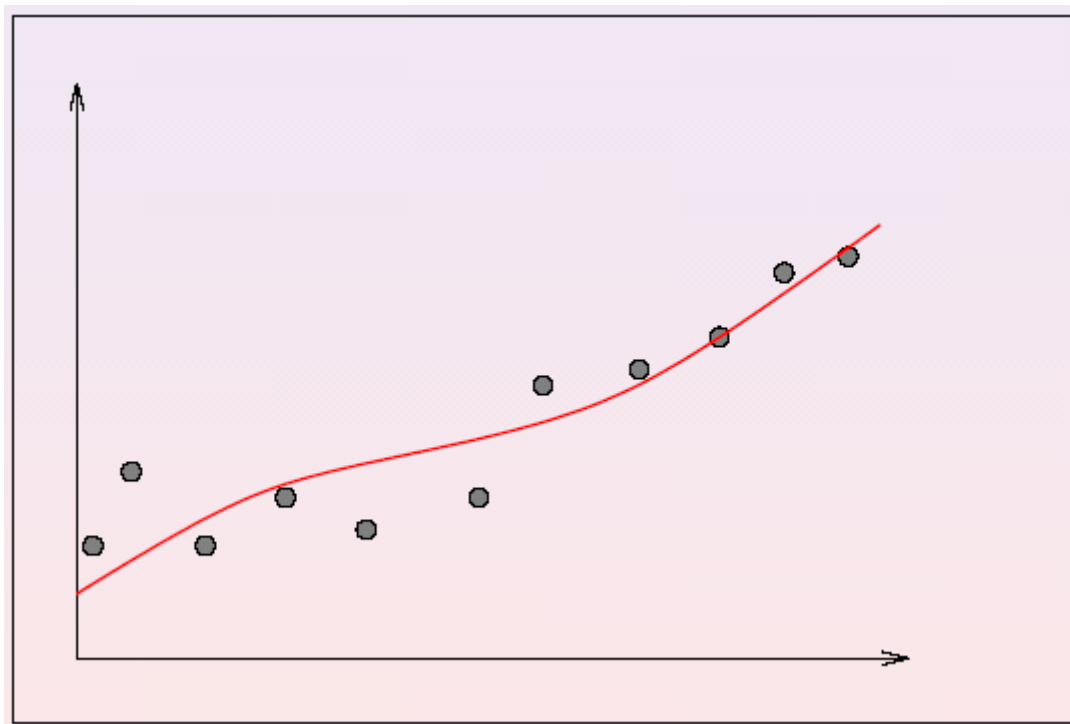
三个基本的机器学习问题—2

► 回归问题：输出 y 是连续变量，它是 $\mathbf{x}$ 的函数，损失函数定义见下式：

$$L(y, f(\mathbf{x}, \omega)) = (y - f(\mathbf{x}, \omega))^2$$

预测、姿态估计等应用。

Fisher 线性回归，逻辑回归



三个基本的机器学习问题—3

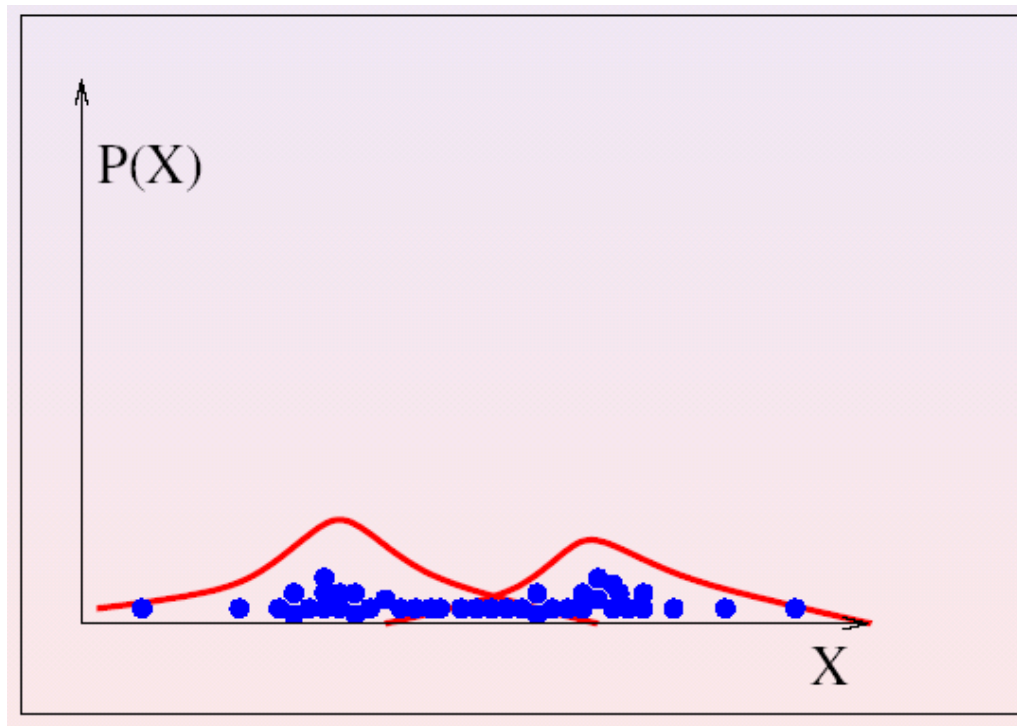
- 概率密度估计问题：根据训练样本确定 $\mathbf{x}$ 的概率分布 $p(\mathbf{x}, \omega)$ ，则损失函数可定义为：

$$L(p(\mathbf{x}, \omega)) = -\log p(\mathbf{x}, \omega)$$

概率密度估计（**Probability Density Estimation**）是指从一系列观测结果中估计 $\mathbf{x}$ 的概率分布。一般在聚类、异常检测中应用。

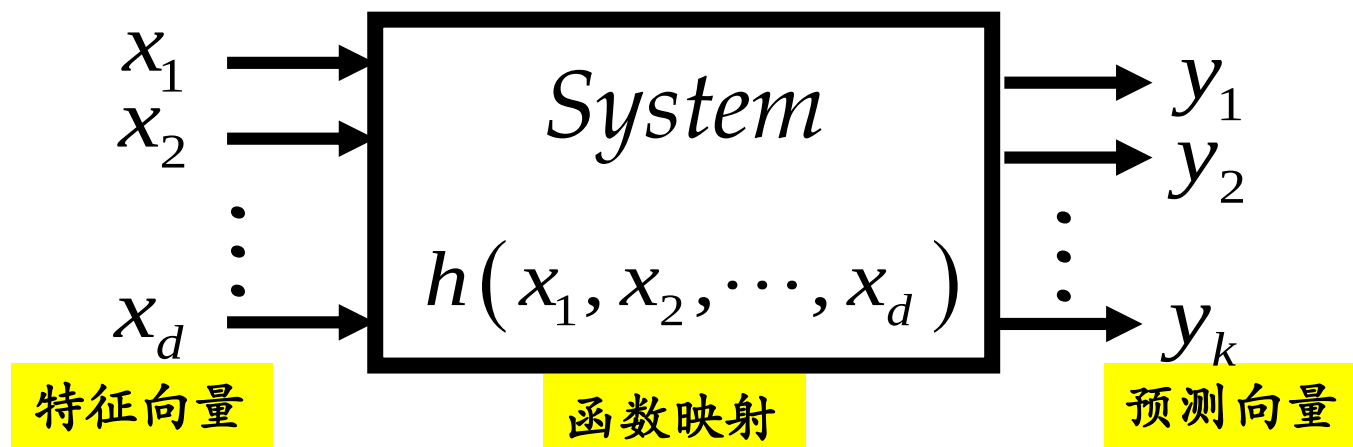
参数法：最大似然估计；

非参数法：K紧邻。



机器学习模型

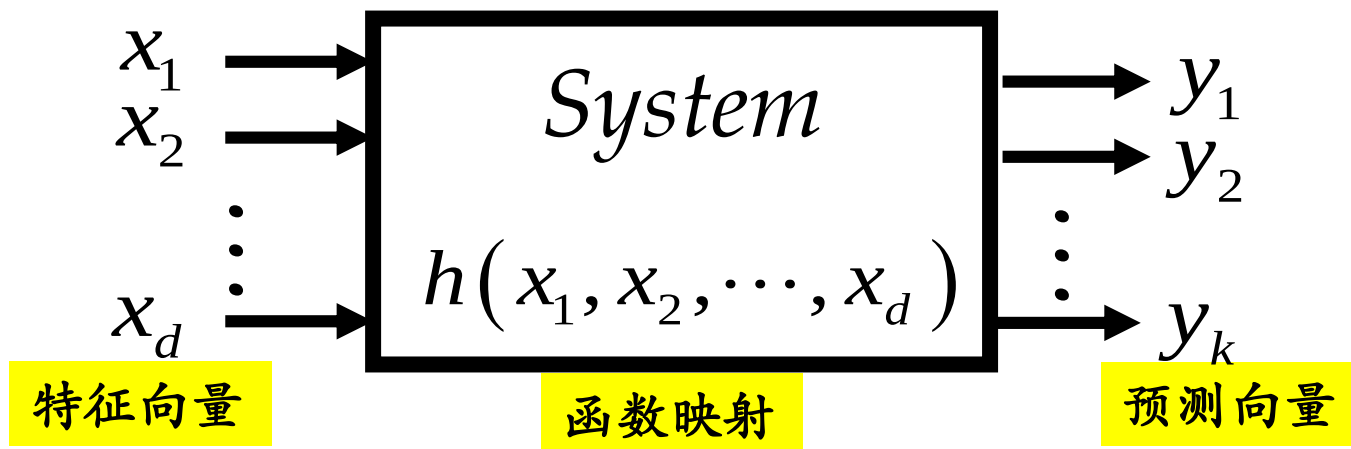
► 机器学习的一般模型



- (1) **分类学习**: y 取值为离散的标签 $y \in \{0, 1, 2, K-1\}$;
- (2) **回归学习**: y 取值为实值 $y \in \mathbb{R}^k$, 例如: MSE

机器学习模型

► 机器学习的一般模型



分类学习： 目标识别(object recognition)



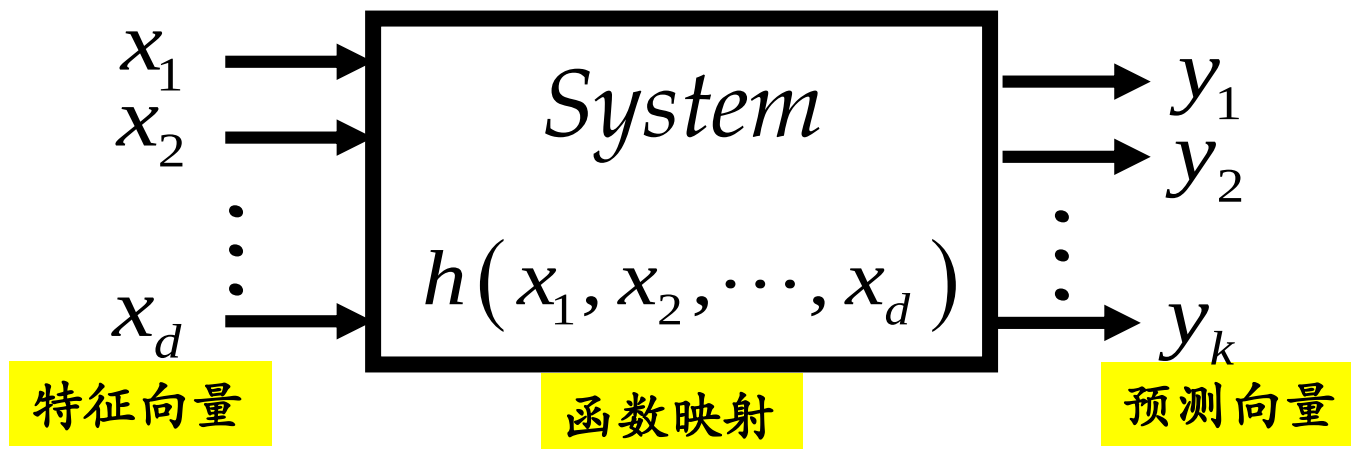
“cat”



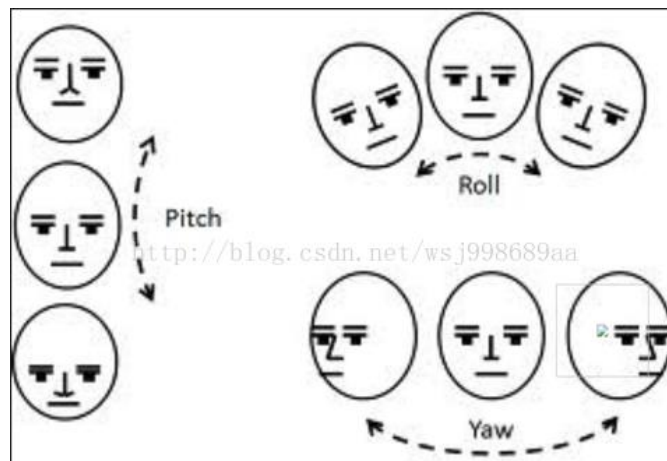
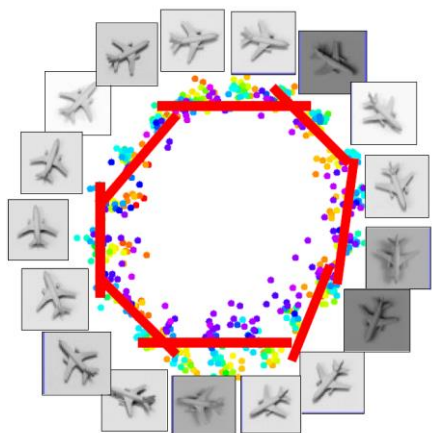
“dog”

机器学习模型

► 机器学习的一般模型

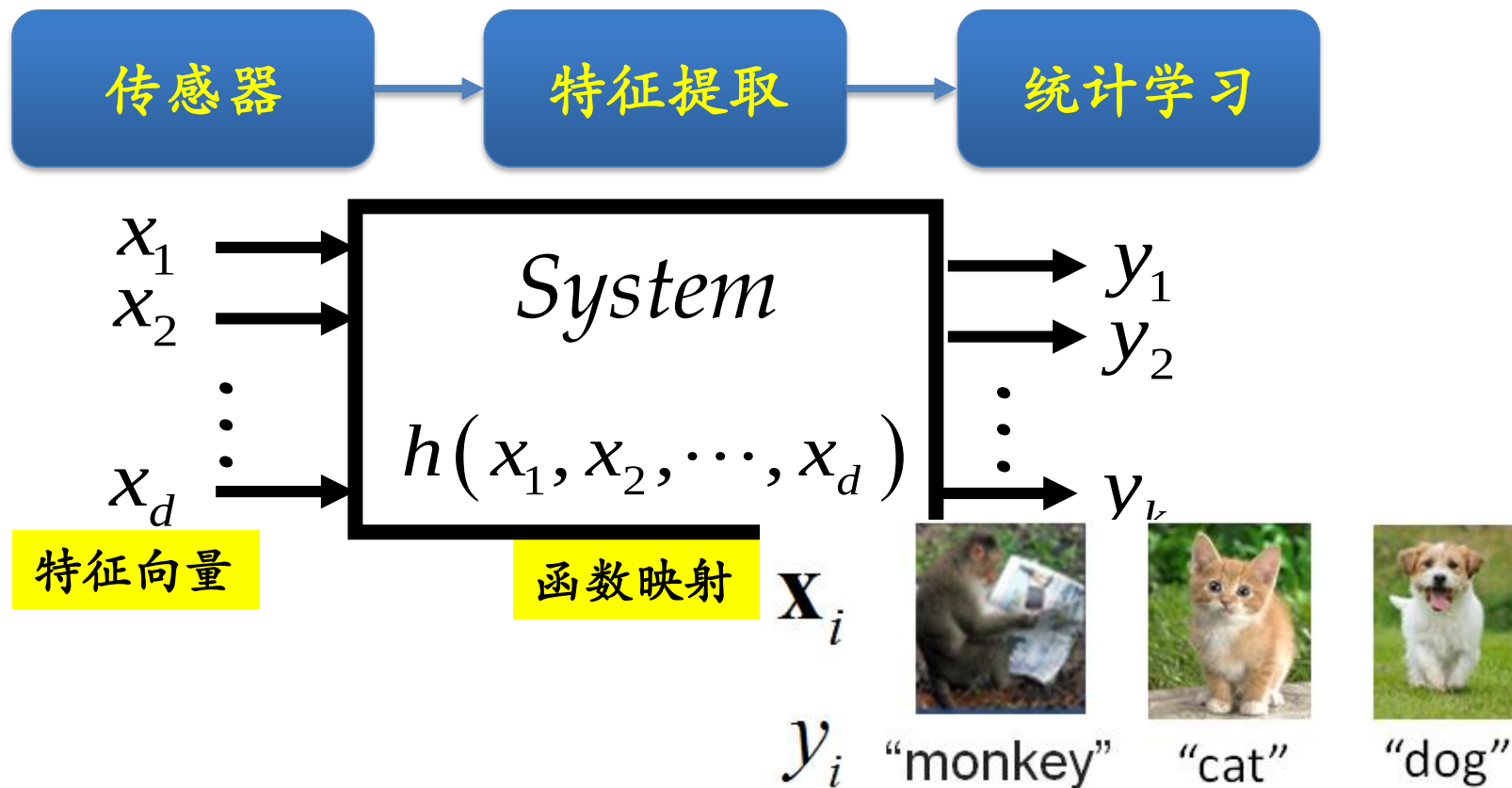


回归学习：姿态估计 (pose estimation)



机器学习模型

机器学习的一般模型



- (1) 有监督学习: 训练集样本标签已知, 记为 $\{\mathbf{x}_i, y_i\}$;
- (2) 无监督学习: 训练集样本标签未知, 记为 $\{\mathbf{x}_i\}$

机器学习模型

► 机器学习的一般模型

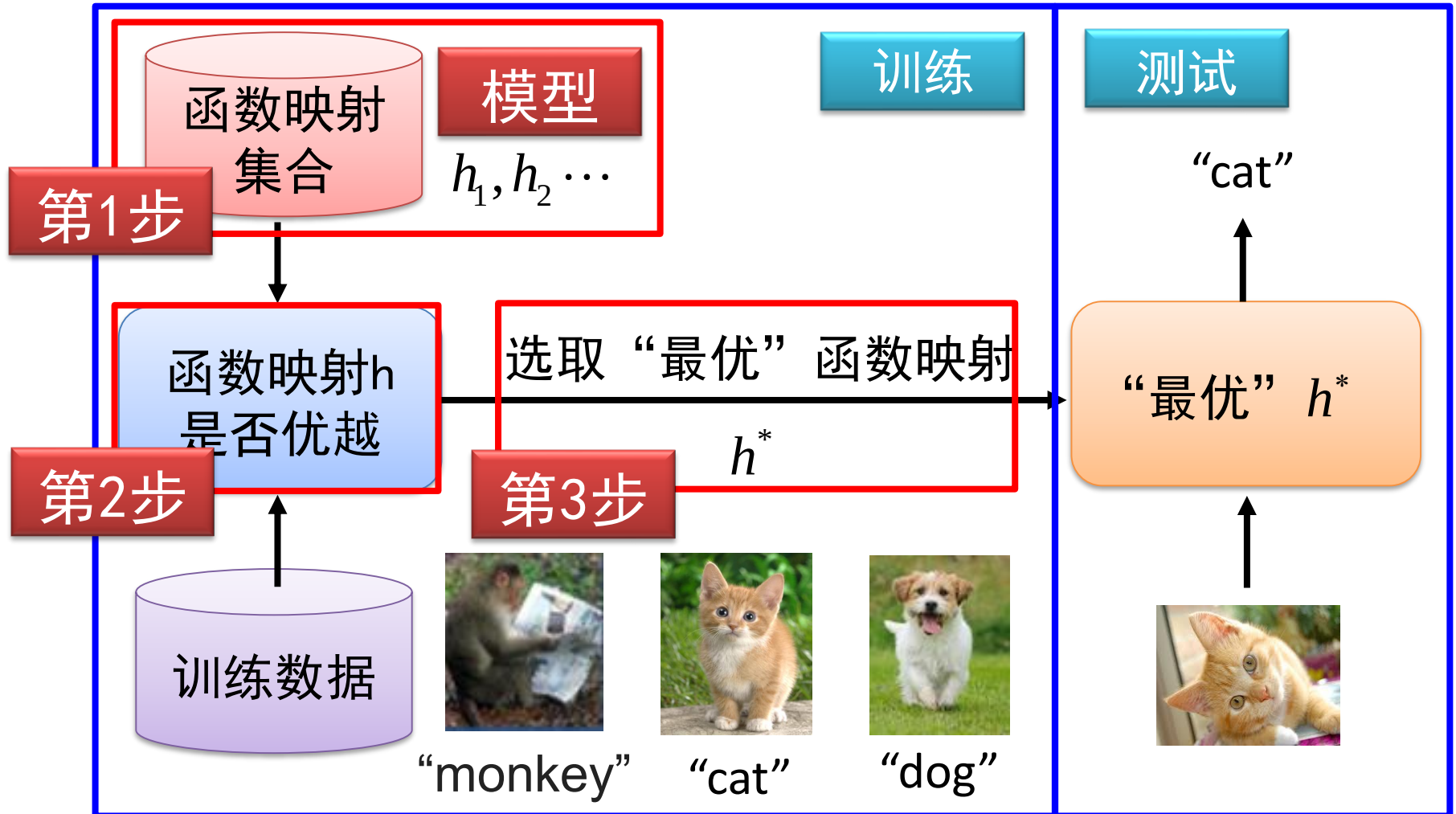
机器学习范式进展

- (1) (有)监督学习：训练集样本标签已知；
- (2) 无监督学习：训练集样本标签未知；
- (3) 弱监督学习：图像级标注；
- (4) 半监督学习：小数据标注建模+无标注数据
- (5) 自监督学习：利用pretask先验知识。
- (6) 跨监督学习：omni-supervised learning，利用所有可用的数据；
全方位学习

机器学习的一般流程

◆ 监督学习

$$h(\text{image of a cat}) = \text{"cat"}$$



机器学习模型

➤ 监督学习：

- ◆ 已知样本数据 $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ ，拟合预测函数 $h(x)$ ，使得 $h(x_i)$ 和 y_i 之间差异最小化
- ◆ 假设产生样本数据标签 y 的真值函数为 $\hat{h}(x)$ ，利用学习获取的预测函数为 $h(x)$ 。由于采样样本数据 $y = \hat{h}(x) + \varepsilon$ ，其中噪声 ε 均值为0，方差 σ^2 ，则

$$E \left\{ \left(y - h(x) \right)^2 \right\} = \sigma^2 + \text{Var} [h(x)] + \left(\text{Bias} [h(x)] \right)^2$$

误差的期望值 模型预测值的方差 预测值相对于真值的偏差的平方

↑
采样自身噪声方差

E的解释见下页。噪声为真实标记与数据集中实际标记间的偏差

机器学习模型

➤ 监督学习：

可表示为 $E[h - E(h)]^2$
或 $E[h^2] - [E(h)]^2$

预测值相对于真值的偏差的平方

$$E\left\{\left(y - h(x)\right)^2\right\} = \sigma^2 + \text{Var}[h(x)] + \left(\text{Bias}[h(x)]\right)^2$$

误差的期望值

模型预测值的方差

$$\begin{aligned} E\left[\left(\hat{h}(x) + \varepsilon - h(x)\right)^2\right] &= E\left[\varepsilon^2 + 2\varepsilon(\hat{h}(x) - h(x)) + (\hat{h}(x) - h(x))^2\right] \\ &= \sigma^2 + E\left[(\hat{h}(x) - h(x))^2\right] = \sigma^2 + E\left\{\left[(h(x) - \hat{h}(x)) - E(h(x) - \hat{h}(x))\right]^2\right\} + \left[E(h(x) - \hat{h}(x))\right]^2 \end{aligned}$$

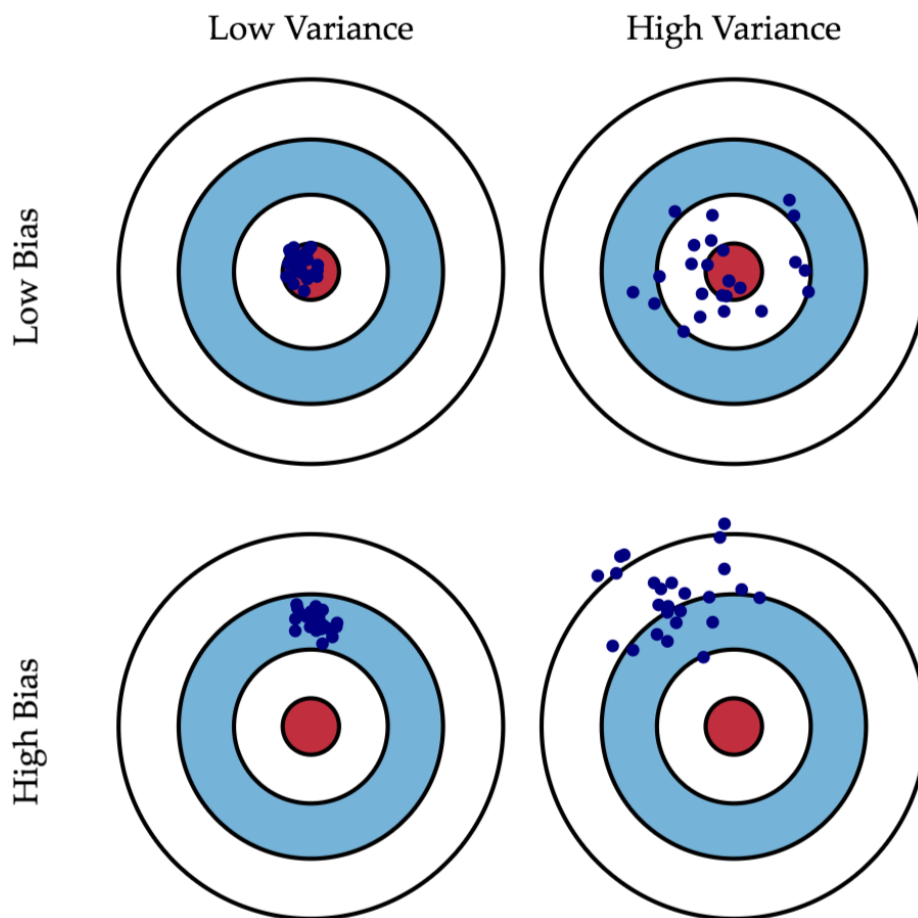
展开后，利用方差等于平方的期望减去期望的平方，得到Var

预测值相对于真值的偏差的平方

➤ 监督学习:

模型预测值的方差

$$E\left\{\left(y-h(x)\right)^2\right\}=\sigma^2+\operatorname{Var}\left[h(x)\right]+\left(\operatorname{Bias}\left[h(x)\right]\right)^2$$

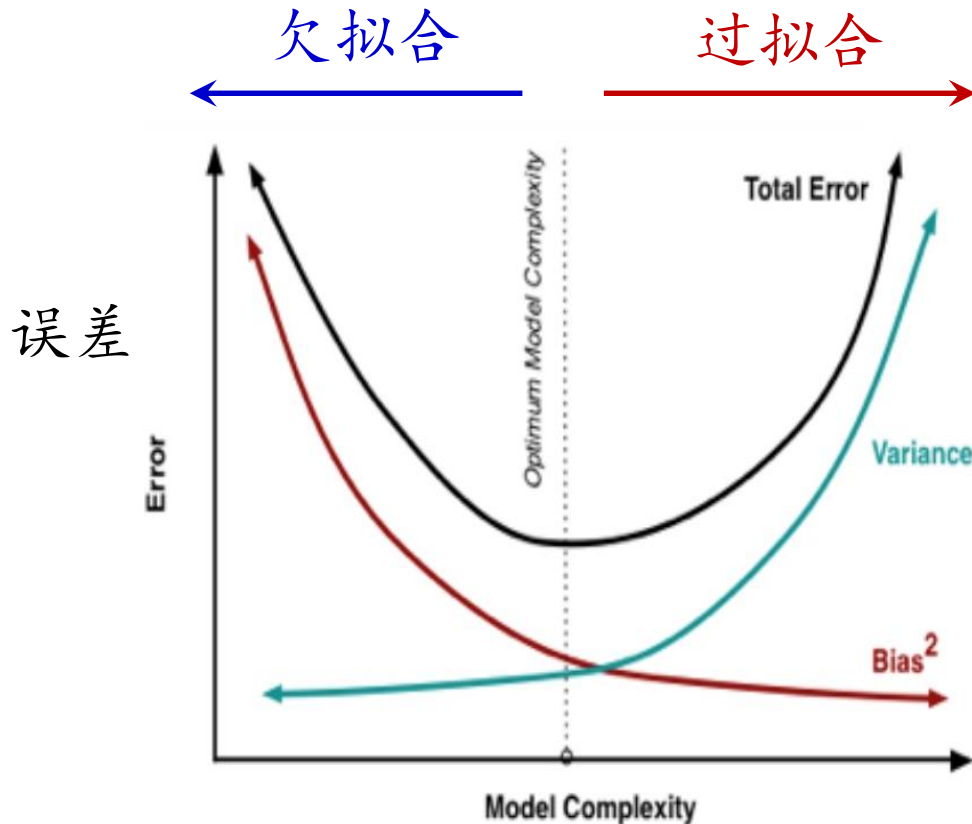


预测值相对于真值的偏差的平方

➤ 监督学习：

模型预测值的方差

$$E\left\{\left(y-h(x)\right)^2\right\}=\sigma^2+\operatorname{Var}\left[h(x)\right]+\left(\operatorname{Bias}\left[h(x)\right]\right)^2$$



从模型适应性能看，通过增加模型的复杂度，方差将增加且偏差减小，而更简单的模型，其偏差增加、方差减小。须选择适当模型复杂度

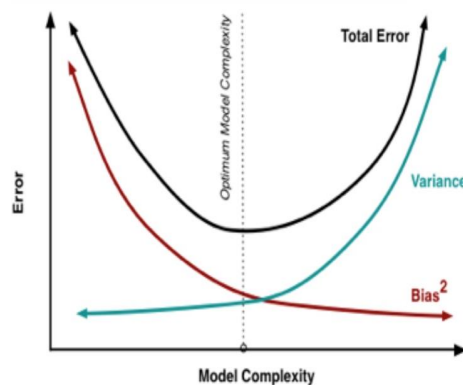
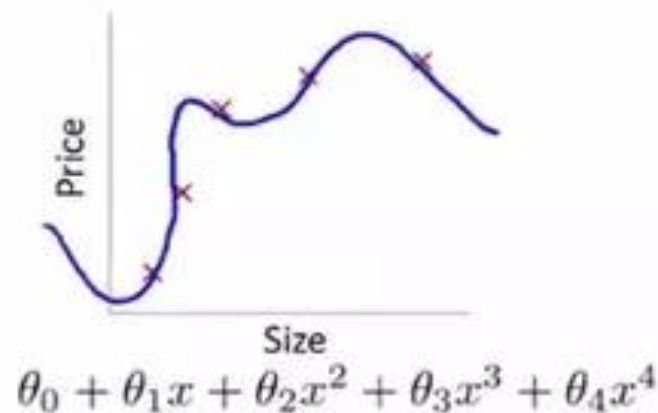
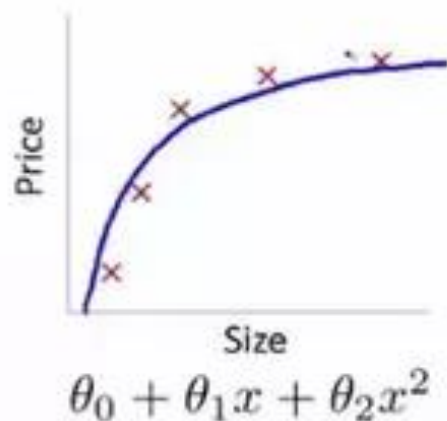
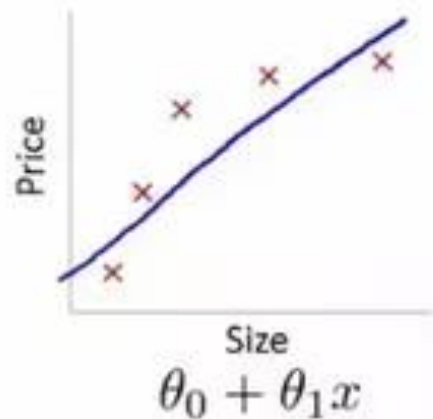
模型复杂度

➤ 监督学习：

模型预测值
的方差

预测值相对
于真值的偏
差的平方

$$E \left\{ (y - h(x))^2 \right\} = \sigma^2 + \text{Var} [h(x)] + \left(\text{Bias} [h(x)] \right)^2$$



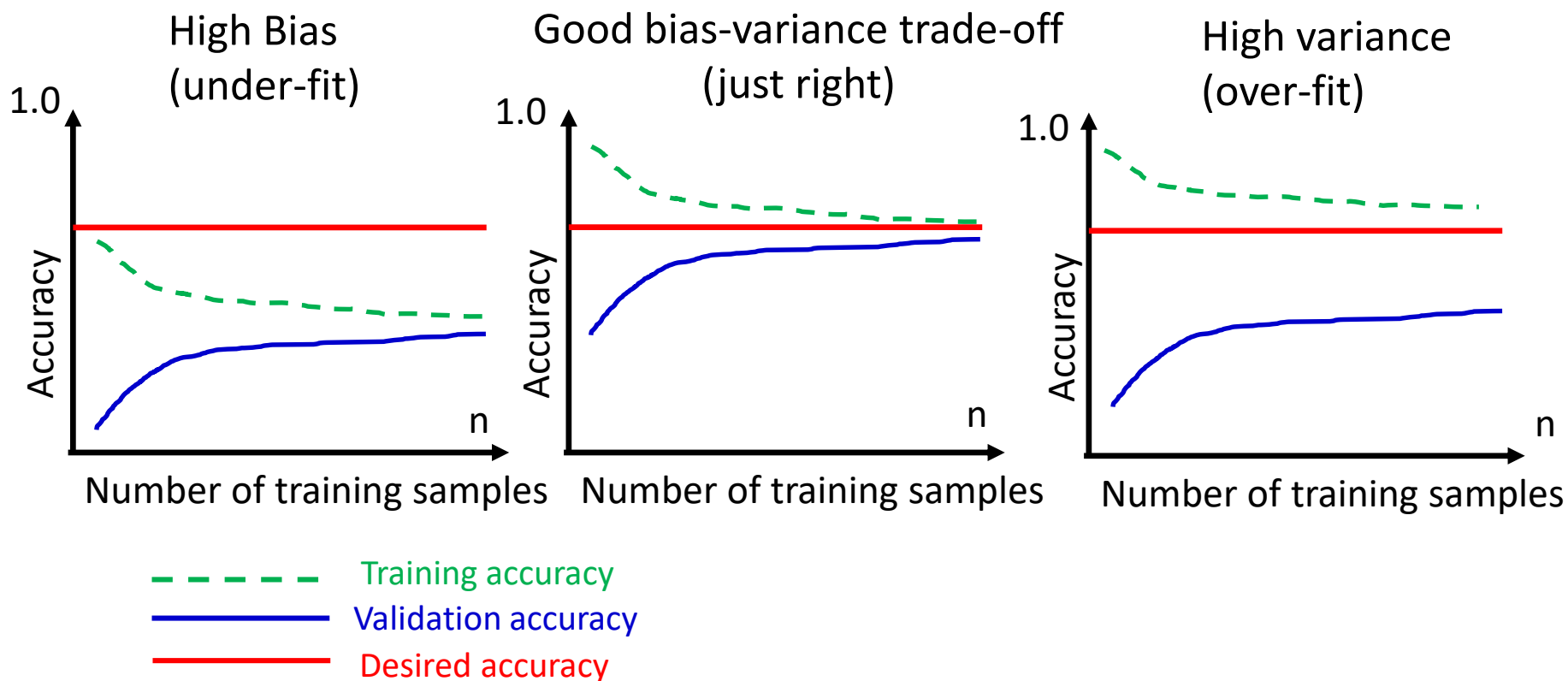
从模型适应性能看

➤ 监督学习：

模型预测值
的方差

预测值相对
于真值的偏
差的平方

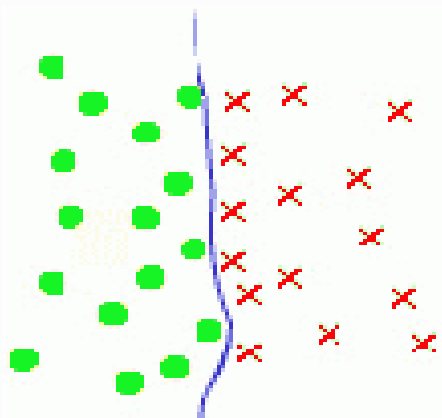
$$E \left\{ (y - h(x))^2 \right\} = \sigma^2 + \text{Var} [h(x)] + \left(\text{Bias} [h(x)] \right)^2$$



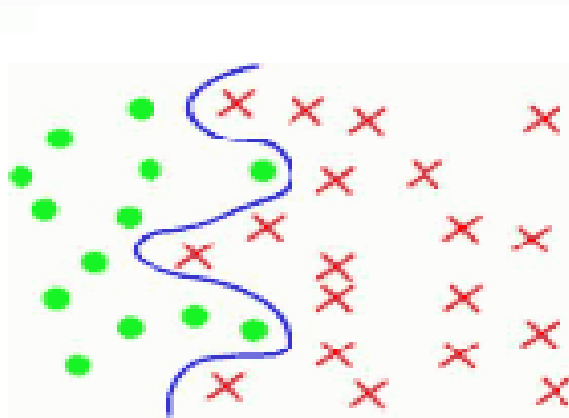
基本问题：样本类的紧致性

► 紧致的同类样本集合

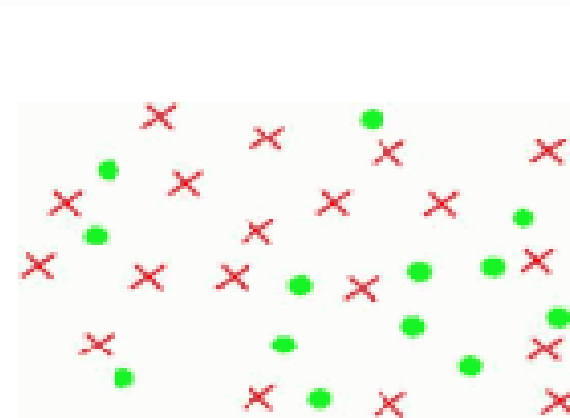
- ◆ 临界点的数量与总的点数相比很少
- ◆ 集合内的任意两点的连线，在线上的点属于同一类别
- ◆ 每一个点有足够大的邻域，在邻域内只包含同一类别的点



(a) 没有临界点



(b) 有较多临界点



(c) 临界点过多

机器学习方法

机器学习：基于数据构建统计模型从而对数据进行预测和分析的学习方法

➤ 有监督/无监督学习（针对样本标注）

- ◆ 有监督(Supervised): 分类、回归
- ◆ 无监督(Unsupervised): 概率密度估计、聚类
- ◆ 半监督(Semi-supervised)

➤ 其他学习方法（针对学习方法）

- ◆ 强化学习(Reinforcement Learning)
- ◆ 元学习(Meta Learning)
- ◆ 多任务学习(Multi-task learning)

3.1 机器学习概要—两个典型模型

➤ 产生式模型 Generative models:

- ◆ 建模(联合)概率分布: $p(\mathbf{x}, \omega)$

- ◆ 利用Bayes' theorem

$$\text{后验 } p(\omega | \mathbf{x}) = \frac{\text{似然 } p(\mathbf{x} | \omega) \text{ 先验 } p(\omega)}{\text{证据因子 } p(\mathbf{x})}$$

- ◆ 典型方法: Bayes, GMM, HMM等

➤ 判别式模型 Discriminative models:

- ◆ 直接用函数（而非概率）对 $p(\omega|x)$ 来建模

- ◆ 典型方法: LDA, SVM, 神经网络

- ◆ 一般来说, 性能更好

➤ 机器学习算法分类

按照学习方式分类可分为

学习方式	英文	描述
监督式学习	Supervised Learning	训练集目标:有标注; 如回归分析, 统计分类
非监督式学习	Unsupervised Learning	训练集目标:无标注;如聚类、GAN(生成对抗网络)
半监督式学习	Semi-supervised Learning	介于监督式与无监督式之间
强化学习	Reinforcement Learning	智能体不断与环境进行交互, 通过试错的方式来获得最佳策略

按照学习任务分类可分为以下三类

学习任务	英文	描述
分类	Classification	分类是预测一个标签（是离散的），属于监督学习
回归	Regression	回归是预测一个数量（是连续的），属于监督学习
聚类	Clustering	属于无监督学习

➤ CNN的前世今生—机器学习领域的小故事之一

很多人认为，Lecun于1989年发表了《Backpropagation Applied to Handwritten Zip Code》是CNN的第一个实现网络，但是通读全文，找不到和CNN模型原理有关的解释说明。为什么大家都认1989年。后来从《Backpropagation Applied to Handwritten Zip Code》中的参考文献找到了答案

Kunihiko Fukushima(福岛邦彦，个人主页：[Kunihiko Fukushima](#))的Neocognitron（新认知机）。普遍认为Neocognitron是1980年的这篇论文提出来的《1980-Fukushima-Neocognitron A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position》

上述论文讲了诸多CNN概念，如卷积和池化（当时没用该词）这两个概念是从猫的视觉系统实验中得到的启发（Hubel和Wiesel, 1962）。Lecun在一次访谈中给了较高评价，提到Kunihiko Fukushima(福岛邦彦)给了他很大启发

作者：余霆嵩

链接：<https://www.zhihu.com/question/47705441/answer/121003616>

来源：知乎

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

➤ CNN的前世今生

那么，为什么Lecun能把Neocogniron的精华提取出来然后加上其1986的BP算法，然后把CNN发扬光大，而Kunihiko Fukushima(福岛邦彦)却没有将Neocognitron发扬光大呢？

分析1982-1994的几篇关于Neocogniron的paper，与Lecun提出的CNN想比，还是差在BP算法上。所以到了1998年Lecun的LeNet-5的提出（《Gradient-Based Learning Applied to Document Recognition》），把CNN推上了一个里程碑，从此Neocognitron也少有paper。

再往后到了2006年，这篇paper《Notes on Convolutional Neural Networks》，里面给了详细的CNN权值更新的公式，是非常值得阅读的一篇paper。

➤ CNN的前世今生

2012年的里程碑，ImageNet LSVRC-2012 contest上，获得了惊人的准确度(top-5: 15.3%)。Alexnet以15.3%的top-5 错误率轻松拔得头筹（第二名top-5错误率为26.2%）。由此，DCNN的潜力受到广泛认可，一炮而红。

Alexnet中用到了Relu、Training on Multiple GPUs、LRN（Local Response Normalization）、Overlapping Pooling、Data Augmentation、Dropout等多种技巧，每种技巧对于模型分类错误率的降低都有贡献。

2012之后，雨后春笋般出现了多个知名的CNN网络结构，如AlexNet, GoogLeNet, VGG, ResNet。比这些工作影响力稍逊一些的，有Regional CNN(以及Fast/Faster RCNN), SPP-Net, Inception，等等

➤ 争论不休的GAN—机器学习领域的小故事之二

- GAN，全称为 Generative Adversarial Network（生成对抗器）
- 一般认为，由Ian Goodfellow，在 2014 年发表在 ICLR 的论文Generative Adversarial Networks，提出并引入深度学习领域
- 2016年，GAN热潮席卷AI领域顶级会议，从ICLR到NIPS，大量高质量论文被发表和探讨
- Yann LeCun曾评价GAN是“20年来机器学习领域最酷的想法”。

➤ 争论不休的GAN



Ian J. Goodfellow
(OpenAI)

作为一种生成模型：

“GAN是更好的生成模型”

“GAN避免了马尔科夫链式的学习机制”

“各种类型的损失函数都可以整合到GAN模型中”

“概率密度不可计算时，GAN仍可以应用”

学术界流传的一则GAN诞生的传说。据说是因为一天晚上Ian Goodfellow在酒吧在喝高的状态下与同事讨论学术问题，突然灵光一闪，提出了GAN初步的想法，当时并没有得到同事的认可。从酒吧回去后，熬夜写了代码，发现还真有效果。经过一番研究后，GAN就诞生了。

➤ 争论不休的GAN

LSTM之父Jürgen Schmidhuber声称，他在92年提出了一种PM（Predictability Minimization）模型，其与GAN有一些相似之处，所以称Ian Goodfellow的“GAN之父”的说法不合适。也有人认为Schmidhuber没有获得2019年图灵奖抱不平

为此，Schmidhuber和Ian不仅在邮件中展开了激烈的争论，甚至在2016 NIPS大会上，趁Ian正在做GAN的tutorial的时候，Schmidhuber站起来打断演讲。首先介绍了一下自己92年提出的PM模型，介绍了PM模型的原理和实现过程等等，然后反问Ian如何看待GAN和PM的相似点。



3.1 机器学习概要—领域应用

机器学习系统应用

➤ 网络海量信息检索与处理

- ◆ 文本、图像、语音等信息检索、分类、转换

➤ 生物特征身份认证

- ◆ 人脸、指纹、虹膜等身份认证

➤ 视频监控

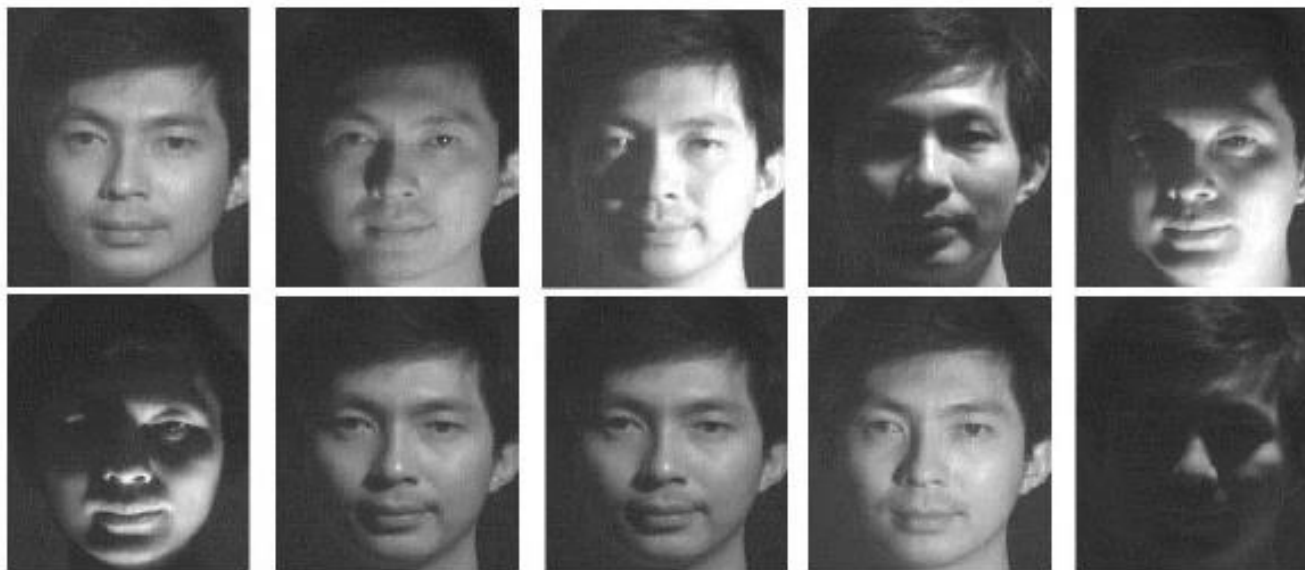
- ◆ 行人、车辆等目标检测、跟踪、识别、分割

➤ 生成式人工智能（AIGC）

机器学习案例：人脸识别系统

► 难点

◆ 姿态或视角、光照、表情变化大

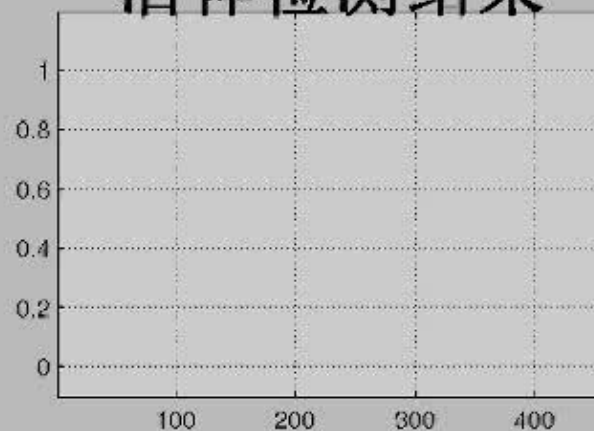


◆ 高维特征和小样本学习

活 体



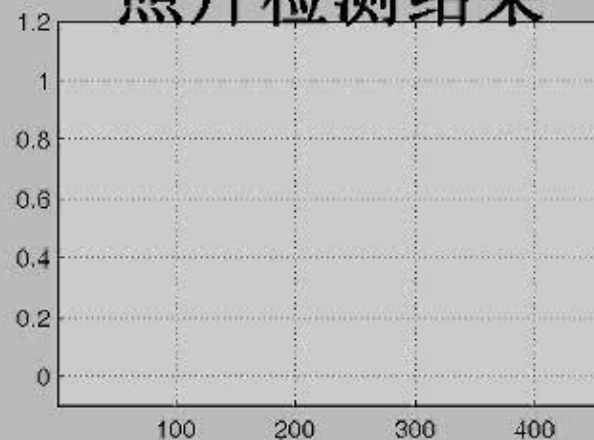
活体检测结果



照 片

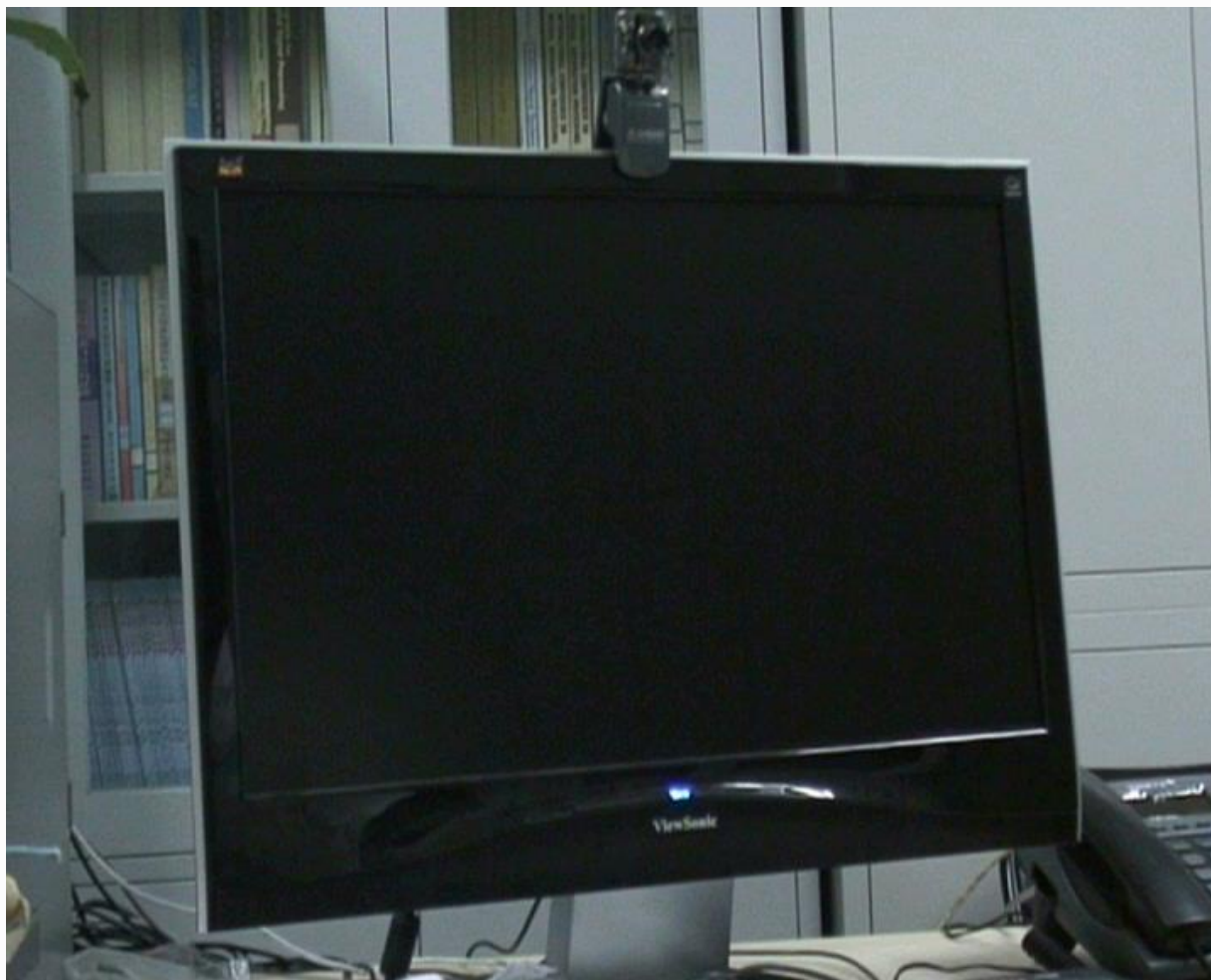


照片检测结果



人脸识别应用于计算机登录

本团队实验室



多模态协作机器人，我们实验室的研究工作



3.1 机器学习概要—示例1：回归

➤ 回归是机器学习的入门算法

朴素贝叶斯，决策树，支持向量机等都是属于离散型的监督分类，本文要讲的是连续型监督分类：**回归 (regression)**

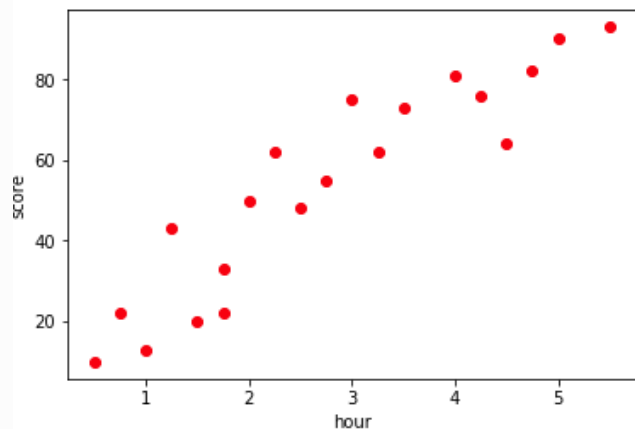


➤ 一个简单的线性回归示例

通过该示例，说明如何建立学习时间和考试分数之间的预测关系，通过建立线性回归模型，根据学习时间预测考试分数。

1、采集数据输入：数值，平均值，标准差，最小值和四分位数

2、绘制散点图观察数据情况



有20个数值对

```
In [8]: examDf.head()
```

Out[8]:

	学习时间	考试分数
0	0.50	10
1	0.75	22
2	1.00	13
3	1.25	43
4	1.50	20

```
In [9]: examDf.describe()
```

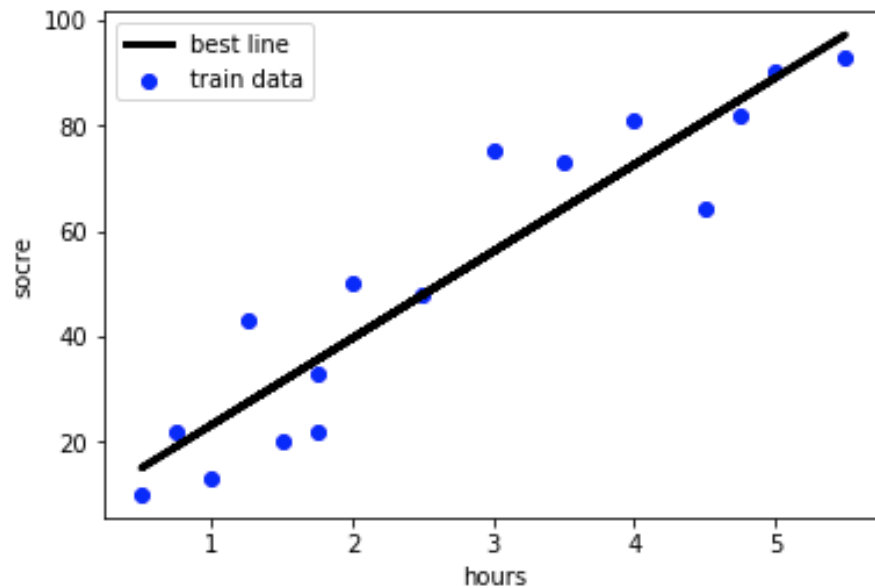
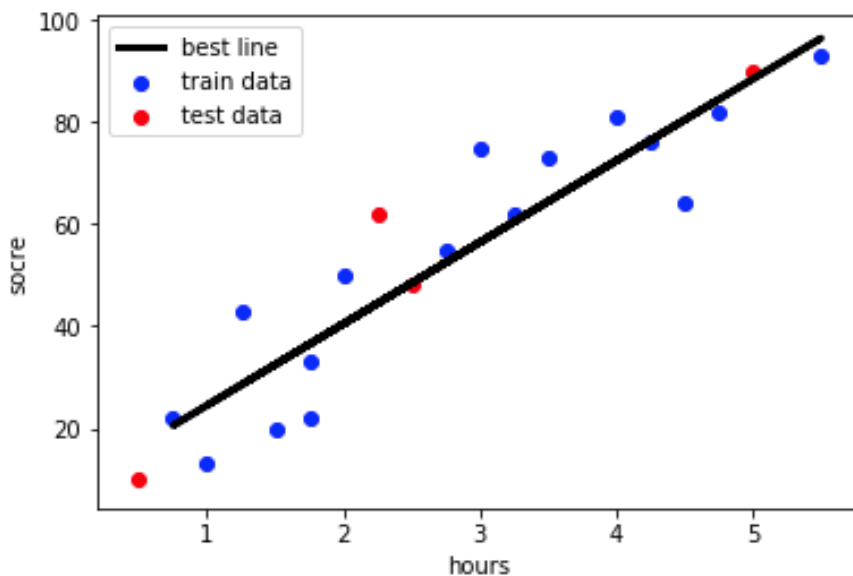
Out[9]:

	学习时间	考试分数
count	20.000000	20.000000
mean	2.787500	53.700000
std	1.507165	26.435821
min	0.500000	10.000000
25%	1.687500	30.250000
50%	2.625000	58.500000
75%	4.062500	75.250000
max	5.500000	93.000000

➤ 一个简单的线性回归示例

3、采用线性回归模型，利用最小二乘法求出最佳拟合曲线的函数，求出最佳拟合曲线函数的截距 b ；回归系数 a ， $y=ax+b$

4、模型评估



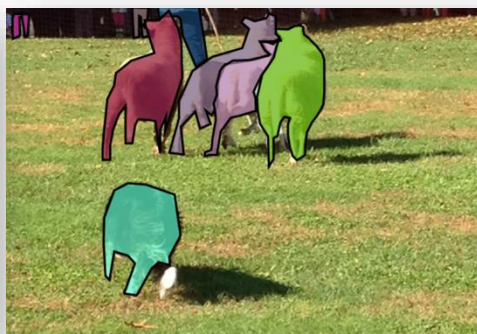
3.1 机器学习概要—示例2： 分类

➤ 分类是机器学习的基本任务

图像分类包括单一类别的图像分类问题，也包括对每一样例图像所从属的多类别进行全面的分类

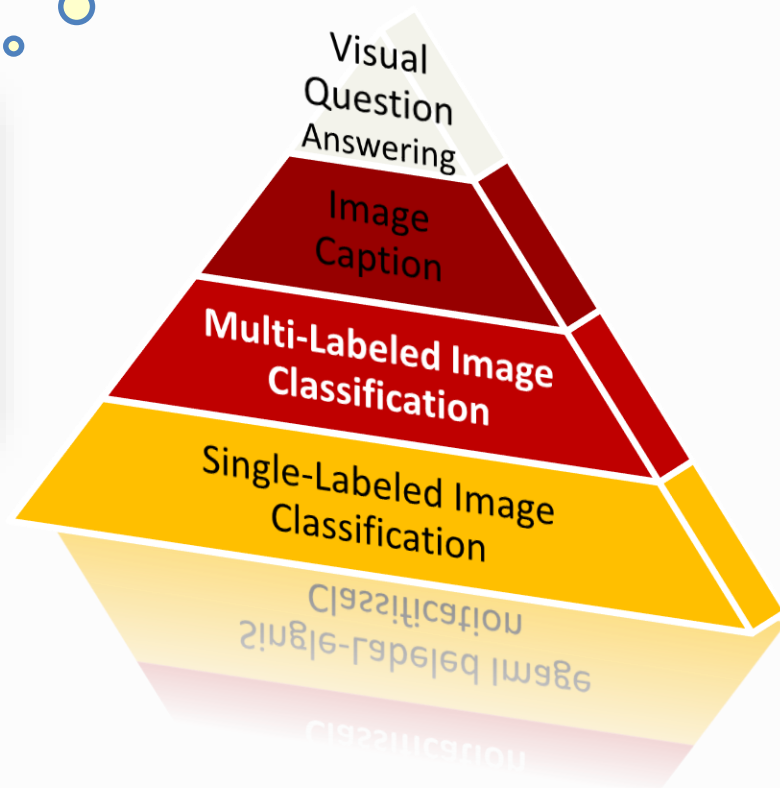


单标签
Dog



多标签
Dog
Sheep
People

思考：分类与分割的异同？



➤ 食堂菜品分类



GT: 凉拌猪耳朵

0 凉拌猪耳朵 score: 3.0146 pr: 0.6740

1 手撕鸡 score: 0.9071 pr: 0.0819

2 水煮肉片 score: 0.6088 pr: 0.0608



GT: 水蒸蛋

0 水蒸蛋 score: 3.5580 pr: 0.7964

1 小南瓜 score: 0.7028 pr: 0.0458

2 鱼块 score: 0.3934 pr: 0.0336



GT: 糖醋里脊

0 糖醋里脊 score: 3.0189 pr: 0.6822

1 红烧肉 score: 0.9268 pr: 0.0842

2 老鸭煲 score: 0.0537 pr: 0.0352



GT: 鱼块

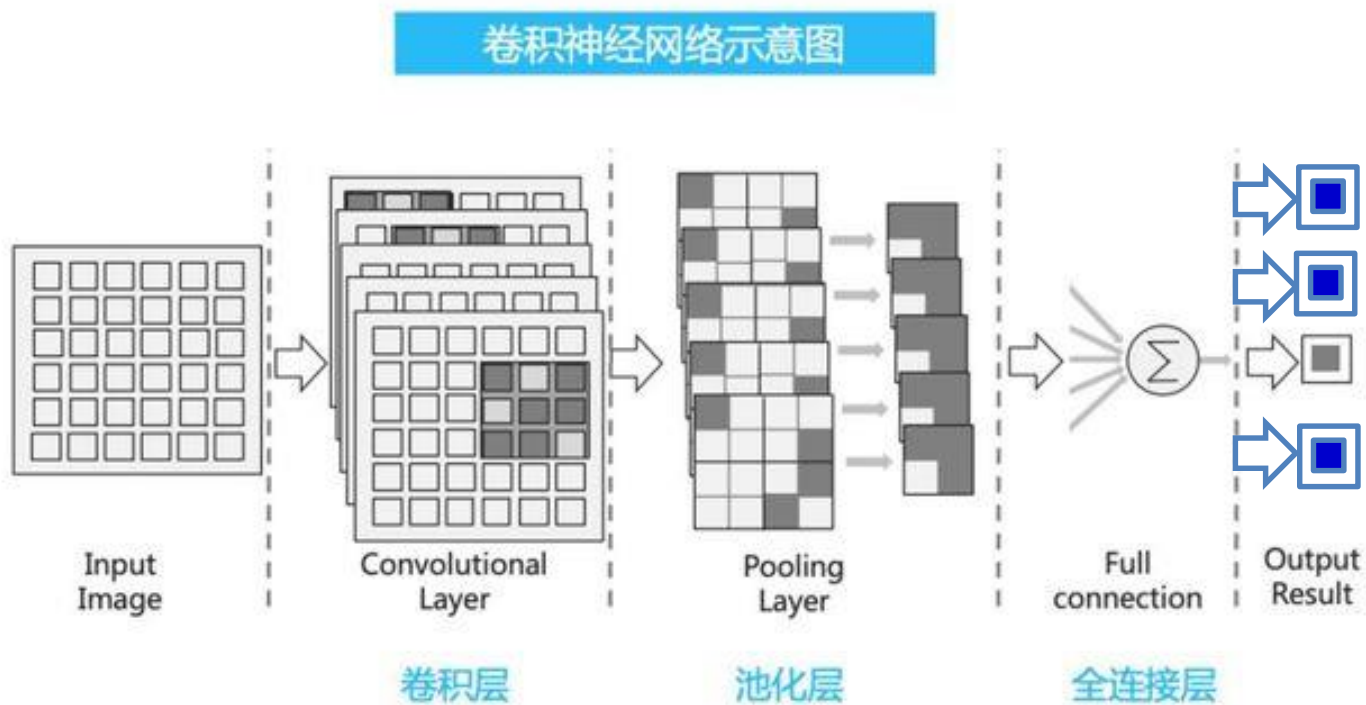
0 鱼块 score: 2.4245 pr: 0.5716

1 老鸭煲 score: 0.5229 pr: 0.0854

2 水蒸蛋 score: 0.0529 pr: 0.0533



➤ 食堂菜品分类



AlexNet
GoogLeNet
VGG
ResNet

知乎@宽涛

3.1 机器学习概要—示例3： 检测

► 检测是机器学习的进阶任务

物体检测涉及公共安全，人机交互，军事技术，工业自动化，健康等多个重要应用领域，是计算机视觉的进阶基础

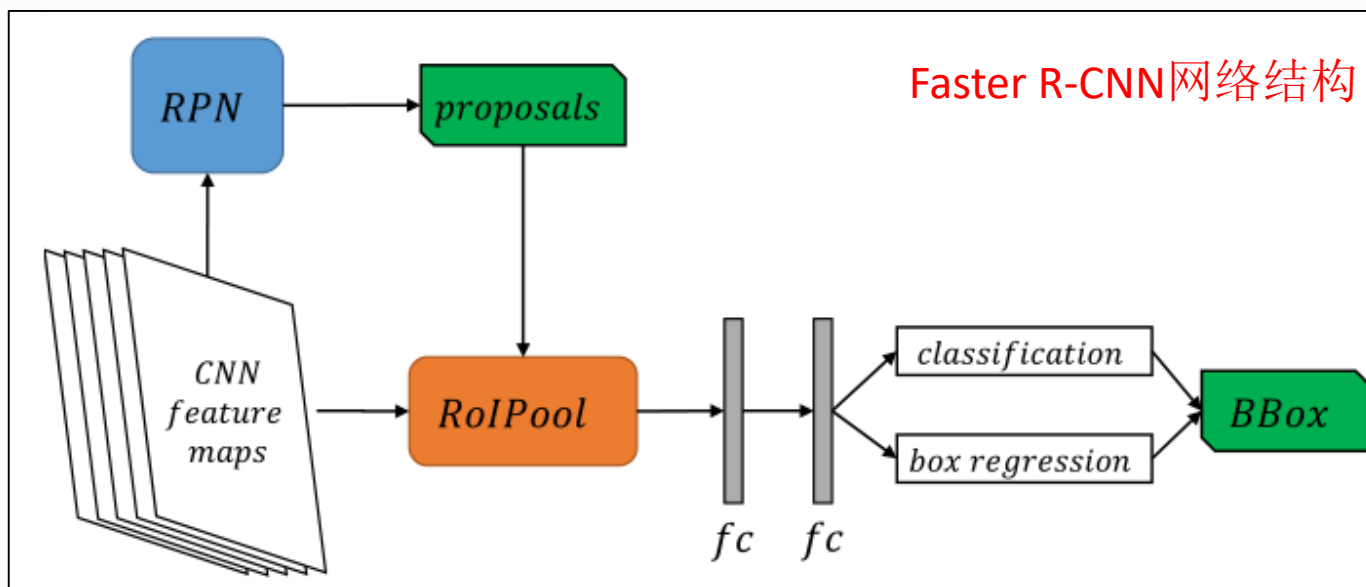


S74E5031 19951114 14:02:02



➤ 车辆检测

KITTI数据集



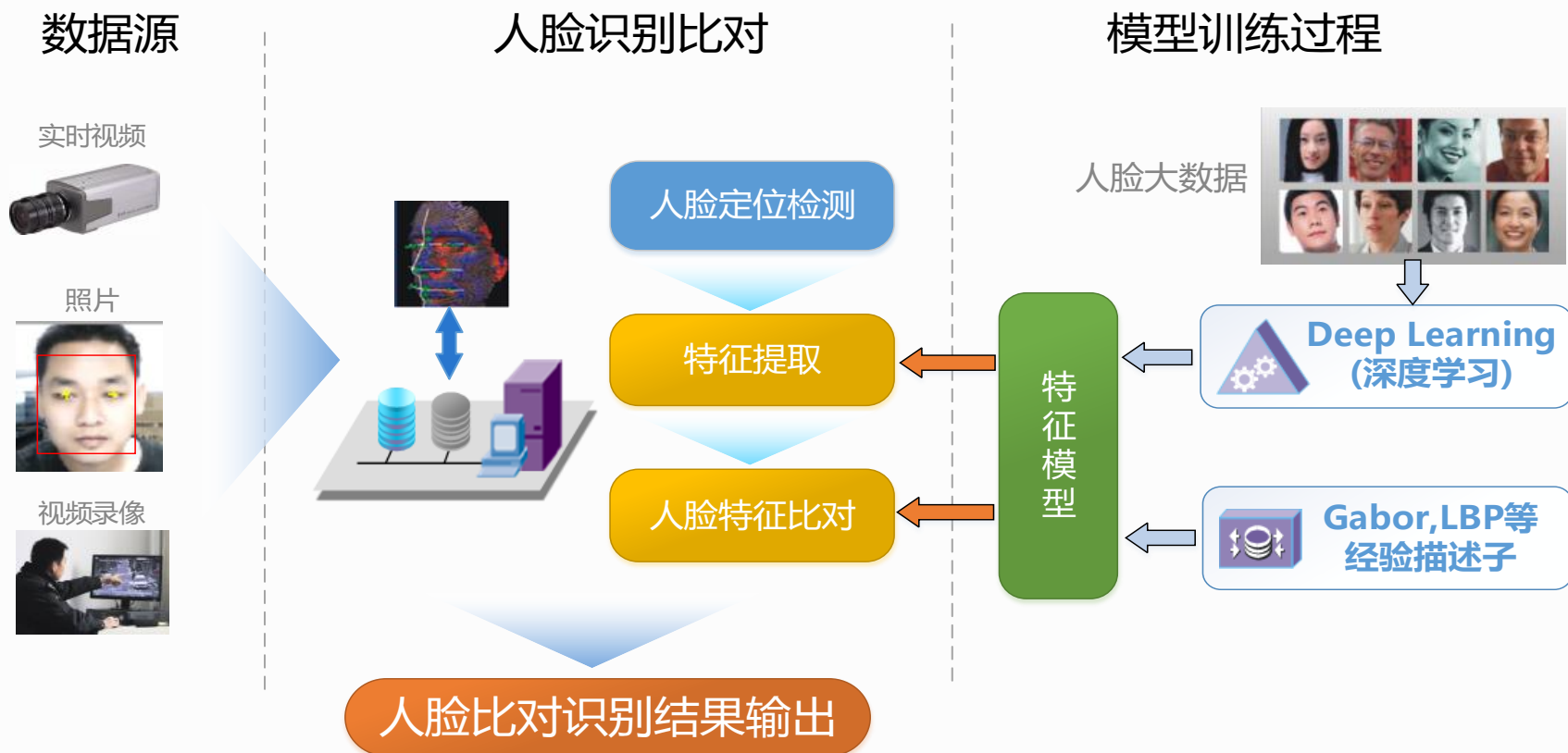
➤ 车辆检测

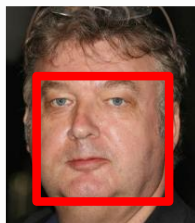


3.1 机器学习概要—示例4： 识别

➤ 识别可以认为是机器学习中的细粒度分类

以人脸识别为例，通过摄像机采集图像，并在图像中检测人脸，进而对人脸目标进行识别。

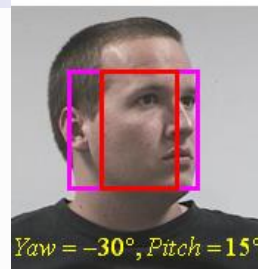




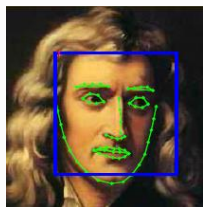
人脸检测



关键点定位

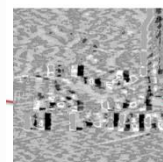


姿态估计



人脸配准

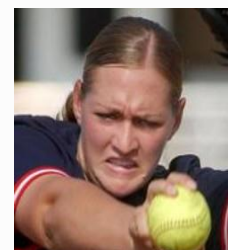
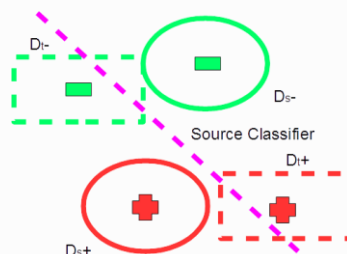
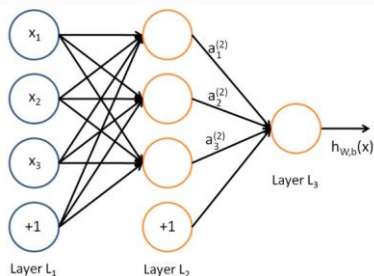
归一化



分类器训练

特征匹配

人物判别



➤ 人脸识别

人脸识别从应用上一般分为人脸检测，人脸五官定位，1:1人脸识别，1: N人脸识别，M: N动态布控。

人脸检测与五官定位应用方向：客流量统计，视频检索等。智能贴图，智能美妆美颜，变脸特效等。

1:1人脸识别应用方向：身份证人脸认证系统，社保人脸识别，人员身份认证，人证合一等。

1: N人脸识别应用方向：主要用于人脸检索，排查犯罪嫌疑人、失踪人口的全库搜寻、一人多证的重复排查。

M: N人脸识别应用方向：非配合动态人脸布控，白名单VIP客户管理系统，校园人脸识别系统，智能楼宇。

➤ 人脸识别

人脸识别认证测试 (LFW, 1:1认证, 2017清华99.73%)

• 同一个人:

来自: 清华大学智能图文信息处理研究室, CV-AI Lab



VS



VS



不同人:

来自: 清华大学智能图文信息处理研究室, CV-AI Lab



VS



VS



- 2005年，由公安部出入境管理局主持集成清华大学人脸技术，世界上首次在我国出入境旅客最多的深圳罗湖口岸开通“旅客自助查验通道”，日均出入境人数在数十万以上。

- 已推广到深圳、珠海两个地区的边检口岸共已开通了近400条自助通道，近300万旅客，验放旅客超过数亿人次，通过率98%，成为世界人脸识别技术大规模成功应用的范例



-人脸检测技术，在国际权威的人脸检测FDDB库上的测试结果，检测性能位居当前国际第一（2017.1）

-人脸识别技术，在国际权威的1:1测试库LFW库上实现超过99%的识别准确率，位于本领域顶级水平。

- 2017年12月，人脸识别团队在国际权威1:N人脸识别测试库MegaFace上，取得一项第一、一项第二的结果

-人脸识别技术在2017年天津第十三届全运会运动员村、湖南长沙黄花机场开放式环境下的动态人脸监控、国家金税三期工程实名制人证合一等项目中实际应用

• 获国家科技进步奖2次（二等奖、三等奖），部委科技进步奖多次

➤ 思考一个当前人脸识别的前沿课题：

（实验室承担了国家十四五课题《高仿鉴伪》）

如何对高仿真人脸进行防伪检测？

采用什么方法？

物理的？ 图像分析的？

Karras等，2017，改进质量、稳定性与变化程度的GAN网络（也被缩写为PGAN或ProGAN）



<https://blog.csdn.net/conglu1891/article/details/100311200>



https://www.sohu.com/a/356696333_354970

第3讲 机器学习

3.1 机器学习概要

3.2 感知机

3.3 回归方法与线性分类器

3.4 支持向量机SVM

机器学习的几个知识点——复习

➤ 机器学习的几个基本问题

- ◆ 分类，回归，（概率密度估计，聚类）

➤ 机器学习的基本方法

- ◆ 有监督学习，无监督学习，半监督学习

➤ 机器学习的基本模型

- ◆ 生成式，鉴别式

➤ 机器学习的准则

- ◆ 偏置，方差，欠拟合，过拟合

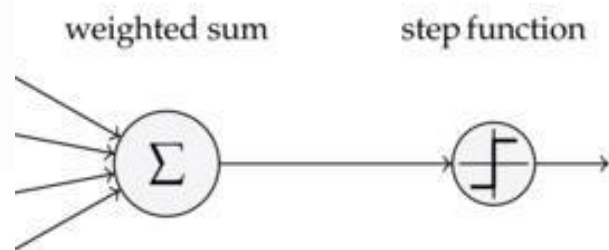
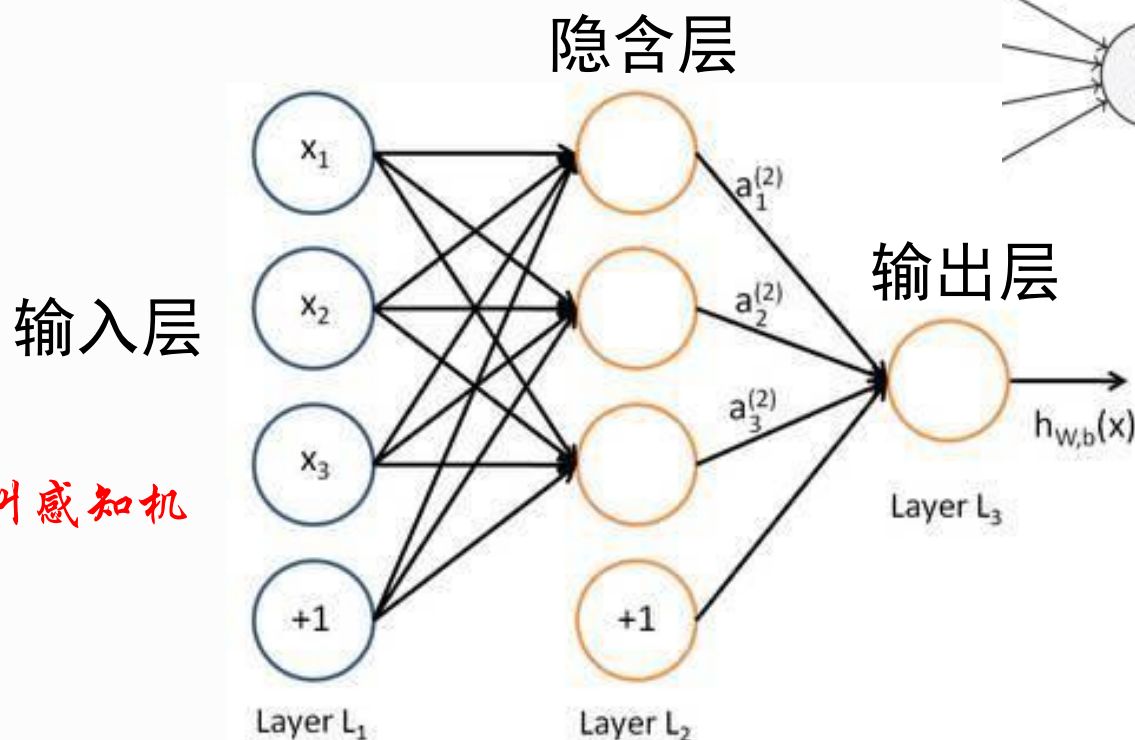
➤ 机器学习的条件

- ◆ 数据，样本，机器，评价准则

3.2 感知机(Perceptron)

➤ 什么是感知机「Perceptron」？

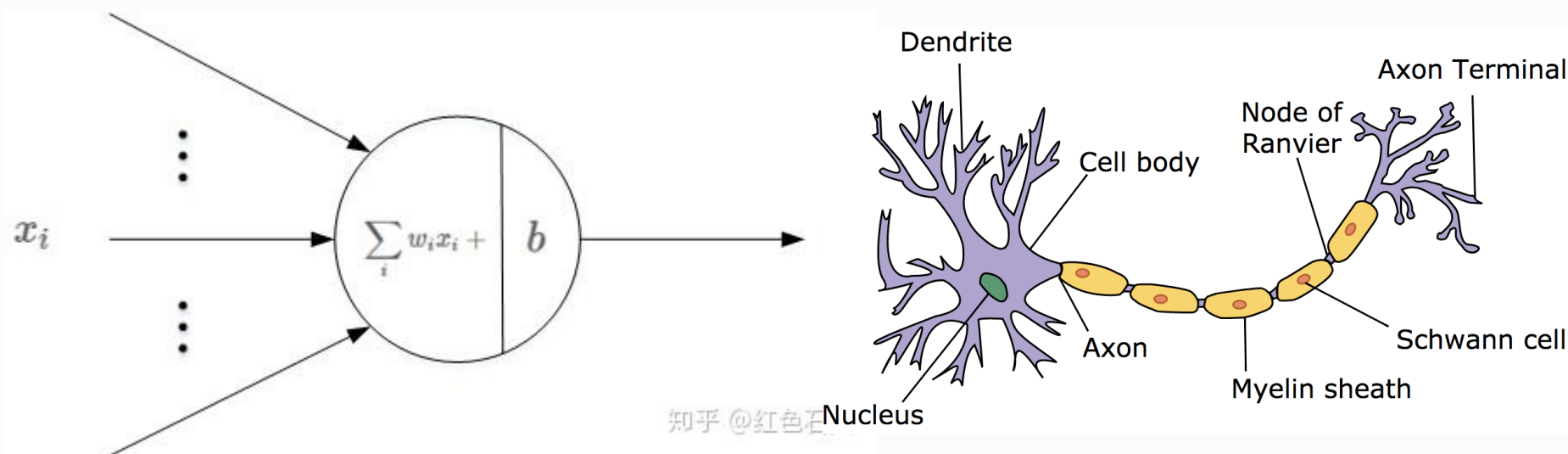
在人工智能领域，有一个方法叫机器学习。在机器学习这个方法里，有一类算法模型叫神经网络。



神经元也叫感知机

3.2 感知机(Perceptron)

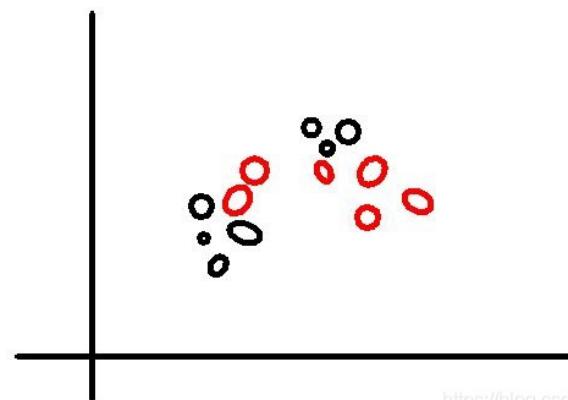
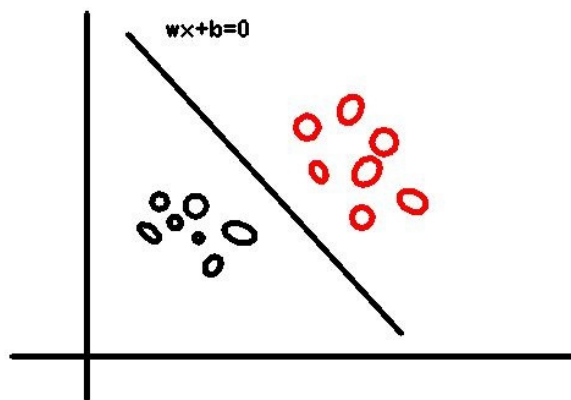
感知机算法，1957年由Rosenblatt提出，是对生物神经细胞的简单抽象。感知机是一种线性分类模型。即它针对线性可分数据。常见例即二维平面内的数据可通过一条直线分开



神经网络又称多层感知机。神经网络之所以能够发挥强大的作用就是在感知机的结构上作了叠加设计。而支持向量机算法的基础即为感知机

3.2 感知机(Perceptron)

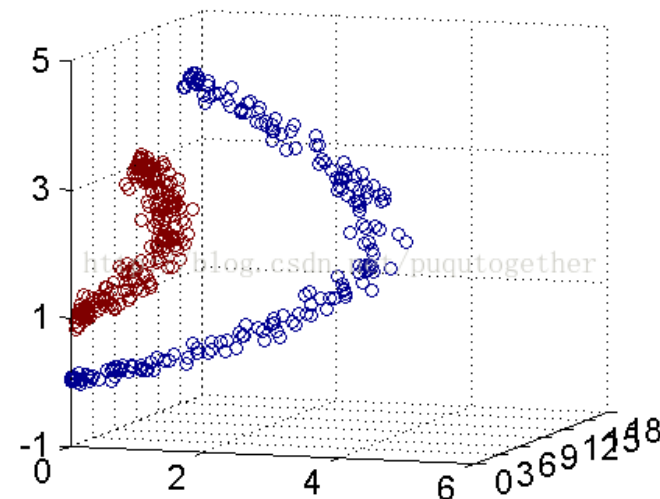
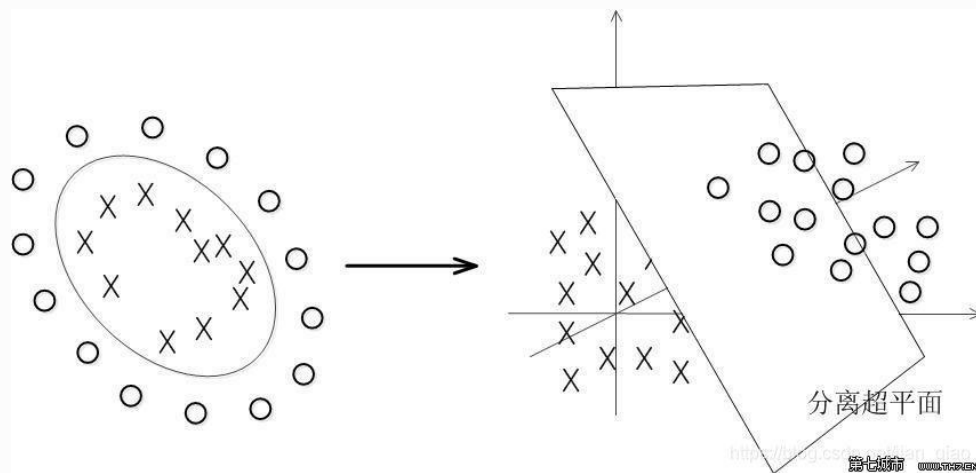
- 感知机模型，是机器学习二分类问题中一个简单的模型
- 输入样本的特征向量，输出为样本的类别代码，记为+1和-1
- 感知机对应于样本空间中的分类超平面，属于鉴别式模型
- 基于误分类样本点到分类界面距离的损失函数，利用梯度下降法，可对损失函数进行极小化



https://blog.csdn.net/jian_qiao

3.2 感知机(Perceptron)

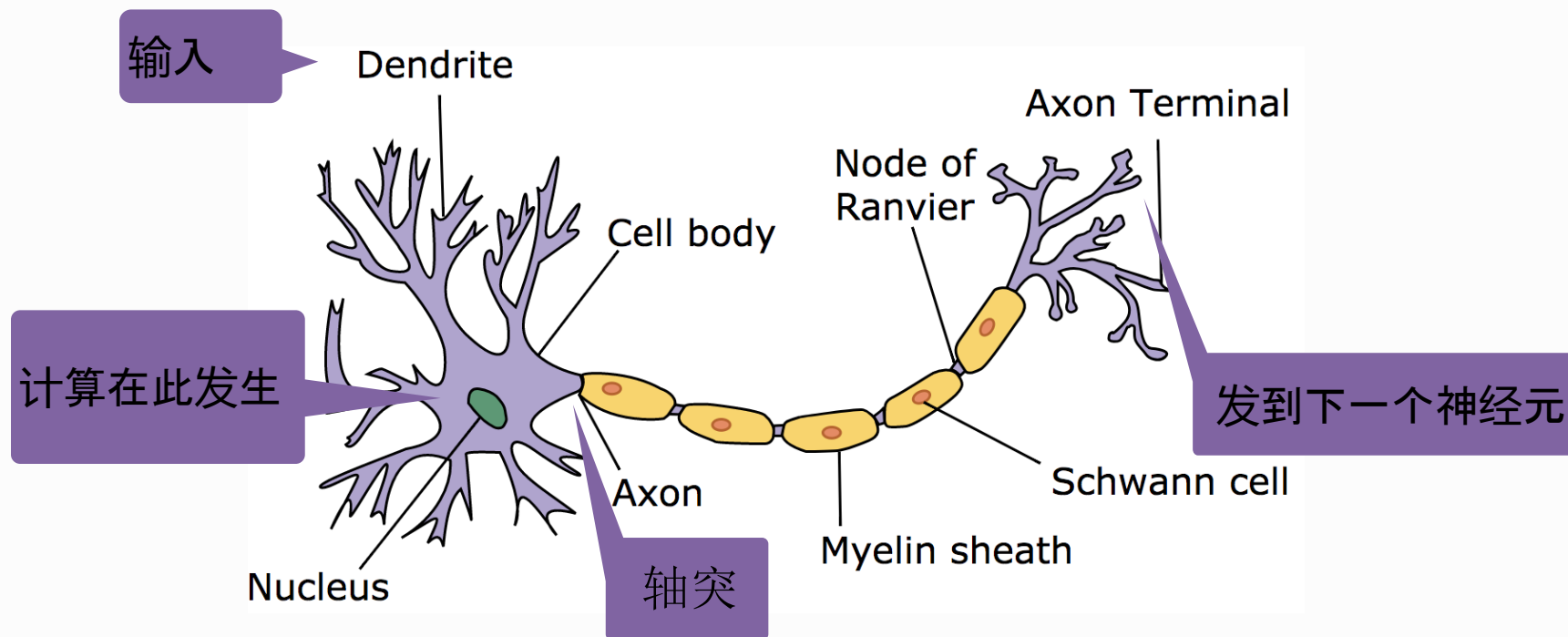
- 在二维平面上，线性可分意味着能用一条直线将正、负样本分开；
- 在三维空间中，线性可分意味着能用一个平面将正、负样本分开；
- 在 n 维空间中，线性可分意味着能用 $n-1$ 维超平面将正、负样本分开。



来自神经生理学的启发

- 人脑是由大量(100亿个以上)的神经元(Neuron)和神经胶质细胞(1000亿个以上)等组成
 - ◆ 神经元的功能是接受和传送信息。
 - ◆ 神经元与神经元通过突触建立的联系，构成了极端复杂的信息传递与加工的神经回路（nerve circuit）。神经回路是脑内信息处理的基本单位。
- 神经元内的神经冲动主要通过电传导，神经元之间的传导主要通过化学传导。

神经元的结构



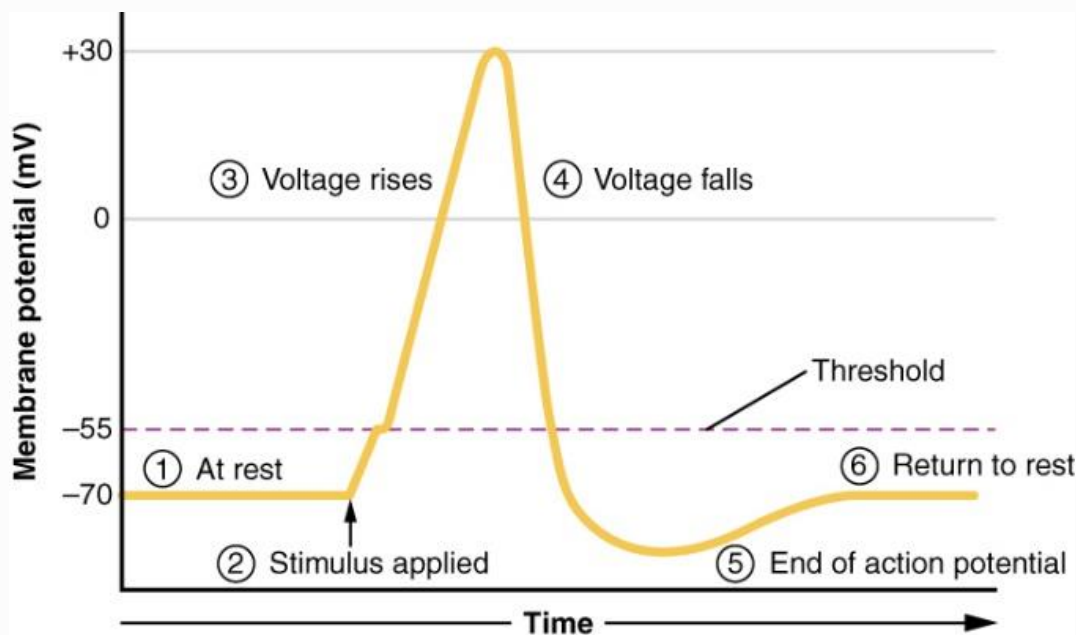
神经冲动电传导的法则

复习一遍

► 全或无的法则：

- ◆ “全或无”法则是指每个神经元都有一个刺激阈值，对阈值以下的刺激不发生反应；对阈值以上的刺激，不论其强弱均给出同样高度（幅值）的神经脉冲发放。

膜电位



神经网络中的神经元

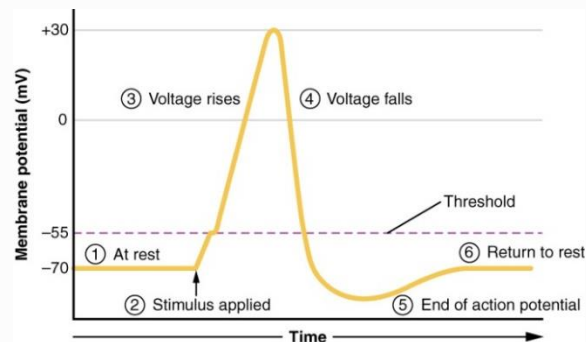
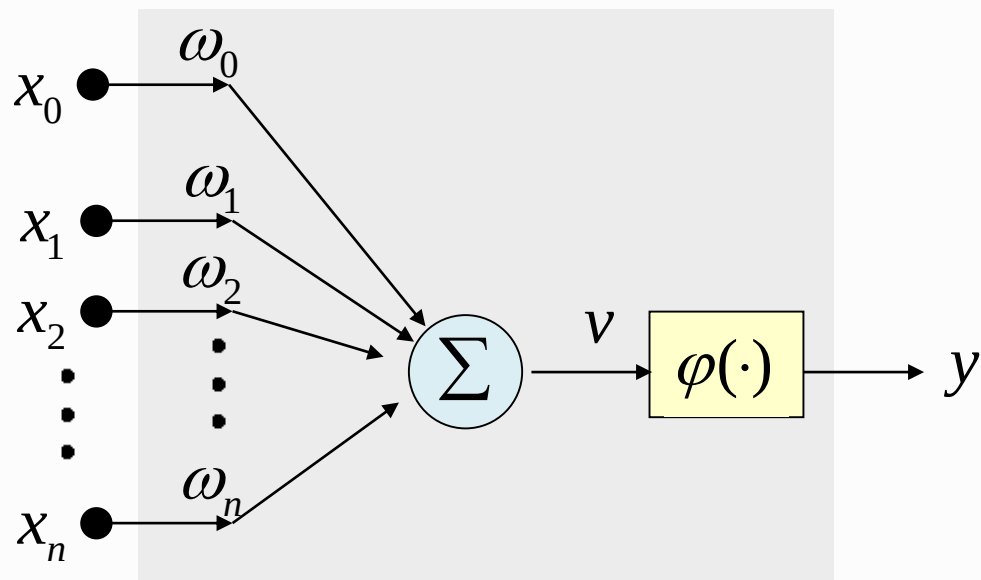
数学模型 $v = \sum_{i=1}^n \omega_i x_i + b$

$$v = \sum_{i=0}^n \omega_i x_i$$

$$x_0 = 1 \quad \omega_0 = b$$

$$y = \varphi(v)$$

神经网络中用
激活函数执行
神经脉冲功能



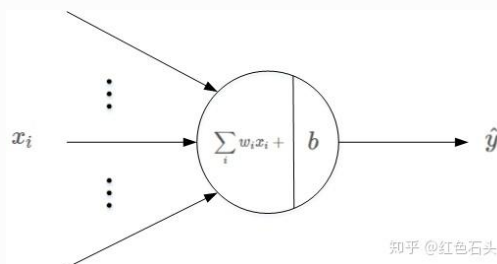
关于激活函数

激活函数出现的原因

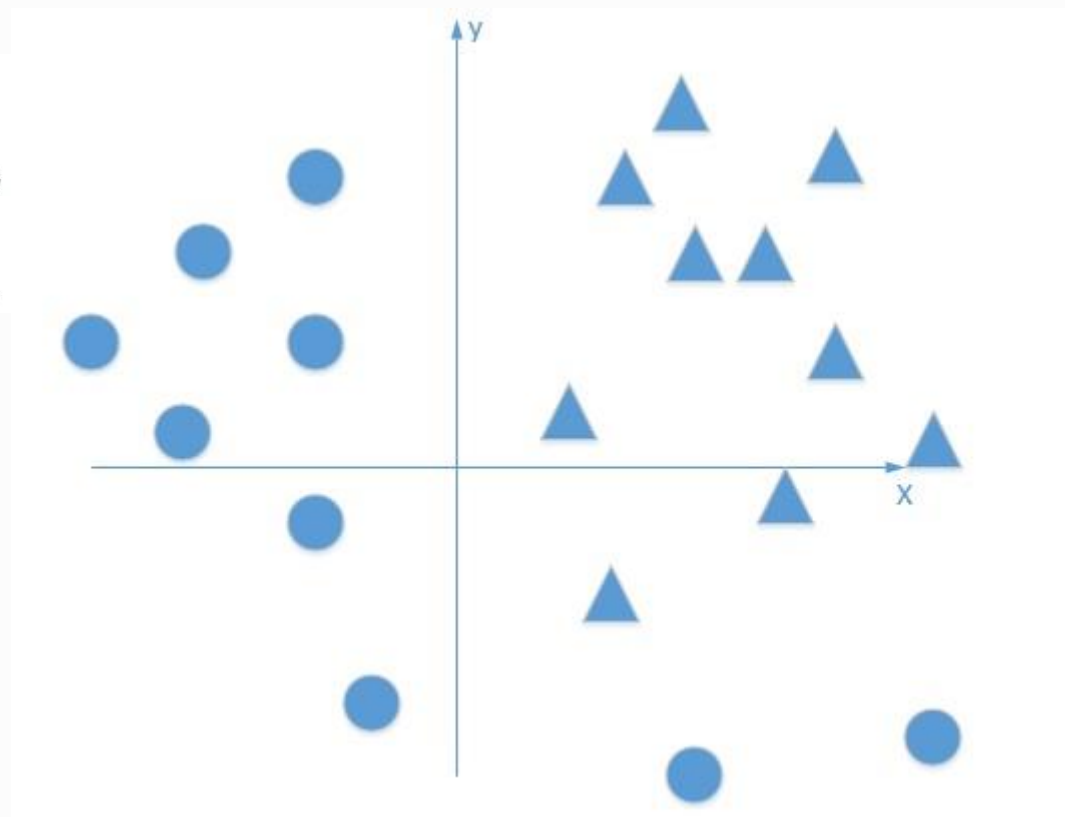
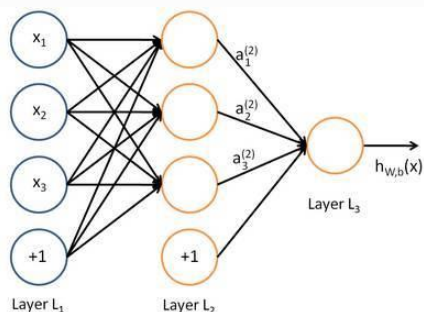
—激活函数是用来加入非线性因素的，解决线性模型所不能解决的问题

没有激活函数的分类器始终是线性组合

不带激活函数的单层感知机分割平面不能解决右侧分类

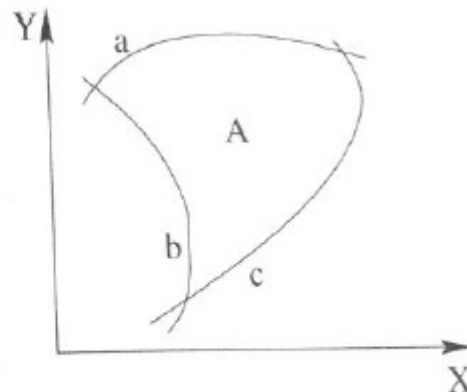
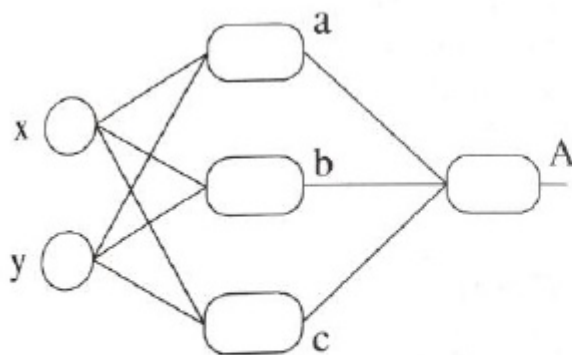
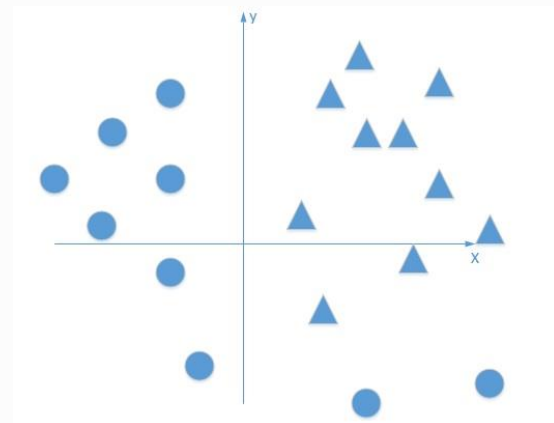
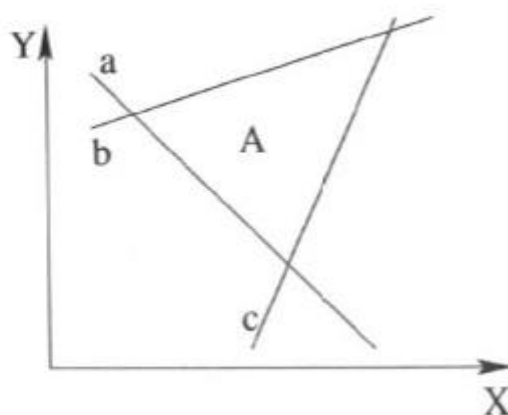
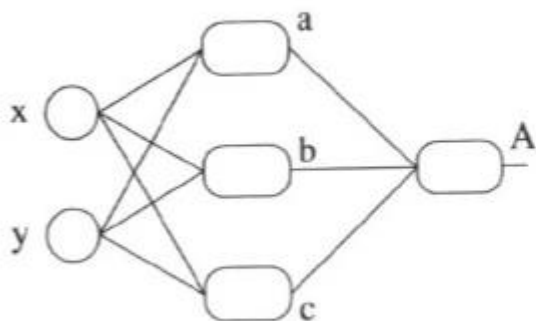


用不带激活函数的多层感知机也不能解决



关于激活函数

激活函数，是用来加入非线性因素的，解决线性模型所不能解决的问题



with sigmoid activation function

关于激活函数

a) 阈值函数: $\varphi(v) = \begin{cases} 1 & v \geq 0 \\ 0 & v < 0 \end{cases}$

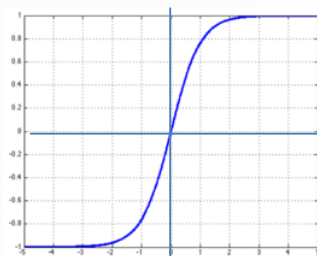
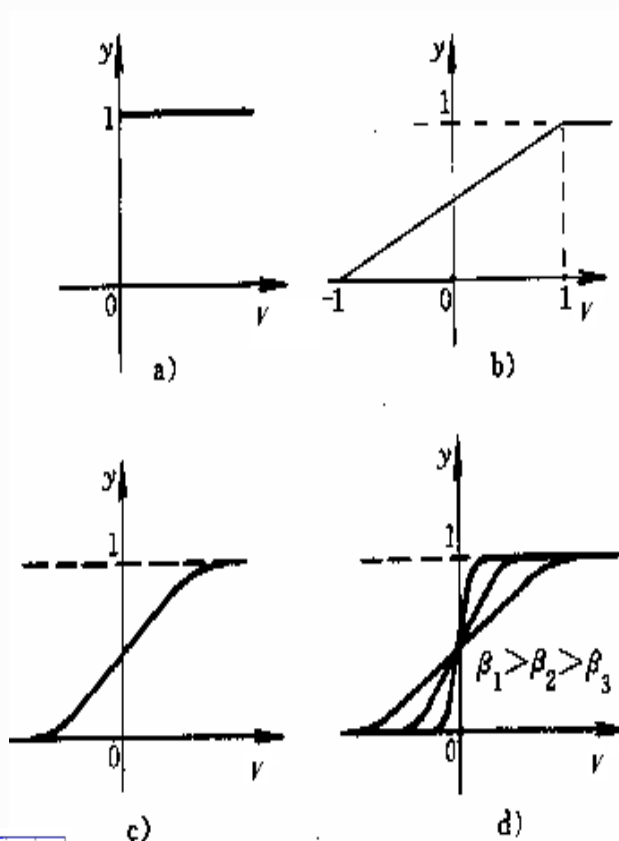
b) 分段线性函数: $\varphi(v) = \begin{cases} 1 & v \geq 1 \\ \frac{1}{2}(1+v) & -1 < v < 1 \\ 0 & v \leq -1 \end{cases}$

c) sigmoid函数: $\varphi(v) = \frac{1}{1 + \exp(-\beta v)}$

d) 双曲正切函数:

$$\varphi(v) = \tanh(\beta v) = \frac{e^{\beta v} - e^{-\beta v}}{e^{\beta v} + e^{-\beta v}}$$

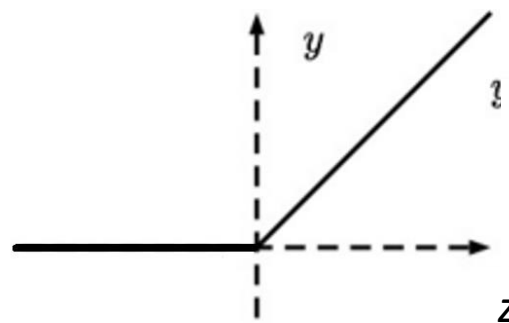
$$\tanh(x) = 2\text{sigmoid}(2x) - 1$$



关于激活函数

e) ReLU:

$$y = f(z) = \max(0, z)$$



ReLU_i

f) LeakyReLU:

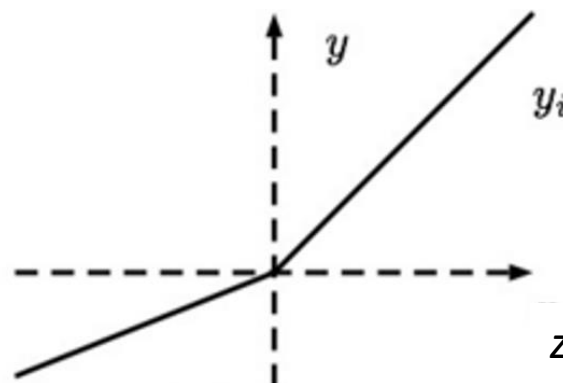
$$y = f(z) = \begin{cases} z & z \geq 0 \\ \alpha z & z < 0 \end{cases}$$

主要解决ReLU输出为0问题, α 为小常数

g) PReLU:

α 为通过学习得到的数值

(Parametric Rectified Linear Unit)



Leaky ReLU/PReLU

关于激活函数

h) Softmax

- ◆ Softmax激活函数适用于多分类任务

$$\text{softmax}(x)_i = q_i = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)}$$

它将原始分数（也称为logits）转换为表示概率分布的数值，且所有类别的概率之和等于1。因此适用于多类别分类问题，其中每个样本只属于一个类别

Softmax回归模型是logistic回归模型在多分类问题上的推广，适用于多分类问题中，**且类别之间互斥的场合**。

Softmax将多个神经元的输出，映射到（0,1）区间内，可以看成是当前输出属于各个分类的概率，从而进行多分类。

关于激活函数

h) Softmax

◆ 适用于多分类任务 $\text{softmax}(x)_i = q_i = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)}$

—**Softmax**常作为网络输出层，自然表示具有 k 个可能值的离散型随机变量的概率分布

—Softmax函数把输出映射成区间在(0,1)的值，

—并且做了归一化，

—所有元素的和累加起来等于1。

—可直接当作概率对待，选取概率最大的分类作为预测目标

关于激活函数

为什么是用Softmax Softmax的一个作用是输出一个最大值

其他很多函数也可以完成选取最大值，并归一化的功能，但是为什么现在神经网络中普遍采用Softmax作为回归分类函数呢？

选Softmax，主要因为Softmax使用了指数，这样可以让大的值更大，让小的更小，增加了区分对比度，学习效率更高。第二个是因为softmax是连续可导的，消除了拐点，这个特性在机器学习的梯度下降法等方面非常必要

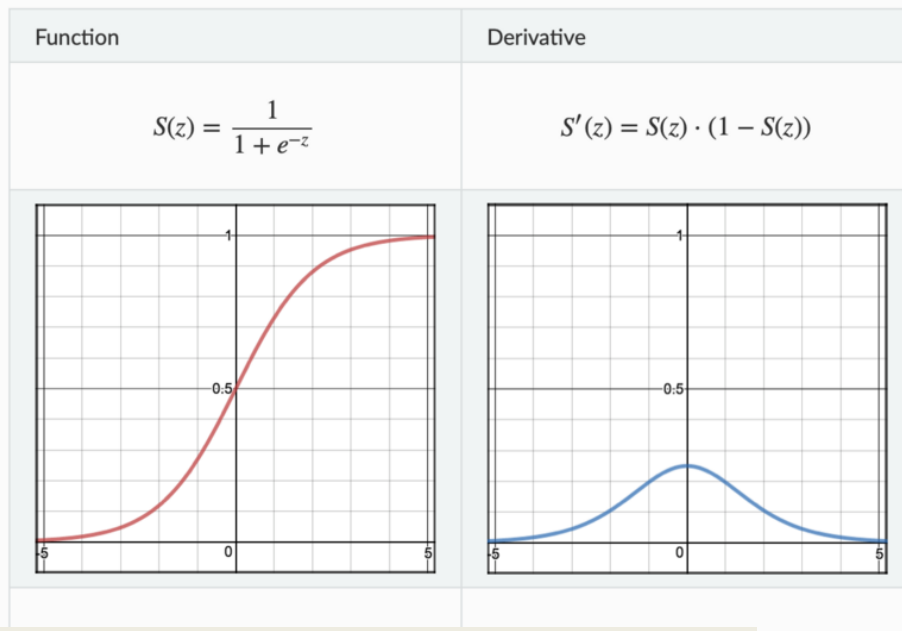
激活函数	Softmax	Sigmoid
本质	离散概率分布	非线性映射
任务	多分类	二分类
定义域	某个一维向量	单个数值
值域	$[0, 1]$	$(0, 1)$
结果之和	一定为1	为某个正数

知乎 @jie.hang

sigmoid函数：

sigmoid激活函数具有“连续可微”，“单调性”，“输出值有限”。通过查看导函数图像，**sigmoid**激活函数最大的问题就是两端饱和，造成梯度消失（解决办法：使用relu激活函数，BN等），此外输出不以0中心（以0中心的好处是可以加快模型收敛）。

目前**sigmoid**激活函数多使用在二分类问题（对于大于二分类问题，如果类别之间存在相互关系使用sigmoid，反之使用softmax）



```
1 import tensorflow as tf
2 tf.enable_eager_execution()
3
4 sigmoid_test=tf.nn.sigmoid([-3., -2., -1., 0.0, 1., 2., 3.], name= 'sigmoid_op' )
5
6 print(sigmoid_test)
```

```
1 tf.Tensor (
2 [ 0.04742587  0.11920292  0.26894143  0.5    0.7310586  0.880797
3   0.95257413 ], shape=(7,), dtype=float32 )
```

感知机定义

- 定义：假设 $x \in X$ 为样本的特征向量，特征空间 $X \subseteq R^n$ ，样本类别标号 $y \in Y$ ， $Y = \{1, -1\}$ ，将特征向量映射为类别标号的函数：

$$f(x) = \text{sign}(w \cdot x + b)$$

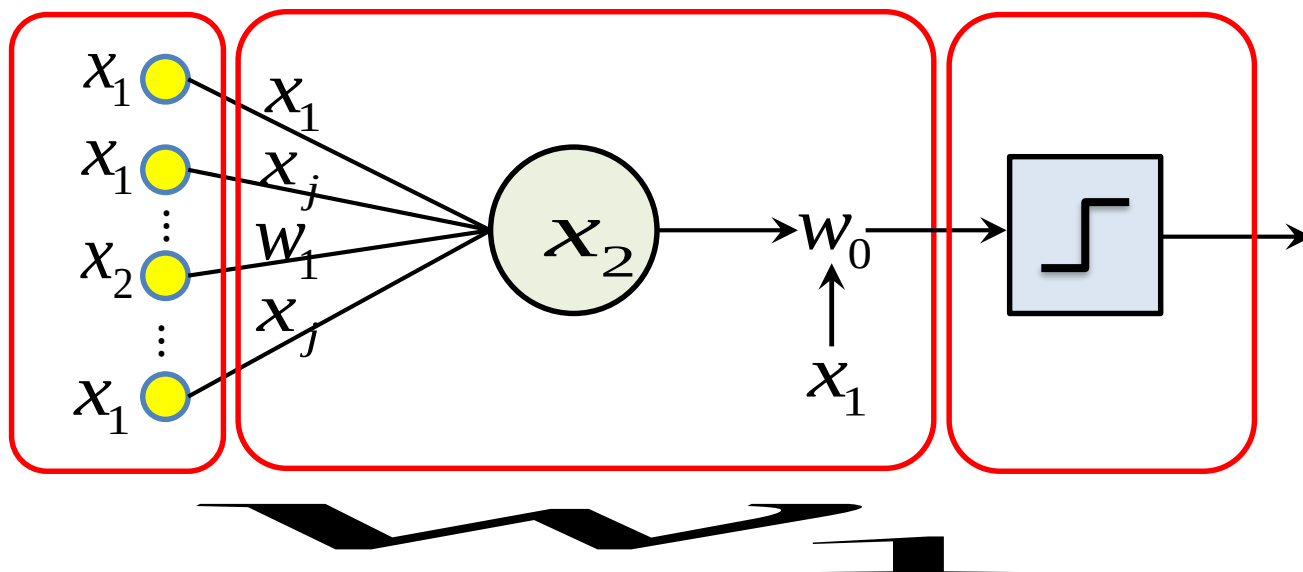
称为感知机

- ◆ 模型参数：权值向量 w ，偏置量 b
- ◆ 符号函数：

$$\text{sign}(x) = \begin{cases} 1, & x \geq 0 \\ -1, & x < 0 \end{cases}$$

➤ 感知机算法 (perceptron)

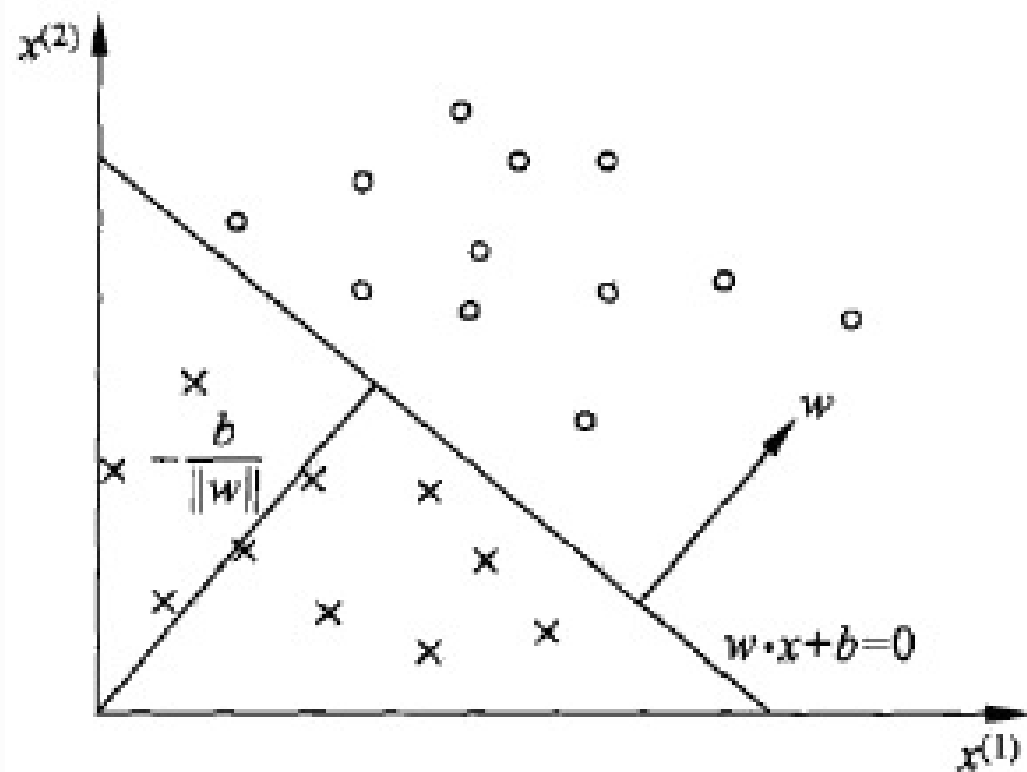
- ◆ 用于二进制分类的监督学习算法；
- ◆ Frank Rosenblatt在1957年发明的一种人工神经网络。感知机是一种最简单形式的前馈神经网络，可应用于二元线性分类。



感知机几何解释

➤ 分类界面对应于线性方程 $w \cdot x + b = 0$

➤ w 为法向量， b 为截距



$$w_1x_1 + w_2x_2 + b = 0$$

设：将大于0的分为+1类，
小于0的分为-1类

感知机学习策略

问题：如何定义损失函数？

➤ 自然选择：用误分类点的数目做损失函数，但这样的损失函数对 w, b 不可导。

➤ 一般选择为：误分类点到超平面的总距离，越小越好

点到直线距离公式： $\frac{1}{\|w\|} |w \cdot x + b|$

想想why?

误分类点： $-y_i(w \cdot x_i + b) > 0$

误分类点距离： $-\frac{1}{\|w\|} y_i(w \cdot x_i + b)$

总距离： $-\frac{1}{\|w\|} \sum_{x \in M} y_i(w \cdot x_i + b)$

因为我们设了：

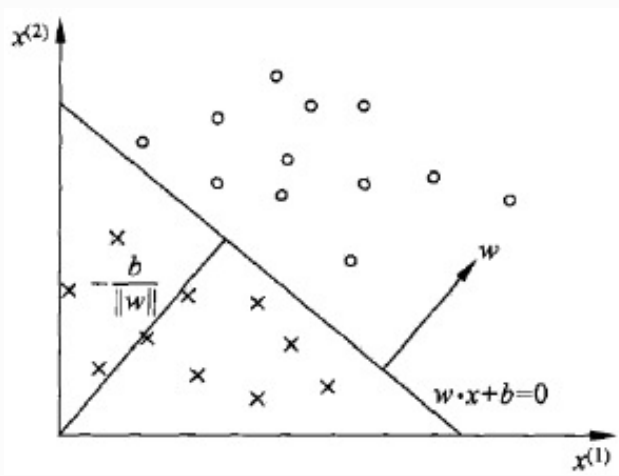
将 $(w \cdot x_i + b)$ 大于0的分为+1类，小于0的分为-1类

感知机学习策略

为什么误分类点有： $-y_i(w \cdot x_i + b) > 0$

数据点正确值为+1的时候，如果误分类了，你判断符号为-1，这时算出来是 $(wx_0+b)<0$ ，所以两者相乘满足 $-y_i(w \cdot x + b) > 0$

当数据点的正确值为-1的时候，你误分类了，那么你判断为+1，此时算出来是 $(wx_0+b>0)$ ，所以也满足 $-y_i(w \cdot x + b) > 0$



感知机学习策略

选择误分类点到超平面距离作为损失函数，越小越好

$$\text{总距离: } -\frac{1}{\|w\|} \sum_{x \in M} y_i (w \cdot x_i + b)$$

不考虑 w 范数分之一 (经过实验表明，考虑和不考虑结果是一样)，可以得到损失函数为：

$$\min_{w,b} L(w,b) = - \sum_{x \in M} y_i (w \cdot x_i + b)$$

➤ 为优化问题： $\min_{w,b} L(w,b) = - \sum_{x \in M} y_i (w \cdot x_i + b)$

◆ M 为误分类点集合

感知机学习步骤

➤ 优化问题: $\min_{w,b} L(w,b) = - \sum_{x \in M} y_i (w \cdot x_i + b)$

◆ M为误分类点集合

➤ 求损失函数梯度, 计算如下(优化时对w, b初始化):

$$\frac{\partial L(w,b)}{\partial w} = - \sum y_i x_i \quad \frac{\partial L(w,b)}{\partial b} = - \sum y_i$$

➤ 更新: 采用随机梯度下降, 每次选取一个误分类点

$$w \leftarrow [w - (-\eta y_i x_i)] \quad b \leftarrow [b - (-\eta y_i)]$$

➤ or更新: 如果采用批量梯度下降, 则使用全部误分点

$$w \leftarrow w + \eta \sum y_i x_i \quad b \leftarrow b + \eta \sum y_i$$

感知机学习步骤

➤ 输入：训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$,
其中, $x_i \in X, X \subseteq R^n, y_i \in Y = \{+1, -1\}, i = 1, 2, \dots, N$;
设定学习率 $\eta, 0 < \eta \leq 1$

➤ 输出: w, b

➤ 算法:

(1) 选取初值 w_0, b_0

(2) 在训练集中选取数据 (x_i, y_i)

(3) 如果 $y_i(w \cdot x_i + b) \leq 0$

$$w \leftarrow w + \eta y_i x_i$$

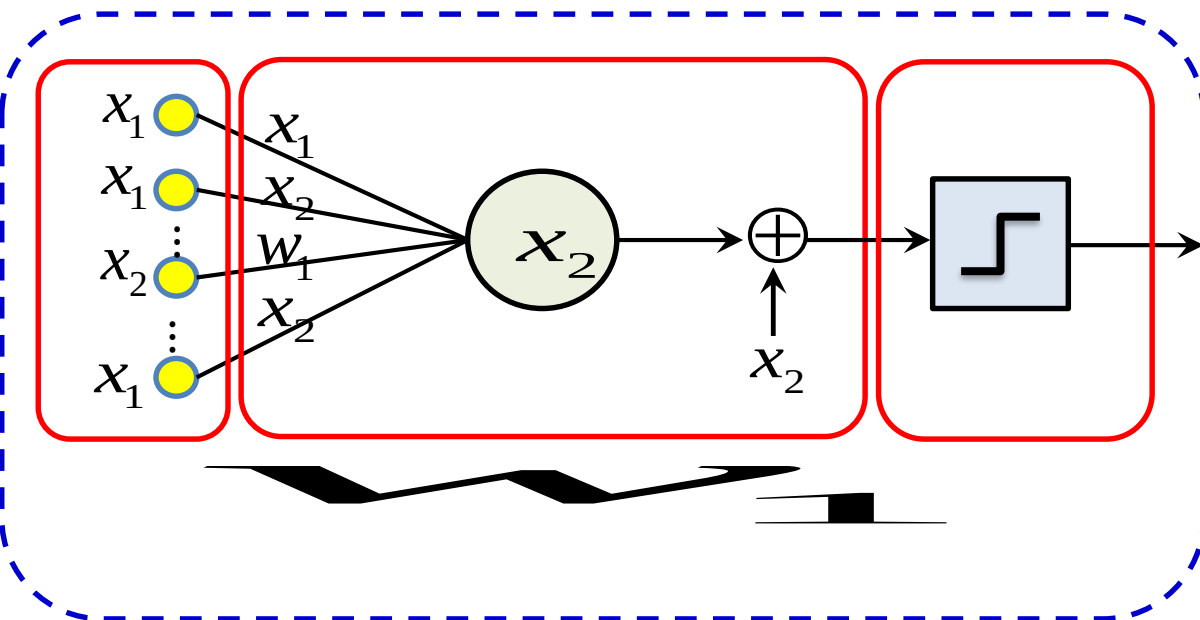
$$b \leftarrow b + \eta y_i$$

说明：采用随机梯度下降法时，感知机每一轮学习，逐点计算 $wx+b$ ，对正确分类点不更新参数，对误分类点按更新公式更新参数，然后计算下一个点。直至没有误分类点或损失函数取得极小值

上面是采用随机梯度下降法求解（一般均采用，但容易收敛到局部最优）。如果采用批量梯度下降法（BGD），则使用全部误分样本进行参数更新，即公式中的学习率后有求和符号

➤ 感知机算法总结 (perceptron)

第2步：损失函数



第1步：模型

$$\min_{\mathbf{w}, w_0} L = - \sum_{\mathbf{x}_i \in M} y_i (\mathbf{w}^T \mathbf{x}_i + w_0)$$

M 是误分类点集合

损失函数梯度：

$$\partial L / \partial \mathbf{w} = - \sum_{\mathbf{x}_i \in M} y_i \mathbf{x}_i$$

$$\partial L / \partial w_0 = - \sum_{\mathbf{x}_i \in M} y_i$$

第3步：采用随机梯度下降时，每次选一个误分类点更新，直至为0或损失函数为极小

➤ 感知机算法总结 (perceptron)

$$h(\mathbf{x}) = \text{sgn}\left(\sum_{j=1}^d w_j x_j + w_0\right) = \text{sgn}\left(\mathbf{w}^T \mathbf{x} + w_0\right)$$

感知机模型如果选择的是采用随机梯度下降法时，每次仅用一个误分类的点来更新梯度。顺序选取样本点，对正确分类点不更新，对误分类点 $(\mathbf{x}_i, y_i)$ ，更新公式为：

感知机迭代规则

1. 初始化权重和偏置项；
2. 迭代修改权重和偏置项：

$$y_i (\mathbf{w}^T \mathbf{x}_i + w_0) \leq 0 \Rightarrow \begin{aligned} w_i &\leftarrow w_i + \eta y_i x_i \\ w_0 &\leftarrow w_0 + \eta y_i \end{aligned}$$

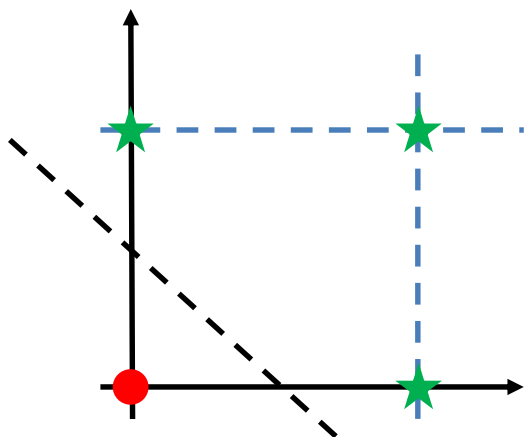
3. 直至训练集中没有误分类点。

感知机学习示例

➤ 感知机算法实现 (perceptron)

◆ 例1: 利用感知机实现 or函数。

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1



令 $w_1 = 0.5, w_2 = 0.5, w_0 = -0.3$

由观察选取感知机 w :

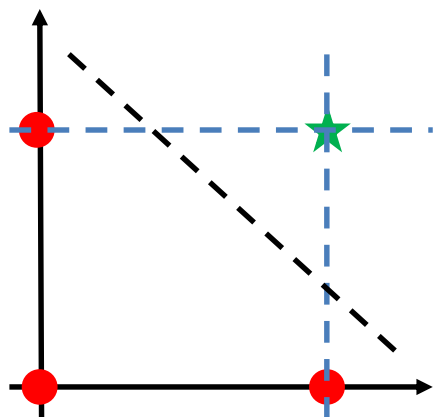
$$f(x) = \text{sign}(w_1 x_1 + w_2 x_2 + w_0)$$

感知机学习示例

➤ 感知器机法实现 (perceptron)

◆ 例2: 利用感知机实现 and函数。

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1



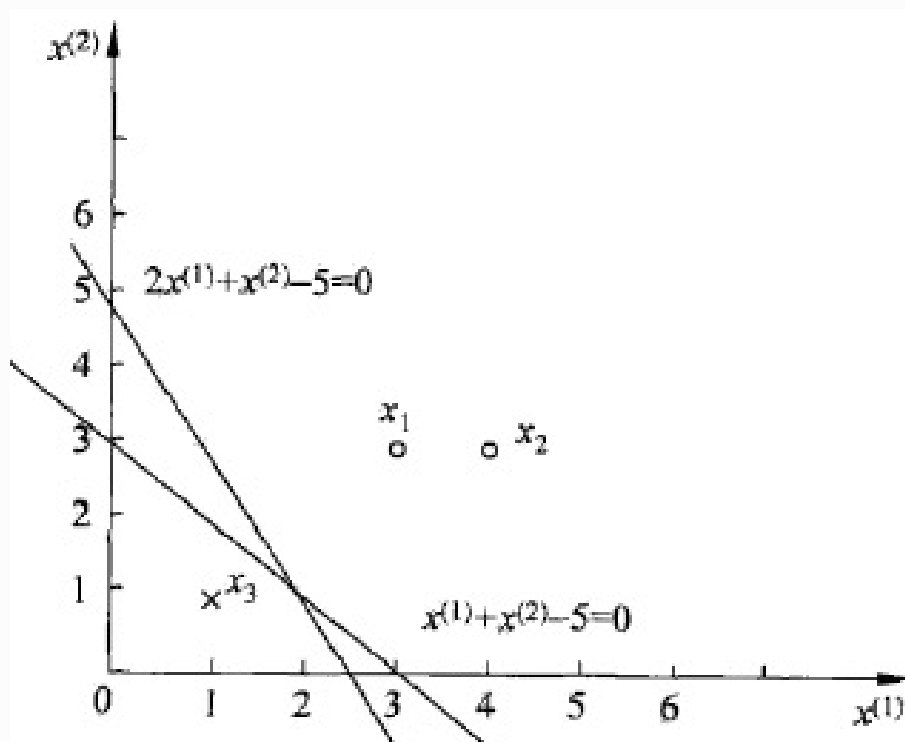
令 $w_1 = 0.5, w_2 = 0.5, w_0 = -0.8$

由观察选取感知机 w :

$$f(x) = \text{sign}(w_1 x_1 + w_2 x_2 + w_0)$$

感知机算法示例

- 例3：正类样本 $x_1 = (3,3)^T$, $x_2 = (4,3)^T$
负类样本 $x_3 = (1,1)^T$



感知机算法示例

➤ 解：构建优化问题

说明：可为写 $w \cdot x$ 或 $w^T x$

$$\min_{w,b} L(w, b) = - \sum_{x \in M} y_i (w \cdot x_i + b)$$

(1) 设学习率 $\eta=1$ ，选取初值 $w_0 = 0, b_0 = 0$

(2) 输入 $x_1 = (3,3)^T$ ， $y_i(w \cdot x_i + b)=0$ ，未能被正确分类(=0也认为未能正确分)，更新 w, b ，

$$w_1 = w_0 + y_1 x_1 = (3,3)^T, b_1 = b_0 + y_1 = 1$$

得到模型 $w \cdot x + b = (3,3)^T x + 1$

(3) 如此迭代，最终得到 $w_7 = (1,1)^T, b_7 = -3$

迭代过程

迭代次数	误分类点	W	b
0		0	0
1	x_1	$(3,3)^T$	1
2	x_3	$(2,2)^T$	0
3	x_3	$(1,1)^T$	-1
4	x_3	$(0,0)^T$	-2
5	x_2	$(3,3)^T$	-1
6	x_3	$(1,1)^T$	-2
7	x_3	$(1,1)^T$	-3
8	-	$(1,1)^T$	-3

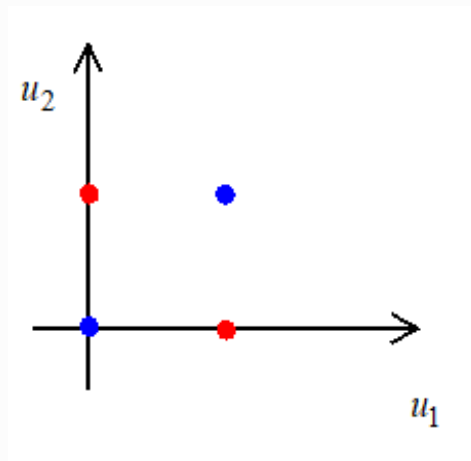
收敛

异或 (XOR) 问题讨论

➤ 二维平面上的两类模式——异或 (exclusive OR, XOR) 问题

- ◆ 二维平面中不存在一条直线，将该输入模式分为两类，此输入模式称线性不可分集合
- ◆ 可见：单个感知机不能解决异或问题

u_1	u_2	y
0	0	0
0	1	1
1	0	1
1	1	0



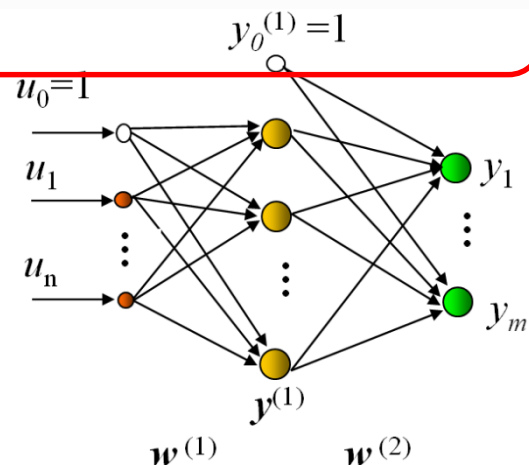
Q2: 感知器是否能实现异或运算？

A2: 本质是线性模型，解决不了异或 (XOR) 问题；其理论研究宣判了人工神经网络的第一次严冬 (1974-1980)。

感知机小结

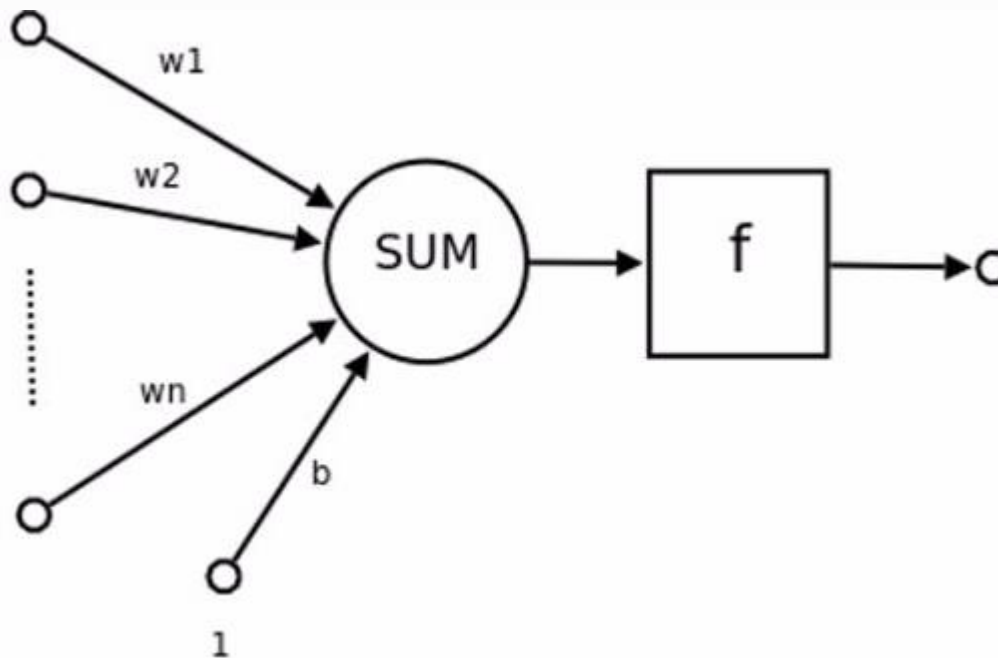
- 当训练数据集**线性可分**时，感知机学习算法是收敛的。对于线性不可分数据集，迭代过程振荡。
- 感知机算法存在许多解，既依赖于初值，也依赖迭代过程中误分类点的选择顺序。
- 为得到唯一分离超平面，需要增加约束，如后续课程中介绍的支持向量机。
- 多层感知机具有更好的模型描述能力。

问题：感知机结果唯一吗？



讨论与思考

► 如何对感知机算法进行拓展？多感知机？



3.1 机器学习概要

3.2 感知机

3.3 回归方法与线性分类器

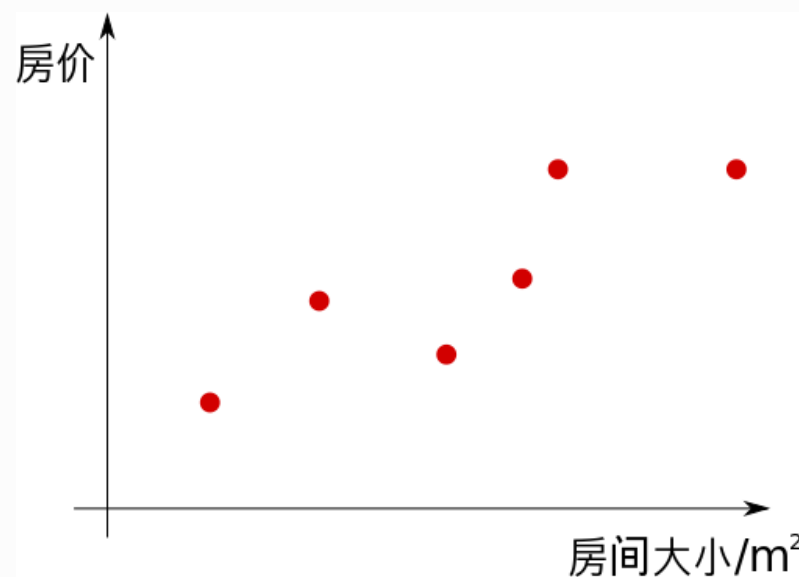
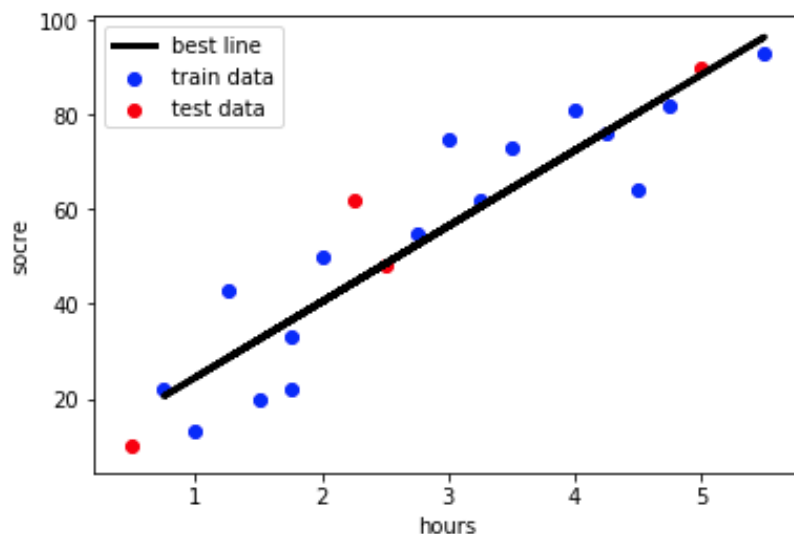
3.4 支持向量机SVM

3.3 回归方法与线性分类器

从线性回归到线性分类——回归

机器学习可以解决很多问题，其中最为重要的两个是回归与分类。这两个问题怎么解决，它们之间又有什么区别呢？

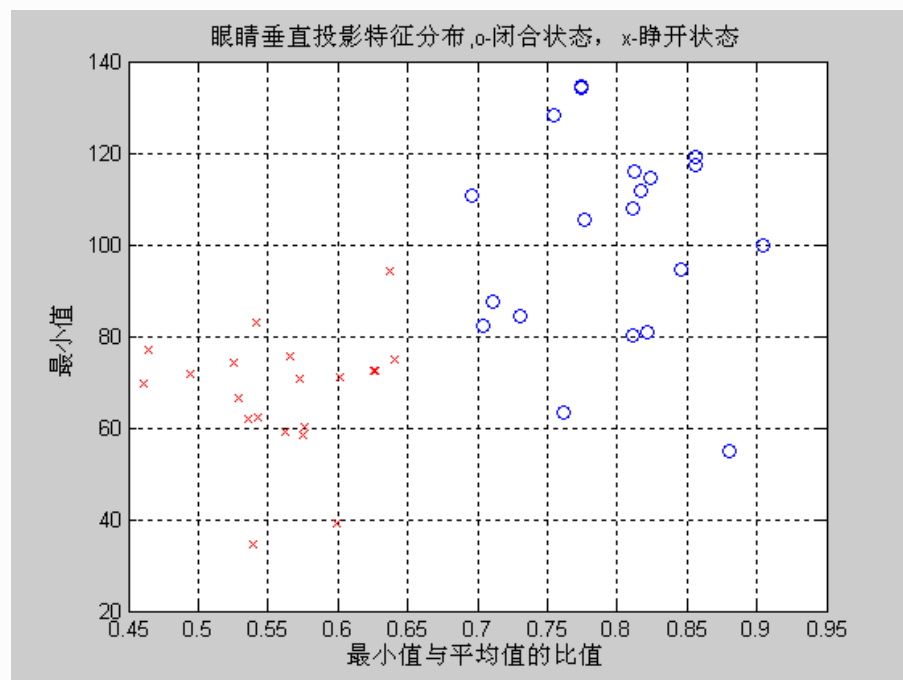
说明如何建立学习时间和
考试分数之间的预测关系。



3.3 回归方法与线性分类器

从线性回归到线性分类——分类

机器学习可以解决很多问题，其中最为重要的两个是 回归与分类。这两个问题怎么解决，它们之间又有什么区别呢？



3.3 回归方法与线性分类器

1、线性回归

线性回归的三个特点。突破线性回归一个或几个特点，则形成统计学习中的众多模型

(1) 线性：

a. 关于特征 x 是线性的。

b. 关于参数 w 是线性的。

c. 从全局看线性组合之后直接输出结果，无任何处理。

(2) 全局性：对所有的特征空间拟合，不是将特征分段拟合。

(3) 数据未加工：没有降维映射等。

突破 a，就有特征转换，比如多项式回归

突破 b，就有了神经网络

突破 c，就有了线性分类，比如加激活函数等。

突破 (2)，就有决策树，对特征空间进行划分，最后划分成很多小区域。

突破 (3)，就有了PCA，流形学习等。

3.3 回归方法与线性分类器

➤ 输入: $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$

➤ 线性方法有 n 个权重系数和偏置量:

$$\mathbf{w} = [w_1, w_2, \dots, w_n]^T, b$$

➤ 输出是输入的加权总和:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

矢量化版本:

$$y = \langle \mathbf{w}, \mathbf{x} \rangle + b$$

$$y = \mathbf{w} \cdot \mathbf{x} + b$$

$$y = \mathbf{w}^T \mathbf{x} + b$$

感知机模型分类: 符号函数

$$\text{sign}(x) = \begin{cases} 1, & x \geq 0 \\ -1, & x < 0 \end{cases}$$

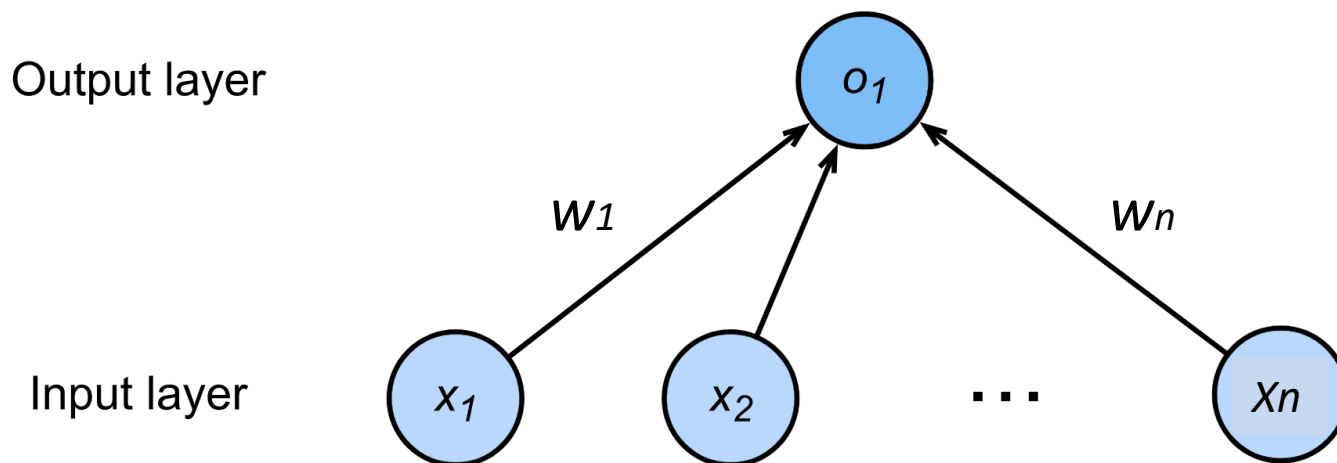
感知机模型回归: 如下函数

$$f(x) = y$$

$$f(x) = \mathbf{w}^T \mathbf{x} + b \quad 97$$

线性回归方法是一个单层网络

➤ 感知机模型解决回归问题



o_1

$$y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

训练数据集构建

➤ 收集多个数据点作为训练数据集

- ◆ 如近几个月的房价 y ，影响房价的因素为 x
- ◆ 又如前面提到的，学习成绩 y ，影响成绩的因素 x

➤ 假设有 n 个数据

$$\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_n]^T \quad \mathbf{y} = [y_0, y_1, \dots, y_n]^T$$

x, y 都是数值

学习回归参数

➤ 定义损失函数

- ◆ 最小二乘法：找到一条直线，使所有样本到直线上预测点的均方误差最小。

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \langle \mathbf{x}_i, \mathbf{w} \rangle - b)^2$$

➤ 求解

$$\mathbf{w}^*, \mathbf{b}^* = \arg \min_{\mathbf{w}, b} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b)$$

求闭式解

➤ 将偏置量添加到权重中，得到增广向量

$$\mathbf{X} \leftarrow [\mathbf{X}, \mathbf{1}] \quad \mathbf{w} \leftarrow \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$$

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \quad \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = -\frac{2}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X}$$

➤ 损失函数是凸函数，因此最优解满足：

$$\frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = 0$$

$$\Leftrightarrow \frac{2}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X} = 0$$

$$\Leftrightarrow \mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

利用优化方法求解：误差反向传播

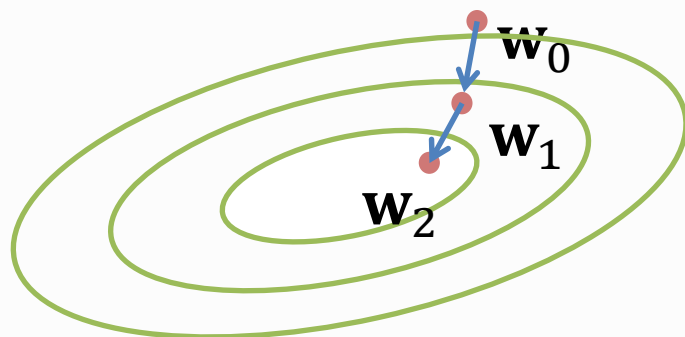
➤ 选择一个起点 $\mathbf{w}_0$

➤ 梯度下降法调整参数，多次迭代

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$$

◆ 梯度：更新权重的方向

◆ 学习率：一个超参数指定更新参数的强度



线性回归方法小结

- 问题：拟合线性数据
- 模型： $y = \langle \mathbf{w}, \mathbf{x} \rangle + b$
- 目标函数：均方误差或误差平方和
- 小批量随机梯度下降法（mini-batch SGD）
 - ◆ 参数初始化
 - ◆ 迭代
 - 计算梯度
 - 更新参数

讨论

► 试分析什么情况下不用考虑下式中的偏置量**b**?

$$y = \mathbf{w}^T \mathbf{x} + b$$

类比于平面直线方程，**b**相当于自变量取值为0，因变量的值。因此，当只需要考虑**x**的取值对**y**的影响的话，则可以不用考虑**b**

3.3 回归方法与线性分类器

2、逻辑回归 Logistic Regression

逻辑回归也称逻辑回归分析，是一种广义线性回归分析模型，常用于目标检测、数据挖掘，疾病诊断等领域。

实质上多用于分类任务。

例如：以病情分析为例，因变量 y 是否有病，值为“是”或“否”，而自变量可包括很多因素，如年龄、性别、饮食习惯、感染等。自变量既可以是连续，也可以是离散。

在病情分析中，通过logistic回归分析，可以得到自变量的权重，从而可以大致了解到底哪些因素是致病因素。

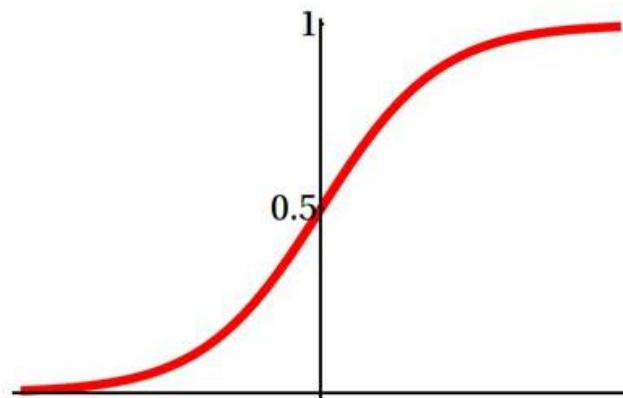
2、逻辑回归 Logistic Regression

模型也具有 $\mathbf{w}\mathbf{x}+\mathbf{b}$ ，其区别在于因变量不同，线性回归直接将 $\mathbf{w}\mathbf{x}+\mathbf{b}$ 作为因变量，而logistic回归则通过函数 L 将 $\mathbf{w}\mathbf{x}+\mathbf{b}$ 对应一个隐状态 p ， $p = L(\mathbf{w}\mathbf{x}+\mathbf{b})$ ，根据 p 值决定因变量值。

虽然被称为回归，但其实际上是分类模型，常用于二分类，估计可能性大小。线性模型只返回实数，需要逻辑回归转换为概率

$$h(\mathbf{x}_i) = \frac{1}{1 + \exp\left(-\left(\mathbf{w}^T \mathbf{x}_i + w_0\right)\right)}$$

逻辑函数也称sigmoid函数，将逻辑回归也可看作感知机Perceptron，不同的是使用了sigmoid激活函数



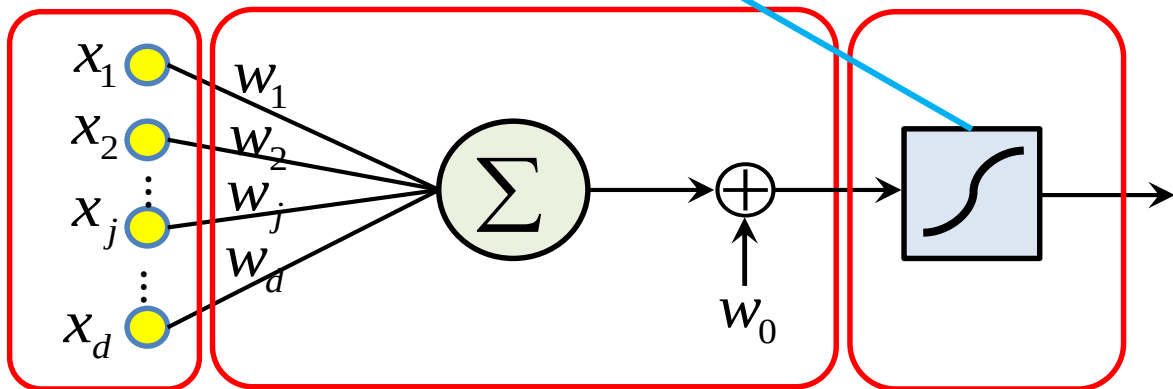
3.3 回归方法与线性分类器

逻辑函数，sigmoid函数

$$h(\mathbf{x}_i) = \frac{1}{1 + \exp(-(\mathbf{w}^T \mathbf{x}_i + w_0))}$$

定义LR损失函数为交叉熵
损失函数 Cross-Entropy
Loss

$$\min_{\mathbf{w}, w_0} L = - \sum_i \left[y_i \log(h(\mathbf{x}_i)) + (1 - y_i) \log(1 - h(\mathbf{x}_i)) \right]$$



交叉熵损失函数

交叉熵函数表征真实样本标签和预测概率之间的差值

逻辑函数也称sigmoid函数，将逻辑回归也可看作感知机Perceptron, 不同的是使用了sigmoid激活函数

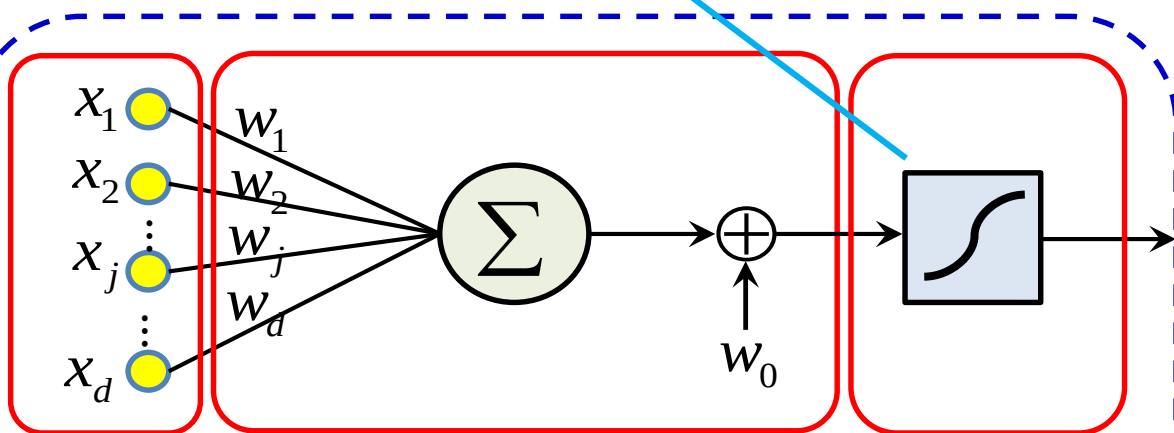
3.3 回归方法与线性分类器

➤ 逻辑回归算法步骤

- ◆ 解决二分类问题机器学习方法，用于估计某种事物可能性

$$h(\mathbf{x}_i) = \frac{1}{1 + \exp(-(\mathbf{w}^T \mathbf{x}_i + w_0))} \quad \min_{\mathbf{w}, w_0} L = - \sum_i \left[y_i \log(h(\mathbf{x}_i)) + (1 - y_i) \log(1 - h(\mathbf{x}_i)) \right]$$

交叉熵函数表征真实样本标签和预测概率之间的差值



$$h(\mathbf{x}) = \text{Sigmoid} \left(\sum_{j=1}^d w_j x_j + w_0 \right)$$

$$= \text{Sigmoid}(\mathbf{w}^T \mathbf{x} + w_0)$$

第1步：模型

交叉熵损失函数

第2步：损失函数

优化方法：梯度下降

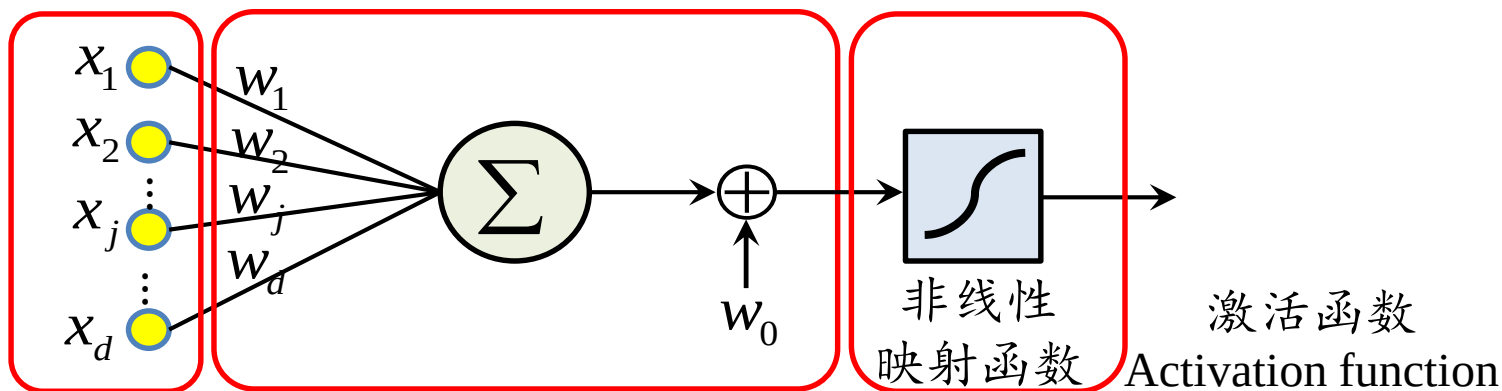
$$\partial L / \partial \mathbf{w} = \sum_i (h(\mathbf{x}_i) - y_i) \mathbf{x}_i$$

$$\partial L / \partial w_0 = \sum_i (h(\mathbf{x}_i) - y_i)$$

第3步：搜索策略

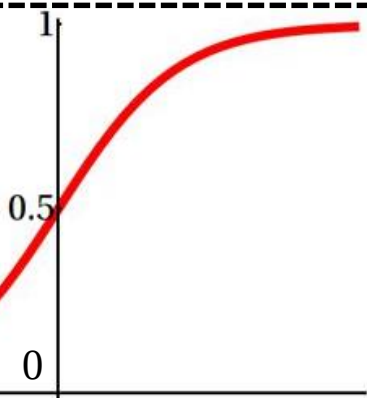
3.3 回归方法与线性分类器

➤ 逻辑回归算法步骤



$$h(\mathbf{x}) = \text{Sigmoid} \left(\sum_{j=1}^d w_j x_j + w_0 \right) = \text{Sigmoid} \left(\mathbf{w}^T \mathbf{x} + w_0 \right) = \frac{1}{1 + \exp \left(- \left(\mathbf{w}^T \mathbf{x}_i + w_0 \right) \right)}$$

Sigmoid



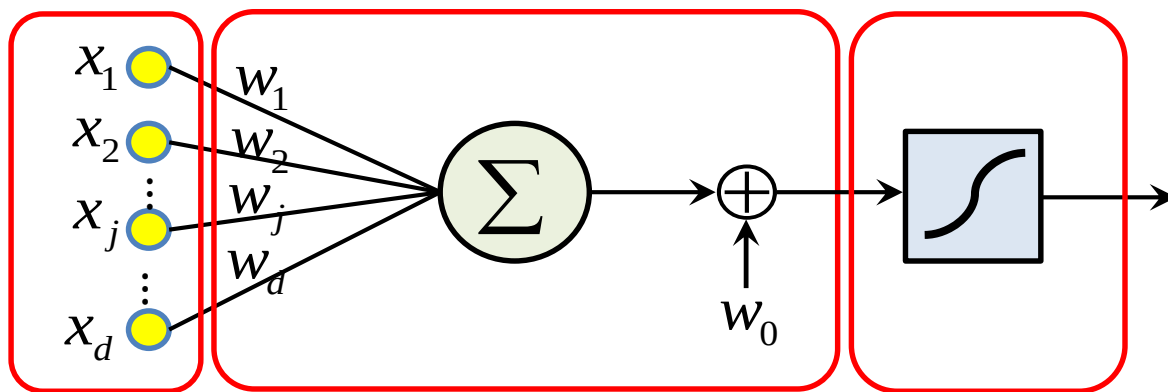
$$\mathbf{w}^T \mathbf{x} + w_0 > 0 \Rightarrow \text{Sigmoid}(\mathbf{w}^T \mathbf{x} + w_0) > 0.5$$

$$\mathbf{w}^T \mathbf{x} + w_0 < 0 \Rightarrow \text{Sigmoid}(\mathbf{w}^T \mathbf{x} + w_0) < 0.5$$

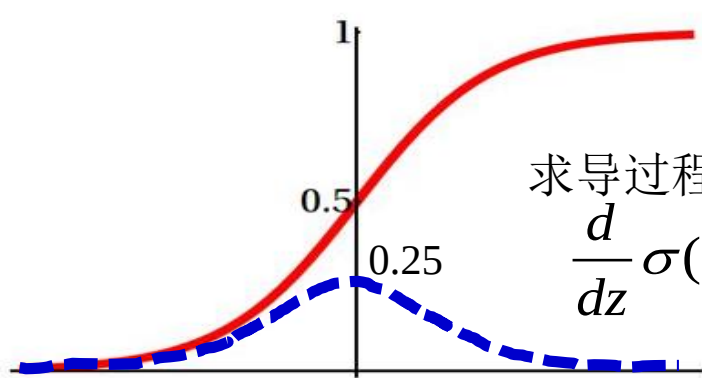
$$0 < h(\mathbf{x}) < 1$$

3.3 回归方法与线性分类器

➤ 逻辑回归函数导数



$$h(\mathbf{x}) = \text{Sigmoid} \left(\sum_{j=1}^d w_j x_j + w_0 \right) = \text{Sigmoid} \left(\mathbf{w}^T \mathbf{x} + w_0 \right) = \frac{1}{1 + \exp \left(- \left(\mathbf{w}^T \mathbf{x}_i + w_0 \right) \right)}$$



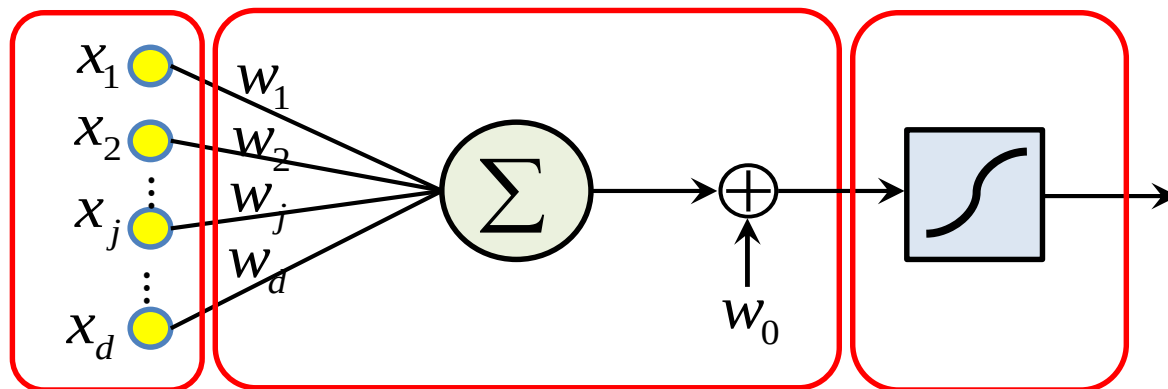
$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \frac{d}{dz} \sigma(z) = \sigma(z) \cdot (1 - \sigma(z))$$

求导过程

$$\frac{d}{dz} \sigma(z) = \frac{-1 \cdot (-e^{-z})}{(1 + e^{-z})^2} = \frac{e^{-z}}{1 + e^{-z}} \cdot \frac{1}{1 + e^{-z}} = \sigma(z) \cdot (1 - \sigma(z))$$

3.3 回归方法与线性分类器

► 考察损失函数



$$h(\mathbf{x}_i) = \frac{1}{1 + \exp(-(\mathbf{w}^T \mathbf{x}_i + w_0))}$$

$$\min_{\mathbf{w}, w_0} L = - \sum_i \left[y_i \log(h(\mathbf{x}_i)) + (1 - y_i) \log(1 - h(\mathbf{x}_i)) \right], y_i \in \{0, 1\}$$

定义LR损失函数为交叉熵损失函数 Cross-Entropy Loss

交叉熵函数表征真实样本标签和预测概率之间的差值

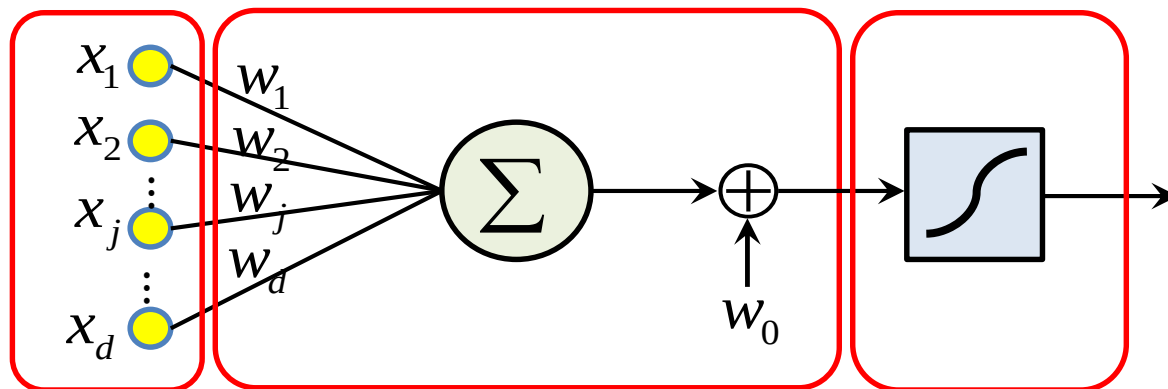
考察 $L_i = - \left[y_i \log(h(\mathbf{x}_i)) + (1 - y_i) \log(1 - h(\mathbf{x}_i)) \right]$

当 $y_i = 1 \Rightarrow L_i = -\log(h(\mathbf{x}_i)) \Rightarrow \mathbf{w}^T \mathbf{x} + w_0$ 越大, $h(\mathbf{x}_i) \rightarrow 1, L_i \rightarrow 0$

$y_i = 0 \Rightarrow L_i = -\log(1 - h(\mathbf{x}_i)) \Rightarrow \mathbf{w}^T \mathbf{x} + w_0$ 越小, $h(\mathbf{x}_i) \rightarrow 0, L_i \rightarrow 0$

右侧分析表明:
 $\mathbf{w}\mathbf{x} + w_0$ 越接近
真实标签, 损失函数越接近0

3.3 回归方法与线性分类器



$$h(\mathbf{x}_i) = \frac{1}{1 + \exp(-(\mathbf{w}^T \mathbf{x}_i + w_0))}$$

$$\min_{\mathbf{w}, w_0} L = - \sum_i \left[y_i \log(h(\mathbf{x}_i)) + (1 - y_i) \log(1 - h(\mathbf{x}_i)) \right], y_i \in \{0, 1\}$$

定义LR损失函数为交叉熵损失函数 Cross-Entropy Loss

交叉熵函数表征
真实样本标签和
预测概率之间的
差值

3.3 回归方法与线性分类器

定义LR损失函数为交叉熵损失函数 Cross-Entropy Loss

$$h(\mathbf{x}_i) = \frac{1}{1 + \exp(-(\mathbf{w}^T \mathbf{x}_i + w_0))}$$

考察
$$L_i = -\left[y_i \log(h(\mathbf{x}_i)) + (1 - y_i) \log(1 - h(\mathbf{x}_i)) \right]$$

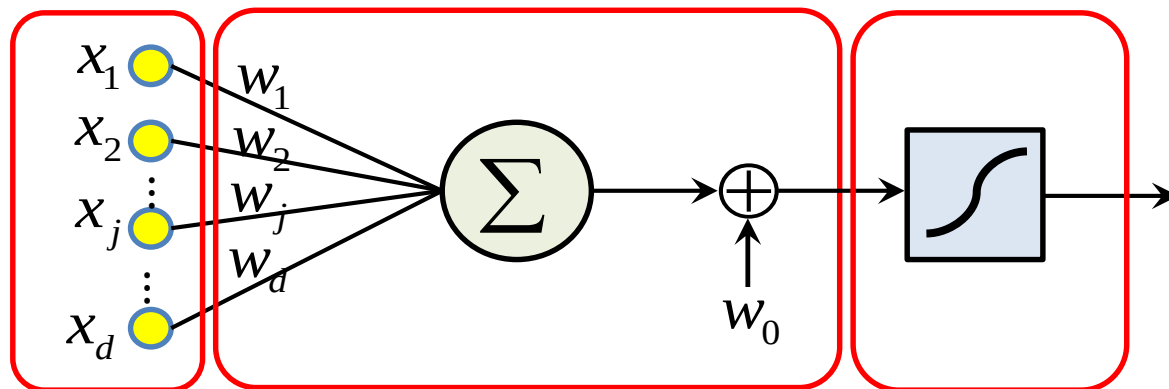
当 $y_i = 1 \Rightarrow L_i = -\log(h(\mathbf{x}_i)) \Rightarrow \mathbf{w}^T \mathbf{x} + w_0$ 越大, $h(\mathbf{x}_i) \rightarrow 1, L_i \rightarrow 0$

$y_i = 0 \Rightarrow L_i = -\log(1 - h(\mathbf{x}_i)) \Rightarrow \mathbf{w}^T \mathbf{x} + w_0$ 越小, $h(\mathbf{x}_i) \rightarrow 0, L_i \rightarrow 0$

右侧分析表明： $\mathbf{w}\mathbf{x} + \mathbf{w}_0$ 越接近真实标签（实际是 $h(\mathbf{x})$ 接近真实标签），损失函数越接近0

3.3 回归方法与线性分类器

➤ 逻辑回归参数求解——梯度下降



$$\min_{\mathbf{w}, w_0} L = - \sum_i \left[y_i \log(h(\mathbf{x}_i)) + (1 - y_i) \log(1 - h(\mathbf{x}_i)) \right], y_i \in \{0, 1\}$$

$$\frac{\partial L}{\partial h(\mathbf{x}_i)} = - \frac{y_i}{h(\mathbf{x}_i)} + \frac{1 - y_i}{1 - h(\mathbf{x}_i)}$$

可推导出

$$\frac{\partial h(\mathbf{x}_i)}{\partial \mathbf{w}} = h(\mathbf{x}_i)(1 - h(\mathbf{x}_i)) \mathbf{x}_i \quad \Rightarrow \quad \frac{\partial L}{\partial \mathbf{w}} = \sum_i (h(\mathbf{x}_i) - y_i) \mathbf{x}_i$$
$$\frac{\partial h(\mathbf{x}_i)}{\partial w_0} = h(\mathbf{x}_i)(1 - h(\mathbf{x}_i)) \quad \Rightarrow \quad \frac{\partial L}{\partial w_0} = \sum_i (h(\mathbf{x}_i) - y_i)$$

3.3 回归方法与线性分类器

► 逻辑回归参数求解——梯度下降

推导过程：

$$g(\mathbf{x}, \mathbf{w}, w_0) = \mathbf{w}^T \mathbf{x} + w_0$$

$$h(\mathbf{x}) = \frac{1}{1 + \exp\left(-(\mathbf{w}^T \mathbf{x}_i + w_0)\right)} = \sigma(g(\mathbf{x}, \mathbf{w}, w_0))$$

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{\partial L}{\partial \sigma} \frac{\partial \sigma}{\partial g} \frac{\partial g}{\partial \mathbf{w}}$$

$$= \sum_i \left[-\frac{y_i}{h(\mathbf{x}_i)} + \frac{1-y_i}{1-h(\mathbf{x}_i)} \right] h(\mathbf{x}_i)(1-h(\mathbf{x}_i)) \mathbf{x}_i$$

链式法则

$$= \sum_i \left[-y_i(1-h(\mathbf{x}_i)) + (1-y_i)h(\mathbf{x}_i) \right] \mathbf{x}_i$$

$$= \sum_i (h(\mathbf{x}_i) - y_i) \mathbf{x}_i$$

$$\frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial \sigma} \frac{\partial \sigma}{\partial g} \frac{\partial g}{\partial w_0} = \sum_i (h(\mathbf{x}_i) - y_i)$$

然后基于梯度下降更新

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \partial L / \partial \mathbf{w}$$

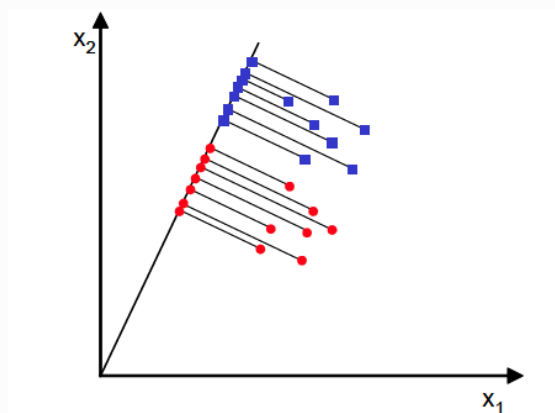
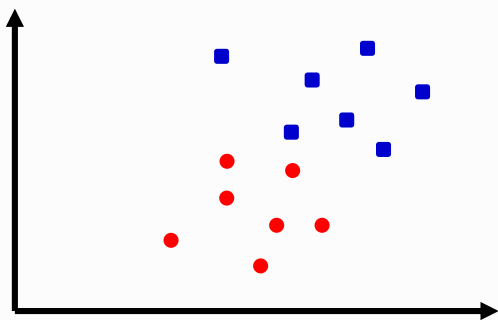
$$w_0 \leftarrow w_0 - \eta \partial L / \partial w_0$$

3.3 回归方法与线性分类器

3、Fisher线性分类器

Fisher线性分类器的基本思想：**通过寻找一个投影方向（线性变换，线性组合），将高维问题降低到一维问题来解决，并且要求变换后的一维数据具有如下性质：同类样本尽可能聚集在一起，不同类的样本尽可能地远。**

Fisher线性判别，即**通过给定的训练数据，确定投影方向 W 和阈值 y_0 ，即确定线性判别函数，然后根据这个线性判别函数，对测试数据进行测试，得到测试数据的类别。**



3.3 回归方法与线性分类器

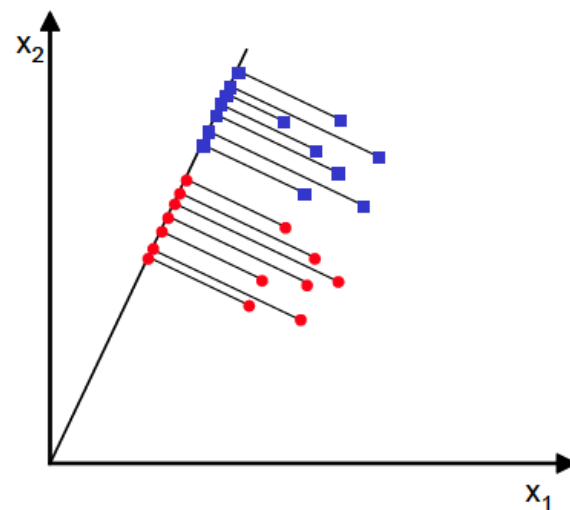
➤ Fisher线性判别 (LDA, Linear Discriminant Analysis)

- ◆ 假设有 n 个样本 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$, 对应标签为 $y_1, y_2, \dots, y_n$
- ◆ 其中 n_1 个属于类别 ω_1 , n_2 个属于类别 ω_2
- ◆ 获取投影向量 $\mathbf{w}$, 通过 $z = \mathbf{w}^T \mathbf{x}$ 将样本投影到一维空间, 并在该一维空间上设置阈值 $-w_0$, 使得

$$z \geq -w_0 \Rightarrow \text{判定为 } \omega_1 \text{ 类}$$

$$z < -w_0 \Rightarrow \text{判定为 } \omega_2 \text{ 类}$$

- ◆ 寻找使得类别之间区分度最大的投影向量



- ◆ **Fisher判别准则**: 最小化类别重叠, 使得不同类均值投影分开大, 而每个类的内部方差小。

3.3 回归方法与线性分类器

➤ Fisher线性判别算法步骤

◆ 假设有 n 个的样本 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$, 对应标签为 $y_1, y_2, \dots, y_n$

◆ 其中 n_1 个属于类别 ω_1 , n_2 个属于类别 ω_2

第1步：样本

◆ 定义类内散度矩阵 $S_i = \sum_{y_m \in \omega_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T$

◆ 定义总类内散度矩阵 $S_w = S_1 + S_2$

◆ 定义总类间散度矩阵 $S_B = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T$

◆ 定义准则函数

$$\max_{\mathbf{w}} J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_w \mathbf{w}}$$

思考如何定的?

第2步：损失函数

◆ 利用拉格朗日乘子法求解准则函数：

$$L(\mathbf{w}, \lambda) = \mathbf{w}^T S_B \mathbf{w} - \lambda(\mathbf{w}^T S_w \mathbf{w} - 1)$$

后面推导 J 取最大值的投影轴为：

$$\mathbf{w} = S_w^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$$

$$\boldsymbol{\mu}_i = \frac{1}{n} \sum_{X \in \omega_i} X, i=1,2$$

3.3 回归方法与线性分类器

$$\max_{\mathbf{w}} J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_w \mathbf{w}}$$

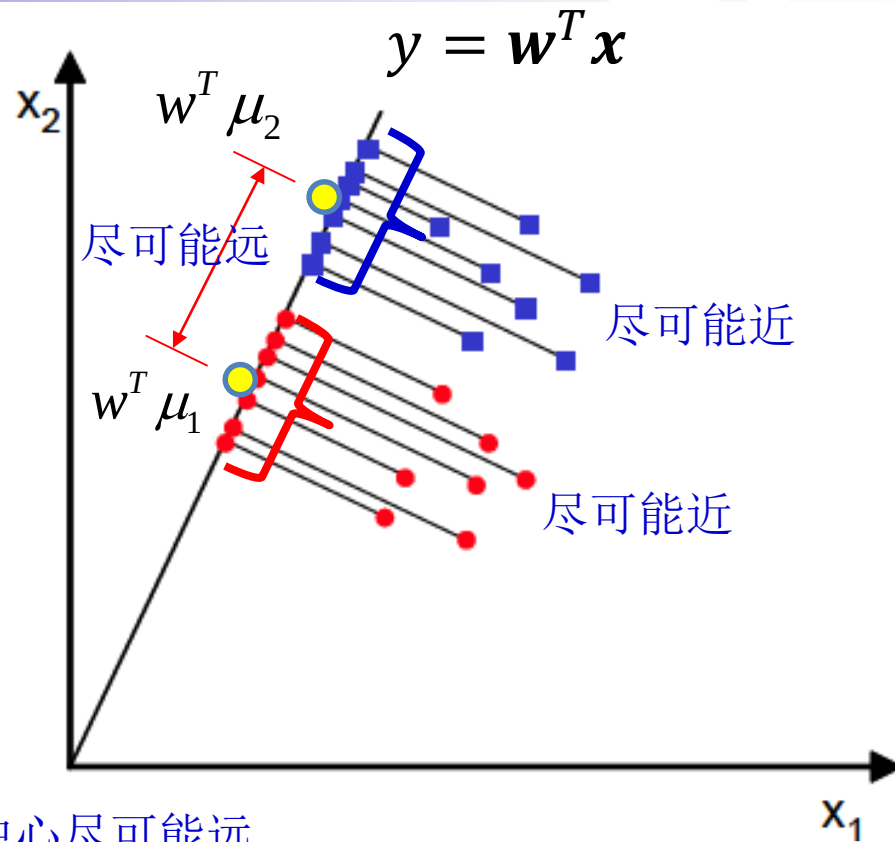
解释：此项如何计算出来

两类样本的中心在直线上的投影分别为

$$\mathbf{w}^T \mu_1 \quad \mathbf{w}^T \mu_2$$

两类样本投影的方差分别为

$$\mathbf{w}^T S_1 \mathbf{w} \quad \mathbf{w}^T S_2 \mathbf{w}$$



目标：同类样本投影点尽可能近，不同类中心尽可能远

$$\mathbf{w}^T S_1 \mathbf{w} + \mathbf{w}^T S_2 \mathbf{w} \text{ 小} \quad \|\mathbf{w}^T \mu_1 - \mathbf{w}^T \mu_2\|^2 \text{ 大}$$

$$\text{最大化目标} = \frac{\|\mathbf{w}^T \mu_1 - \mathbf{w}^T \mu_2\|^2}{\mathbf{w}^T S_1 \mathbf{w} + \mathbf{w}^T S_2 \mathbf{w}} \xrightarrow{\text{red arrow}} \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_w \mathbf{w}}$$

3.3 回归方法与线性分类器

➤ Fisher线性判别——拉格朗日乘子法求解

◆ 利用拉格朗日乘子法求解：

$$L(\mathbf{w}, \lambda) = \mathbf{w}^T S_B \mathbf{w} - \lambda (\mathbf{w}^T S_w \mathbf{w} - 1)$$

◆ 求导，可得

$$\frac{\partial L(\mathbf{w}, \lambda)}{\partial \mathbf{w}} = 2S_B \mathbf{w} - 2\lambda S_w \mathbf{w} = 0 \Rightarrow S_B \mathbf{w} = \lambda S_w \mathbf{w} \Rightarrow S_w^{-1} S_B \mathbf{w} = \lambda \mathbf{w}$$

广义特征值分解

第3步：优化方法

◆ 又因为 $S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$

◆ 投影向量 $\mathbf{w} = S_w^{-1}(\mu_1 - \mu_2)$ ⇐ 得到

◆ 投影阈值

两类均值的中点 $-w_0 = \frac{\tilde{\mu}_1 + \tilde{\mu}_2}{2}$

投影后数据的均值 $-w_0 = \frac{n_1 \tilde{\mu}_1 + n_2 \tilde{\mu}_2}{n_1 + n_2}$
 n_1, n_2 是两类样本个数

$$\mathbf{w} = \frac{S_w^{-1} S_B}{\lambda} \mathbf{w}$$

$$= \frac{1}{\lambda} S_w^{-1} (\mu_1 - \mu_2) (\mu_1 - \mu_2)^T \mathbf{w}$$

是标量

将 S_B 代入

$$\propto S_w^{-1} (\mu_1 - \mu_2)$$

3.3 回归方法与线性分类器

➤ Fisher线性判别——拉格朗日乘子法求解

- ◆ 利用拉格朗日乘子法求解：

$$L(\mathbf{w}, \lambda) = \mathbf{w}^T S_B \mathbf{w} - \lambda(\mathbf{w}^T S_w \mathbf{w} - 1)$$

- ◆ 求导，可得

$$\frac{\partial L(\mathbf{w}, \lambda)}{\partial \mathbf{w}} = 2S_B \mathbf{w} - 2\lambda S_w \mathbf{w} = 0 \Rightarrow \overset{\text{广义特征值分解}}{S_B \mathbf{w} = \lambda S_w \mathbf{w}} \Rightarrow S_w^{-1} S_B \mathbf{w} = \lambda \mathbf{w}$$

第3步：优化方法

- ◆ 又因为 $S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$

- ◆ 投影向量 $\mathbf{w} = S_w^{-1}(\mu_1 - \mu_2)$

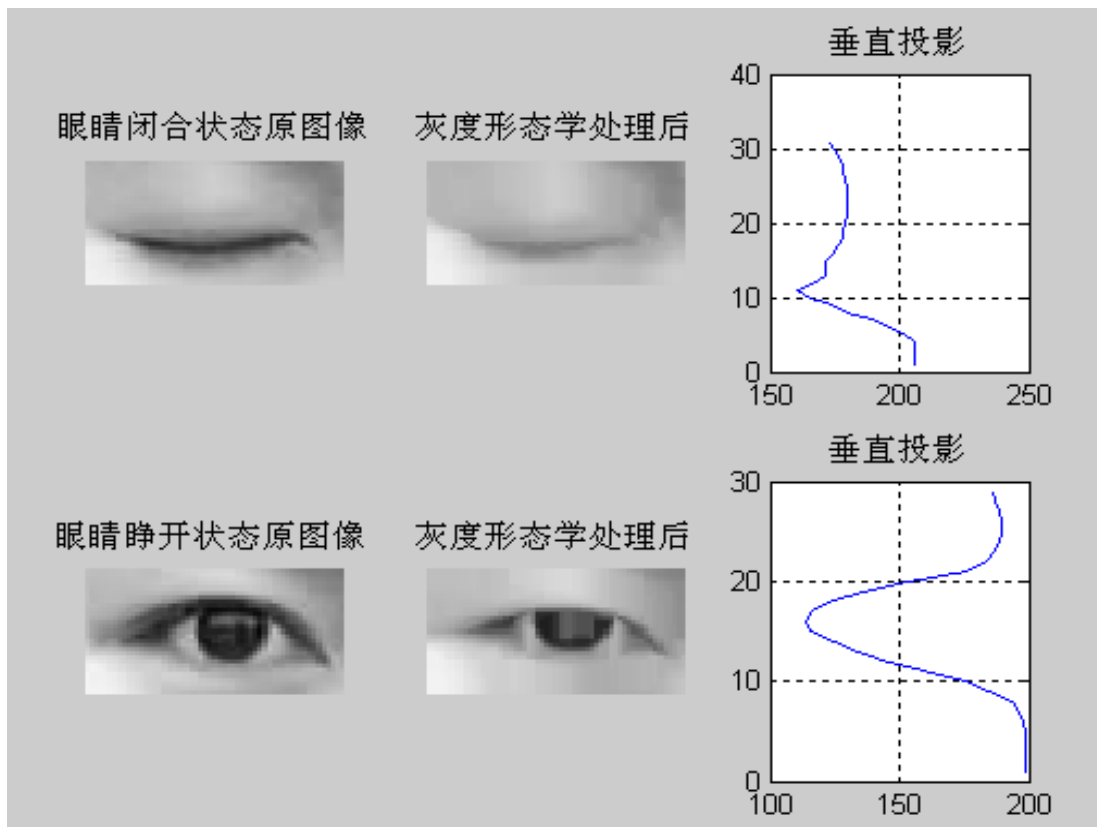
- ◆ 投影阈值
两类均值的中点 $-w_0 = \frac{\tilde{\mu}_1 + \tilde{\mu}_2}{2}$
投影后数据的均值 $-w_0 = \frac{n_1 \tilde{\mu}_1 + n_2 \tilde{\mu}_2}{n_1 + n_2}$
 $n_1 n_2$ 是两类样本个数

Q3: 对于Fisher线性判别而言，其对应的判别函数形式是什么？

A3: 可利用投影之后的两类均值的中心点作为阈值；可引入先验概率

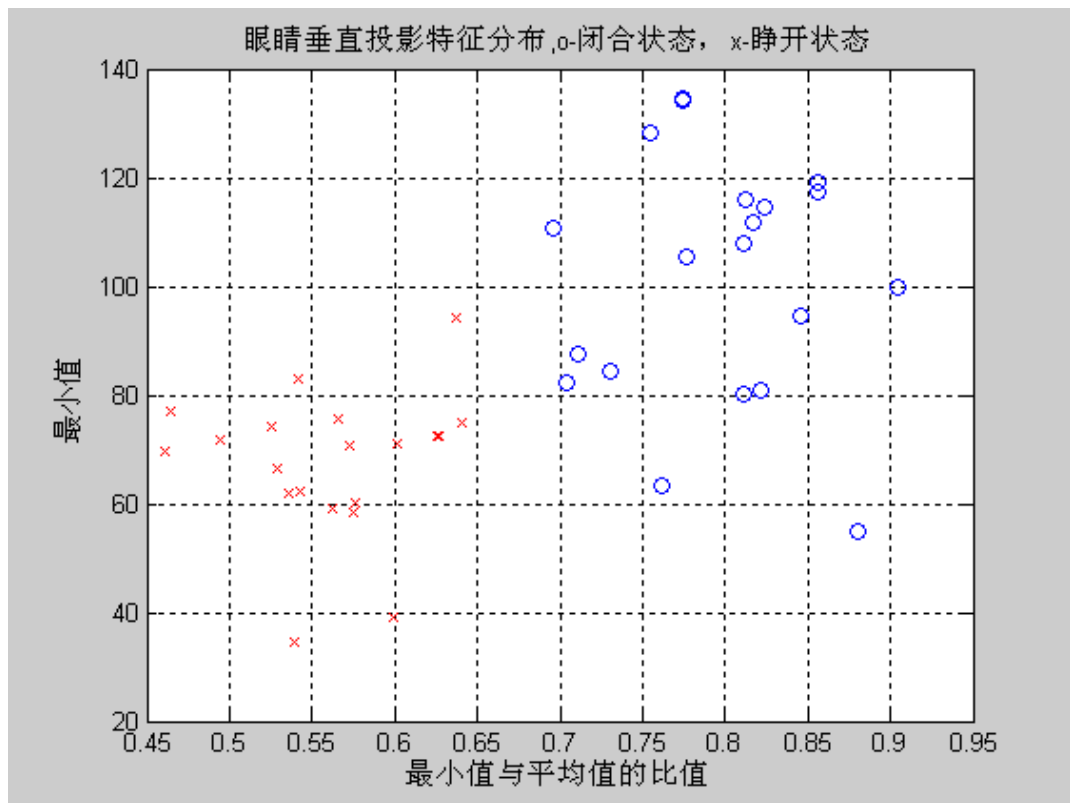
线性分类示例——眼睛开闭合识别

- 眼睛在睁闭两种状态下的垂直投影曲线的谷点值差别很大，因此可使用投影曲线的最小值作为特征。



线性分类示例——眼睛开闭合识别

- 由于光照及人肤色原因，眼睛区域投影的最小值分布具有较强的离散性，因此引入一个辅助判别特征：投影曲线的平均值，同最小值一起构成特征向量。



线性分类示例——眼睛开闭合识别，实验结果

- 采用中科院CAS-PEAL-R1人脸图像库，其中有1040幅睁眼图像，377幅闭眼图像。
- 选取96幅闭眼图像+99幅睁眼图像作为线性分类器的训练样本集；选取400幅睁眼图像+376幅闭眼图像作为测试：

辅助特征	眼睛状态	准确率	每对眼睛判别时间
最小值与平均值的比值	闭眼图像	95.92%	1.3686ms
	睁眼图像	92.24%	
最小值与最大值的比值	闭眼图像	95.92%	0.9258ms
	睁眼图像	91.58%	

from: Li Yali, Master course, EE., Tsinghua Univ.

在logistic regression中，使用了Sigmoid非线性映射(激活)函数，试结合其导数形式分析双曲

正切 $\text{Tanh}(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ 的导数形式为 ()

☒ A $\frac{4e^{-2z}}{(1+e^{-2z})^2}$

☐ B $\frac{e^{-z}}{(1+e^{-z})}$

☐ C $\frac{4e^{-2z}}{(1+e^{-2z})}$

☐ D $\frac{e^{-2z}}{(1+e^{-2z})}$

$$\text{Tanh}'(z) = \frac{4e^{-2z}}{(1+e^{-2z})^2} = 1 - \text{Tanh}(z)^2$$

在深度神经网络中，经常使用ReLU激活函数，其形式为 $f(z) = \max(0, z)$

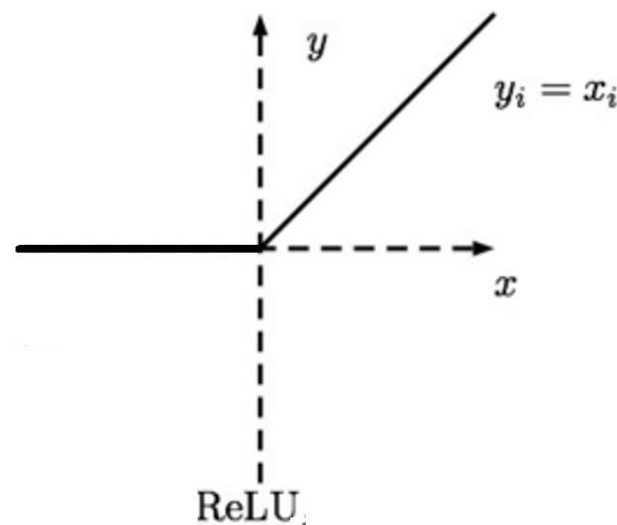
试问下面()可作为其在实际编程中的导数。

☒ A $f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$

☐ B $f'(z) = 1$

☐ C $f'(z) = 0$

☐ D 不存在



3.1 机器学习概要

3.2 感知机

3.3 回归方法与线性分类器

3.4 支持向量机SVM

请继续学习“机器学习-2”

x : 数据集

y : 真实标签 (实际值)

y_d : 当前标签 (标签值, 有可能是错误的, 因为收集过程可能出错)

$f(x; D)$: 在 D 模型中 x 的预测值

$f(x)$ 横: x 在某个模型中的期望值 (可以说是概率也可以说是平均值, 比如硬币正反面的期望是0.5, 不用管怎么来的, 就看作平均值即可)

求取平均值 (期望值): $\bar{f}(x) = \mathbb{E}_D[f(x; D)]$

噪声平方定义 (当前标签和实际标签差): $\xi^2 = \mathbb{E}_D[(y_d - y)^2]$

方差 (预测值和平均值的差): $var(x) = \mathbb{E}_D[(f(x; D) - \bar{f}(x))^2]$

偏差 (实际值和平均值的差): $bias^2(x) = (\bar{f}(x) - y)^2$

$$\begin{aligned} E(f; D) &= \mathbb{E}_D [(f(x; D) - y_D)^2] \\ &= \mathbb{E}_D [(f(x; D) - \bar{f}(x) + \bar{f}(x) - y_D)^2] \\ &= \mathbb{E}_D [(f(x; D) - \bar{f}(x))^2] + \mathbb{E}_D [(\bar{f}(x) - y_D)^2] \\ &\quad + \mathbb{E}_D [2(f(x; D) - \bar{f}(x))(\bar{f}(x) - y_D)] \\ &= \mathbb{E}_D [(f(x; D) - \bar{f}(x))^2] + \mathbb{E}_D [(\bar{f}(x) - y_D)^2] \\ &= \mathbb{E}_D [(f(x; D) - \bar{f}(x))^2] + \mathbb{E}_D [(\bar{f}(x) - y + y - y_D)^2] \\ &= \mathbb{E}_D [(f(x; D) - \bar{f}(x))^2] + \mathbb{E}_D [(\bar{f}(x) - y)^2] + \mathbb{E}_D [(y - y_D)^2] \\ &\quad + 2\mathbb{E}_D [(\bar{f}(x) - y)(y - y_D)] \\ &= \mathbb{E}_D [(f(x; D) - \bar{f}(x))^2] + (\bar{f}(x) - y)^2 + \mathbb{E}_D [(y_D - y)^2] , \end{aligned}$$

$$E_D(f; D) = bias^2(x) + var(x) + \varepsilon^2.$$

机器学习模型

$$E\left\{\left(y-h(x)\right)^2\right\}=\sigma^2+Var\left[h(x)\right]+\left(Bias\left[h(x)\right]\right)^2$$

误差的期望值

模型预测值的方差

预测值相对于真值的偏差的平方

$$\begin{aligned}E\left[\left(\hat{h}(x)+\varepsilon-h(x)\right)^2\right]&=E\left[\varepsilon^2+2\varepsilon\left(\hat{h}(x)-h(x)\right)+\left(\hat{h}(x)-h(x)\right)^2\right] \\&=\sigma^2+E\left[\left(\hat{h}(x)-h(x)\right)^2\right]=\sigma^2+E\left[\left(\hat{h}(x)-h(x)-E\left(h(x)-\hat{h}(x)\right)+E\left(h(x)-\hat{h}(x)\right)\right)^2\right] \\&=\sigma^2+E\left\{\left[\left(h(x)-\hat{h}(x)\right)-E\left(h(x)-\hat{h}(x)\right)\right]^2\right\}+E\left[2\left(\hat{h}(x)-h(x)-E\left(h(x)-\hat{h}(x)\right)\right)E\left(h(x)-\hat{h}(x)\right)\right]+E\left[\left(h(x)-\hat{h}(x)\right)^2\right] \\&=\sigma^2+E\left\{\left[\left(h(x)-\hat{h}(x)\right)-E\left(h(x)-\hat{h}(x)\right)\right]^2\right\}+E\left[\left(h(x)-\hat{h}(x)\right)^2\right]\end{aligned}$$

$$Var = E[(X-\mu)^2] = E[X^2-2X\mu+\mu^2] = E(X^2)-2\mu^2+\mu^2 = E(X^2)-\mu^2$$

展开后，利用方差等于平方的期望减去期望的平方，得到Var

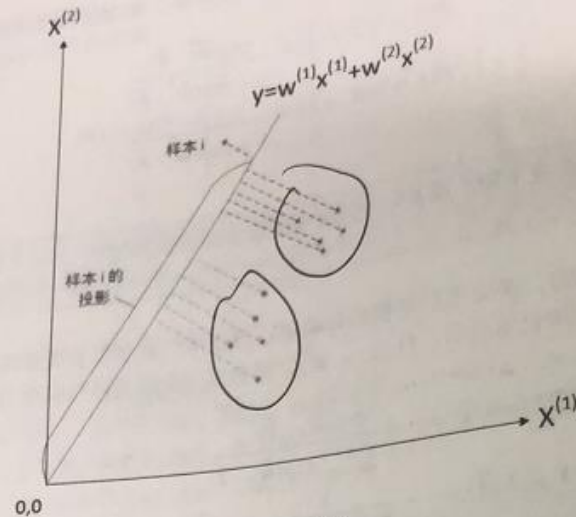


图 1.1 LDA

将数据投影到直线上，则：

- 两类样本的中心在直线上的投影分别为 $\bar{\mathbf{w}}^T \bar{\boldsymbol{\mu}}_0$ 和 $\bar{\mathbf{w}}^T \bar{\boldsymbol{\mu}}_1$ 。
- 两类样本投影的方差分别为 $\bar{\mathbf{w}}^T \Sigma_0 \bar{\mathbf{w}}$ 和 $\bar{\mathbf{w}}^T \Sigma_1 \bar{\mathbf{w}}$ 。

我们的目标是：同类样本的投影点尽可能地接近、异类样本的投影点尽可能地远离。那么可以使同类样例投影点的方差尽可能地小，即 $\bar{\mathbf{w}}^T \Sigma_0 \bar{\mathbf{w}} + \bar{\mathbf{w}}^T \Sigma_1 \bar{\mathbf{w}}$ 尽可能地小；可以使异类样例的中心的投影点尽可能地远，即 $\|\bar{\mathbf{w}}^T \bar{\boldsymbol{\mu}}_0 - \bar{\mathbf{w}}^T \bar{\boldsymbol{\mu}}_1\|_2^2$ 尽可能地大。于是得到最大化的目标：

$$J = \frac{\|\bar{\mathbf{w}}^T \bar{\boldsymbol{\mu}}_0 - \bar{\mathbf{w}}^T \bar{\boldsymbol{\mu}}_1\|_2^2}{\bar{\mathbf{w}}^T \Sigma_0 \bar{\mathbf{w}} + \bar{\mathbf{w}}^T \Sigma_1 \bar{\mathbf{w}}} = \frac{\bar{\mathbf{w}}^T (\bar{\boldsymbol{\mu}}_0 - \bar{\boldsymbol{\mu}}_1)(\bar{\boldsymbol{\mu}}_0 - \bar{\boldsymbol{\mu}}_1)^T \bar{\mathbf{w}}}{\bar{\mathbf{w}}^T (\Sigma_0 + \Sigma_1) \bar{\mathbf{w}}}$$

定义类内散度矩阵 within-class scatter matrix:

$$S_w = \Sigma_0 + \Sigma_1 = \sum_{\tilde{\mathbf{x}} \in T_0} (\tilde{\mathbf{x}} - \bar{\boldsymbol{\mu}}_0)(\tilde{\mathbf{x}} - \bar{\boldsymbol{\mu}}_0)^T + \sum_{\tilde{\mathbf{x}} \in T_1} (\tilde{\mathbf{x}} - \bar{\boldsymbol{\mu}}_1)(\tilde{\mathbf{x}} - \bar{\boldsymbol{\mu}}_1)^T$$

定义类间散度矩阵 between-class scatter matrix: $S_b = (\bar{\boldsymbol{\mu}}_0 - \bar{\boldsymbol{\mu}}_1)(\bar{\boldsymbol{\mu}}_0 - \bar{\boldsymbol{\mu}}_1)^T$ ，它是向量 $(\bar{\boldsymbol{\mu}}_0 - \bar{\boldsymbol{\mu}}_1)$ 与它自身的外积，则LDA最大化的目标为：

$$J = \frac{\bar{\mathbf{w}}^T S_b \bar{\mathbf{w}}}{\bar{\mathbf{w}}^T S_w \bar{\mathbf{w}}}$$

J 也称为 S_b 与 S_w 的广义瑞利商。现在求解最优化问题:

$$\tilde{\mathbf{w}}^* = \arg \max_{\tilde{\mathbf{w}}} \frac{\tilde{\mathbf{w}}^T S_b \tilde{\mathbf{w}}}{\tilde{\mathbf{w}}^T S_w \tilde{\mathbf{w}}}$$

由于分子与分母都是关于 $\tilde{\mathbf{w}}$ 的二次项, 因此上式的解与 $\tilde{\mathbf{w}}$ 的长度无关。令 $\tilde{\mathbf{w}}^T S_w \tilde{\mathbf{w}} = 1$, 则最优化问题改写为:

$$\begin{aligned} \tilde{\mathbf{w}}^* &= \arg \min_{\tilde{\mathbf{w}}} -\tilde{\mathbf{w}}^T S_b \tilde{\mathbf{w}} \\ \text{s.t. } &\tilde{\mathbf{w}}^T S_w \tilde{\mathbf{w}} = 1 \end{aligned}$$

应用拉格朗日乘法:

$$S_b \tilde{\mathbf{w}} = \lambda S_w \tilde{\mathbf{w}}$$

令 $(\tilde{\mu}_0 - \tilde{\mu}_1)^T \tilde{\mathbf{w}} = \lambda_{\tilde{\mathbf{w}}}$, 其中 $\lambda_{\tilde{\mathbf{w}}}$ 为实数。则 $S_b \tilde{\mathbf{w}} = (\tilde{\mu}_0 - \tilde{\mu}_1)(\tilde{\mu}_0 - \tilde{\mu}_1)^T \tilde{\mathbf{w}} = \lambda_{\tilde{\mathbf{w}}}(\tilde{\mu}_0 - \tilde{\mu}_1)$ 。

$S_b \tilde{\mathbf{w}} = \lambda_{\tilde{\mathbf{w}}}(\tilde{\mu}_0 - \tilde{\mu}_1) = \lambda S_w \tilde{\mathbf{w}}$ 。由于与 $\tilde{\mathbf{w}}$ 的长度无关, 可以令 $\lambda_{\tilde{\mathbf{w}}} = \lambda$, 则有:

$$(\tilde{\mu}_0 - \tilde{\mu}_1) = S_w \tilde{\mathbf{w}} \implies \tilde{\mathbf{w}} = S_w^{-1}(\tilde{\mu}_0 - \tilde{\mu}_1)$$

上述讨论的是二类分类LDA算法。可以将它推广到多分类任务中: 假定存在 M 个类, 属于第 i 个类的样本的集合为 T_i , T_i 中的样例数为 m_i , 则有: $\sum_{i=1}^M m_i = N$, 其中 N 为样本总数。设 T_i 表示类别为 i , $i = 1, 2, \dots, M$ 的样例的集合, 这些样例的均值向量为:

$$\tilde{\mu}_i = (\mu_i^{(1)}, \mu_i^{(2)}, \dots, \mu_i^{(n)})^T = \frac{1}{m_i} \sum_{\tilde{\mathbf{x}}_i \in T_i} \tilde{\mathbf{x}}_i$$

这些样例的特征之间协方差矩阵为 Σ_i (协方差矩阵大小为 $n \times n$)。定义 $\tilde{\mu} = (\mu^{(1)}, \mu^{(2)}, \dots, \mu^{(n)})^T = \frac{1}{N} \sum_{i=1}^N \tilde{\mathbf{x}}_i$ 是所有样例的均值向量。

- 要使得同类样例的投影点尽可能地接近, 则可以使同类样例投影点的方差尽可能地小, 因此定义类别的类内散度矩阵为 $S_{wi} = \sum_{\tilde{\mathbf{x}} \in T_i} (\tilde{\mathbf{x}} - \tilde{\mu}_i)(\tilde{\mathbf{x}} - \tilde{\mu}_i)^T$; 定义类内散度矩阵为 $S_w = \sum_{i=1}^M S_{wi}$ 。



类别的类内散度矩阵为 $S_{wi} = \sum_{\tilde{\mathbf{x}} \in T_i} (\tilde{\mathbf{x}} - \tilde{\mu}_i)(\tilde{\mathbf{x}} - \tilde{\mu}_i)^T$, 实际上就等于样本集 T_i 的协方差矩阵 Σ_i 。

- 要使异类样例的投影点尽可能地远, 则可以使异类样例中心的投影点尽可能地远, 由于这里不止两个中心点, 因此不能简单地套用二类LDA的做法 (即两个中心点的距离)。这里用每一类样本集和的中心点距总的中心点的距离作为度量。考虑到每一类样本集的大小可能不同 (密度分布不均), 故我们对这个距离加以权重, 因此定义类间散度矩阵 $S_b = \sum_{i=1}^M m_i (\tilde{\mu}_i - \tilde{\mu})(\tilde{\mu}_i - \tilde{\mu})^T$ 。g.csdn.net/gwplovekimi



$(\vec{\mu}_i - \vec{\mu})(\vec{\mu}_i - \vec{\mu})^T$ 也是一个协方差矩阵，它刻画的是第 i 类与总体之间的关系。

设 $W \in \mathbb{R}^{n \times (M-1)}$ 是投影矩阵。经过推导可以得到最大化的目标：

$$J = \frac{\text{tr}(W^T S_b W)}{\text{tr}(W^T S_w W)}$$

其中 $\text{tr}(\cdot)$ 表示矩阵的迹。一个矩阵的迹是矩阵对角线的元素之和，它是一个矩阵不变量，也等于所有特征值之和。



还有一个常用的矩阵不变量就是矩阵的行列式，它等于矩阵的所有特征值之积。

多分类 LDA 将样本投影到 $M-1$ 维空间，因此它是一种经典的监督降维技术。之所以叫监督是因为对于每个训练样本，我们知道它所属的类别。